INTERVIEW PREPARATION GUIDE

150 Full-Stack Interview Q&A

Theory · Strong Answers · Example Code · Architecture Diagrams

150	10	150	8
Questions	Sections	Diagrams	Languages

Prepared for Mohammed Rinshad M I
Full-Stack Engineer · React · Next.js · Node.js · Golang · AI/LLM · 2026

How to Use This Guide

This guide turns your resume into **150 realistic, interview-ready questions** across the full stack you actually work in — JavaScript & TypeScript, React/Next.js, Node.js, Golang, databases, real-time, AI/LLM features, payments and DevOps. Every question is answered in four layers so you understand it, not just memorise it:

THEORY	The concept and the *why* — what the interviewer is really probing for.
ANSWER	A natural, confident spoken answer you can deliver out loud. **Bold** terms are keywords to land.
CODE	A concise, correct example — what good looks like in real code.
ARCHITECTURE	An ASCII diagram of the flow or system, so you can draw it on a whiteboard.

PRO TIP

Read the **answers out loud**. Interviews reward fluent delivery, not perfect recall. When you give a number from your resume (35% latency cut, 30% faster delivery), have a one-line 'how I measured it' ready for the follow-up.

Contents

1 JavaScript & TypeScript Core

Closures, this, event loop, promises, types, generics, utility types, JS to TS.

Q1–Q15

Q1 JAVASCRIPT & TYPESCRIPT CORE

What is the difference between var, let, and const, and what is the Temporal Dead Zone?

THEORY

Probes understanding of scoping rules, hoisting behavior, and reassignment semantics. The interviewer wants to know if you understand block scoping and why TDZ exists.

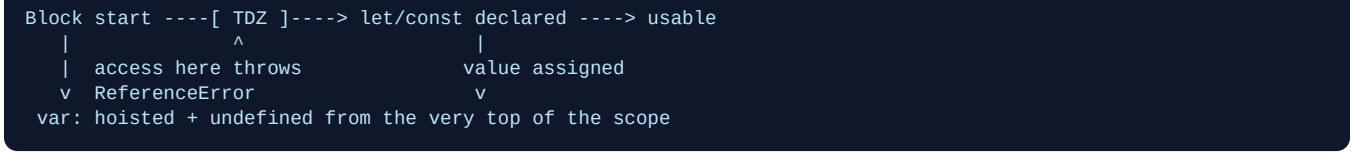
STRONG ANSWER

The key difference is scope and mutability. **var** is function-scoped and gets hoisted with an initial value of undefined, which causes subtle bugs. **let** and **const** are **block-scoped**, and while they're technically hoisted, you can't access them before declaration. That window between the start of the block and the declaration is the **Temporal Dead Zone**, and touching the variable there throws a ReferenceError. In practice I default to **const**, reach for **let** only when I need to reassign, and avoid **var** entirely.

CODE · JavaScript

```
console.log(a); // undefined (var hoisted)
var a = 1;

console.log(b); // ReferenceError: in the TDZ
let b = 2;

const c = 3;
c = 4; // TypeError: Assignment to constant variable

function scope() {
  if (true) { var x = 'fn-scoped'; let y = 'block'; }
  console.log(x); // 'fn-scoped'
  console.log(y); // ReferenceError: y is not defined
}
```

ARCHITECTURE / FLOW



Q2 JAVASCRIPT & TYPESCRIPT CORE

What is a closure and can you give a real-world use case?

THEORY

Tests grasp of lexical scope and how functions retain access to their defining environment. Interviewers look for a concrete, practical application, not just the textbook definition.

STRONG ANSWER

A **closure** is when a function remembers and keeps access to the **variables from its lexical scope** even after the outer function has returned. The inner function holds a live reference to those variables, not a copy. A real use case I rely on is **data privacy** and factory functions, like a counter or a memoization cache where state stays hidden behind the returned function. I also use closures constantly in React via **hooks**, where event handlers close over props and state from the render they were created in.

CODE · JavaScript

```
function createCounter() {
  let count = 0; // private, captured by closure
  return {
    increment: () => ++count,
    get: () => count,
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.get()); // 2
console.log(counter.count);  // undefined (truly private)
```

ARCHITECTURE / FLOW

```
createCounter() scope
+--------------------------+
| count = 0                |  <--+ both functions keep
|                          |     | a live reference
|  increment() ----------------+ even after
|  get() ----------------------+ createCounter returns
+--------------------------+
```

Q3 JAVASCRIPT & TYPESCRIPT CORE

How does `this` work in JavaScript, and how do arrow functions change it?

THEORY

Examines understanding of dynamic binding rules and the lexical `this` of arrow functions. The interviewer probes whether you know the four binding rules and common pitfalls.

STRONG ANSWER

In a regular function, **this** is determined by how the function is called, not where it's defined. The rules in order are **new binding**, **explicit binding** with call/apply/bind, **implicit binding** from the object before the dot, and otherwise the **default** which is undefined in strict mode. **Arrow functions** don't have their own this; they capture it **lexically** from the surrounding scope at definition time. That's why arrows are great for callbacks inside methods, but a bad choice for object methods or anything you'd call with new.

CODE · JavaScript

```
const obj = {
  name: 'Ada',
  regular() { return this.name; },        // implicit -> 'Ada'
  arrow: () => this?.name,                 // lexical -> undefined
};
console.log(obj.regular()); // 'Ada'

const loose = obj.regular;
```

CODE · JavaScript (continued)

```
console.log(loose());                     // undefined (default)
console.log(loose.call({ name: 'Lin' }));  // 'Lin' (explicit)

function Timer() {
  this.secs = 0;
  setInterval(() => this.secs++, 1000);    // arrow keeps `this`
}
```

ARCHITECTURE / FLOW

```
How was it called?
  new Fn()        --> this = new instance
  fn.call(o)      --> this = o (explicit)
  obj.fn()        --> this = obj (implicit)
  fn()            --> this = undefined (strict)

Arrow ()=>{}      --> ignores all above, copies enclosing this
```

Q4 JAVASCRIPT & TYPESCRIPT CORE

Explain the event loop, including the call stack and the microtask vs macrotask queues.

THEORY

Assesses understanding of JavaScript's concurrency model and async execution order. Interviewers want to see you know microtasks drain before the next macrotask.

STRONG ANSWER

JavaScript is **single-threaded**, so the **event loop** coordinates async work. Synchronous code runs on the **call stack** until it's empty. Then the loop drains the entire **microtask queue**, which holds **Promise** callbacks and queueMicrotask, before it picks up a single **macrotask** like a setTimeout or I/O callback. The crucial rule is that microtasks always run to completion between each macrotask, so a resolved promise fires before a zero-delay timer. Knowing this explains tricky ordering bugs and why heavy microtask loops can starve rendering.

CODE · JavaScript

```
console.log('1: sync');

setTimeout(() => console.log('4: macrotask'), 0);

Promise.resolve().then(() => console.log('3: microtask'));

console.log('2: sync');

// Output order:
// 1: sync
// 2: sync
// 3: microtask   (queue drained before timers)
// 4: macrotask
```

ARCHITECTURE / FLOW

```
  Call Stack (runs sync code)
        | empty?
        v
  Microtask Queue  --> drain ALL (promises)
        | empty?
        v
  Macrotask Queue  --> take ONE (setTimeout, I/O)
        |
        +--> loop back to drain microtasks again
```

Q5 JAVASCRIPT & TYPESCRIPT CORE

What is the difference between Promises and async/await, and how do you handle errors in each?

THEORY

Tests fluency with asynchronous patterns and robust error handling. The interviewer probes whether you understand async/await is syntactic sugar over Promises.

STRONG ANSWER

Promises represent a future value with .then and .catch chaining, while **async/await** is **syntactic sugar** on top of promises that lets you write asynchronous code in a synchronous-looking style. With promises you handle errors using **.catch** at the end of the chain; with async/await you wrap awaited calls in **try/catch**. An async function always returns a promise, and a thrown error becomes a rejected promise. For independent operations I use **Promise.all** to run them concurrently instead of awaiting sequentially, which is a common performance win.

CODE · JavaScript

```javascript
// Promise style
fetch('/api/user')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error('failed', err));

// async/await style
async function loadUser() {
  try {
    const res = await fetch('/api/user');
    return await res.json();
  } catch (err) {
    console.error('failed', err);
    throw err;
  }
}

// concurrent, not sequential
const [a, b] = await Promise.all([fetch('/a'), fetch('/b')]);
```

ARCHITECTURE / FLOW

```
Promise:  call --> .then --> .then --> .catch (errors)
                                          ^
                                          |
async/await:  try {                       |
                await call  --- rejects ---+
              } catch (err) { handle }

async fn always returns a Promise (resolve/reject)
```

Q6 JAVASCRIPT & TYPESCRIPT CORE

How does hoisting work for variables and functions?

THEORY

Probes understanding of the two-phase nature of execution: creation and execution. Interviewers check if you know declarations vs initializations and the function declaration vs expression distinction.

STRONG ANSWER

During the **creation phase**, the engine scans the scope and **hoists declarations** to the top before any code runs. **Function declarations** are hoisted completely, so you can call them before they appear in the source. **var** is hoisted but initialized to **undefined**, so the value isn't there yet. **let** and **const** are hoisted too but stay in the **Temporal Dead Zone** until their line executes. Function expressions and arrow functions follow their variable's rules, so they aren't callable before assignment.

CODE · JavaScript

```
greet(); // 'hello' -- function declaration fully hoisted
function greet() { console.log('hello'); }

console.log(x); // undefined -- var declaration hoisted
var x = 5;

try { sayHi(); } catch (e) { console.log(e.name); } // TypeError
var sayHi = function () { console.log('hi'); };
// sayHi is undefined at call time, so it's not a function
```

ARCHITECTURE / FLOW

```
Source order            After hoisting (creation phase)
-------------           -------------------------------
greet()                 function greet(){...}  (whole fn)
function greet(){}      var x = undefined
x                       var sayHi = undefined
var x = 5               -- run code top to bottom --
var sayHi = fn
```

Q7 JAVASCRIPT & TYPESCRIPT CORE

Explain prototypal inheritance and the prototype chain.

THEORY

Tests understanding of JavaScript's object model versus classical inheritance. The interviewer wants to see you understand property lookup delegation up the chain.

STRONG ANSWER

JavaScript uses **prototypal inheritance**, where objects delegate to other objects instead of copying from classes. Every object has an internal link, accessible via **Object.getPrototypeOf**, pointing to its **prototype**. When you access a property, the engine walks up the **prototype chain** until it finds the property or hits null. **Class** syntax is just **syntactic sugar** over this mechanism; methods live on the prototype so all instances share one copy. This is why adding a method to a constructor's prototype instantly makes it available to every existing instance.

CODE · JavaScript

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function () {
  return `${this.name} makes a sound`;
};

function Dog(name) { Animal.call(this, name); }
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;
Dog.prototype.speak = function () { return `${this.name} barks`; };

const d = new Dog('Rex');
console.log(d.speak());           // 'Rex barks'
```

CODE · JavaScript (continued)

```
console.log(d.hasOwnProperty('name')); // true (own), speak is inherited
```

ARCHITECTURE / FLOW

```
d (instance)
  | __proto__
  v
Dog.prototype  { speak }
  | __proto__
  v
Animal.prototype { speak }
  | __proto__
  v
Object.prototype { hasOwnProperty, toString }
  | __proto__
  v
null  (lookup stops here)
```

Q8 JAVASCRIPT & TYPESCRIPT CORE

What is the difference between == and ===, and how does type coercion work?

THEORY

Probes awareness of coercion rules and why strict equality is preferred. Interviewers look for knowledge of the pitfalls of loose equality.

STRONG ANSWER

Triple equals is **strict equality**: it compares value and type with no conversion. **Double equals** is **loose equality**: if the types differ it applies **type coercion** before comparing, following a set of rules that are often surprising. For example, an empty string equals zero loosely because both coerce to falsy numbers. I default to **strict equality** everywhere to avoid those surprises, with one common idiom being a loose `== null` check to catch both null and undefined at once.

CODE · JavaScript

```
0 == '';        // true  (both coerce to 0)
0 == '0';       // true
'' == '0';      // false
null == undefined; // true
NaN === NaN;    // false (use Number.isNaN)
[] == false;    // true  ([] -> '' -> 0)

0 === '';       // false (different types, no coercion)

if (value == null) { /* catches null AND undefined */ }
```

ARCHITECTURE / FLOW

```
x === y :  type(x) !== type(y) ? --> false
           else compare values directly

x ==  y :  same type?  --> compare directly
           number vs string --> string -> number
           boolean involved --> boolean -> number
           null == undefined --> true (special case)
```

Q9 JAVASCRIPT & TYPESCRIPT CORE

What is the difference between debounce and throttle, and can you implement them?

THEORY

Tests practical knowledge of rate-limiting techniques using closures and timers. Interviewers want to know when to use each and see a clean implementation.

STRONG ANSWER

Both limit how often a function runs, but the timing differs. **Debounce** waits until activity **stops** for a set delay before firing once, which is perfect for a **search input** or resize handler. **Throttle** guarantees the function runs at most once per interval **during** continuous activity, which suits **scroll** or mouse-move handlers. The implementation difference is that debounce **resets the timer** on every call, while throttle **ignores calls** until the cooldown elapses. Both rely on **closures** to hold the timer reference between invocations.

CODE · JavaScript

```javascript
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

function throttle(fn, interval) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= interval) {
      last = now;
      fn(...args);
    }
  };
}
```

ARCHITECTURE / FLOW

```
calls:    x x x x x        x
debounce: ...........|fire   ....|fire
          (fires only after calls stop for delay)

calls:    x x x x x x x x x x x x
throttle: |fire ... |fire ... |fire
          (fires once per fixed interval)
```

Q10 JAVASCRIPT & TYPESCRIPT CORE

How do map, filter, and reduce work, and how do they relate to immutability?

THEORY

Assesses functional programming fluency and understanding of non-mutating array methods. Interviewers probe whether you favor immutable transformations.

STRONG ANSWER

These are **higher-order** array methods that return **new arrays** instead of mutating the original. **map** transforms each element one-to-one, **filter** keeps elements that pass a predicate, and **reduce** folds the array into a single accumulated value. Because they don't mutate, they're a natural fit for **immutability**, which matters a lot in **React** where state must be replaced rather than changed in place. I chain them for readable pipelines and reach for reduce when I need to build up an object or aggregate.

CODE · JavaScript

```
const orders = [
  { id: 1, total: 30, paid: true },
  { id: 2, total: 20, paid: false },
  { id: 3, total: 50, paid: true },
];

const revenue = orders
  .filter(o => o.paid)          // [#1, #3]
  .map(o => o.total)            // [30, 50]
  .reduce((sum, t) => sum + t, 0); // 80

// immutable update -- original untouched
const next = orders.map(o =>
  o.id === 2 ? { ...o, paid: true } : o
);
```

ARCHITECTURE / FLOW

```
[ o1, o2, o3 ]
     | filter(paid)
     v
[ o1, o3 ]
     | map(=> total)
     v
[ 30, 50 ]
     | reduce(+)
     v
  80      (original array never mutated)
```

Q11 JAVASCRIPT & TYPESCRIPT CORE

In TypeScript, what is the difference between a type alias and an interface?

THEORY

Tests knowledge of TypeScript's two main ways to describe object shapes and when to use each. Interviewers probe understanding of declaration merging and extensibility.

STRONG ANSWER

Both can describe the shape of an object, and for most cases they're interchangeable. The key differences are that **interface** supports **declaration merging**, so two declarations with the same name combine, and it's the idiomatic choice for public object contracts that may be extended. A **type alias** is more flexible: it can represent **unions**, **intersections**, tuples, mapped types, and primitives that interfaces can't express. My rule of thumb is **interfaces for object shapes**, especially in libraries, and **type aliases** when I need unions or more advanced type composition.

CODE · TypeScript

```
interface User { id: number; name: string; }
interface User { email: string; } // merges into one User

type ID = string | number;          // union -- only type can do this
```

CODE · TypeScript (continued)

```ts
type Point = { x: number; y: number };
type Point3D = Point & { z: number }; // intersection

// both can extend
interface Admin extends User { role: string; }
type AdminT = User & { role: string };
```

ARCHITECTURE / FLOW

```
interface              type alias
---------              ----------
object shapes  yes     object shapes  yes
decl. merging  yes     decl. merging  no
extends        yes     unions/&       yes
unions         no      primitives     yes

rule: interface for shapes, type for unions/composition
```

Q12 JAVASCRIPT & TYPESCRIPT CORE

What are generics in TypeScript and why do they matter?

THEORY

Probes whether the candidate can write reusable, type-safe abstractions. Interviewers want to see understanding of preserving type information across functions.

STRONG ANSWER

Generics let you write components and functions that work over **multiple types** while **preserving the type relationship** between input and output. Instead of resorting to `any` and losing safety, a type parameter like `<T>` flows through so the compiler still knows the exact type at the call site. They matter because they give you **reusability without sacrificing type safety**, which is essential for utilities, data structures, and API wrappers. You can also add **constraints** with `extends` to require a type to have certain properties.

CODE · TypeScript

```ts
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const n = first([1, 2, 3]);     // n: number
const s = first(['a', 'b']);    // s: string

// constrained generic
function prop<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
const name = prop({ id: 1, name: 'Ada' }, 'name'); // string

interface ApiResponse<T> { data: T; status: number; }
```

ARCHITECTURE / FLOW

```
without generics            with generics
----------------            -------------
first(arr: any[]): any      first<T>(arr: T[]): T
      |                           |
      v                           v
 return type = any          return type = T (preserved)
  (lost safety)              callsite knows exact type
```

Q13 JAVASCRIPT & TYPESCRIPT CORE

Explain the utility types Partial, Pick, Omit, and Record.

THEORY

Tests familiarity with built-in type transformations that reduce boilerplate. Interviewers check if you can manipulate existing types instead of redefining them.

STRONG ANSWER

These are **built-in mapped types** that transform existing types so you don't repeat yourself. **Partial<T>** makes every property optional, which is perfect for update payloads. **Pick<T, K>** keeps only the named keys, and **Omit<T, K>** is the inverse, dropping the named keys. **Record<K, V>** builds an object type with a fixed set of keys all mapped to the same value type, great for lookups and dictionaries. Composing these keeps types **DRY** and tightly coupled to a single source of truth.

CODE · TypeScript

```typescript
interface User {
  id: number;
  name: string;
  email: string;
}

type UserUpdate = Partial<User>;        // all optional
type UserPreview = Pick<User, 'id' | 'name'>; // { id, name }
type UserNoEmail = Omit<User, 'email'>;       // { id, name }

type RolePerms = Record<'admin' | 'user', boolean>;
// { admin: boolean; user: boolean }
```

ARCHITECTURE / FLOW

```
User { id; name; email }
  |
  +-- Partial -> { id?; name?; email? }
  +-- Pick<'id'|'name'> -> { id; name }
  +-- Omit<'email'>     -> { id; name }

Record<'a'|'b', number> -> { a: number; b: number }
```

Q14 JAVASCRIPT & TYPESCRIPT CORE

Explain union and intersection types, narrowing, and unknown versus any.

THEORY

Probes understanding of composing types and safely working with uncertain values. Interviewers want to see you prefer unknown over any and know how to narrow.

STRONG ANSWER

A **union** means a value is one of several types, while an **intersection** combines multiple types into one that has all their members. With unions you use **narrowing**, things like typeof checks, truthiness, or discriminated unions, so the compiler knows the exact type inside a branch. The big distinction is **unknown vs any**: **any** disables type checking entirely and is unsafe, whereas **unknown** is the type-safe top type that **forces you to narrow** before you can use it. I prefer **unknown** at boundaries like API responses, then validate and narrow down to a concrete type.

CODE · TypeScript

```ts
type Shape =
  | { kind: 'circle'; r: number }
  | { kind: 'square'; side: number };

function area(s: Shape): number {
  switch (s.kind) {              // discriminated union narrowing
    case 'circle': return Math.PI * s.r ** 2;
    case 'square': return s.side ** 2;
  }
}

function handle(input: unknown) {
  // input.length; // error -- must narrow first
  if (typeof input === 'string') return input.length; // ok
}
```

ARCHITECTURE / FLOW

```
Union  A | B   : value is A OR B  --> narrow to use
Inter  A & B   : value is A AND B (has all members)

any     : anything goes, NO checks   (unsafe)
unknown : top type, MUST narrow first (safe)
          unknown --typeof/guard--> concrete type
```

Q15 JAVASCRIPT & TYPESCRIPT CORE

How would you migrate a large JavaScript codebase to TypeScript?

THEORY

Tests real-world migration strategy and pragmatism. Interviewers want an incremental plan that keeps the app shipping, tied to hands-on experience.

STRONG ANSWER

I approach it **incrementally** rather than as a big-bang rewrite. First I enable TypeScript with **allowJs** and **checkJs** so JS and TS coexist, then turn on **strict mode** gradually, often starting with noImplicitAny off and tightening over time. I migrate **file by file**, usually starting at the **leaf modules** and shared utilities and types, since they have the fewest dependencies and give the biggest safety payoff. For third-party gaps I pull in **@types** packages or write minimal declaration files, and I lean on **CI** to prevent regressions. In my own JS-to-TS migration this let us ship continuously while steadily driving the any count down to near zero.

CODE · TypeScript

```ts
// tsconfig.json -- coexistence + gradual tightening
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": false,
    "strict": false,
    "noImplicitAny": false,
    "outDir": "dist"
  }
}

// 1. rename leaf util.js -> util.ts, add types
// 2. flip noImplicitAny -> true, fix errors
// 3. enable strict once the bulk is typed
// 4. CI: tsc --noEmit gate on every PR
```

ARCHITECTURE / FLOW

```
Phase 0  allowJs:true  (JS + TS coexist, no breakage)
   |
   v
Phase 1  type leaf modules / utils / shared types
   |
   v
Phase 2  noImplicitAny:true, migrate file by file
   |
   v
Phase 3  strict:true everywhere + tsc CI gate
   |
   v
Done     any-count -> ~0, full type safety
```

2 React & Next.js

Components, hooks, JSX, RSC, App Router, rendering, code-splitting, performance.

Q16–Q30

Q16 REACT & NEXT.JS

What is the Virtual DOM and how does reconciliation work?

THEORY

Tests whether you understand React's rendering model and why it can be efficient. The interviewer wants to know you grasp diffing, not that the Virtual DOM is magically fast.

STRONG ANSWER

The **Virtual DOM** is a lightweight JavaScript representation of the actual DOM tree. When state changes, React builds a new virtual tree and runs **reconciliation**, a **diffing** algorithm that compares it against the previous tree to find the minimal set of changes. It then **batches** those mutations and applies them to the real DOM, which is the expensive part. React assumes elements of different types produce different trees, and it uses **keys** to match children across renders. The key insight is that the Virtual DOM isn't faster than the DOM, it just minimizes the slow DOM operations.

CODE · TypeScript / TSX

```tsx
// React builds a tree of these objects, not real DOM nodes
const vnode = {
  type: 'div',
  props: { className: 'card' },
  children: [{ type: 'h1', props: {}, children: ['Hi'] }]
};

// On update React diffs old vs new vnode tree,
// then patches only what changed in the real DOM.
```

ARCHITECTURE / FLOW

```
State change
     |
     v
  Render -> New Virtual Tree
     |
     v
  Diff (old tree vs new tree)
     |
     v
  Minimal patch set -> Real DOM
```

Q17 REACT & NEXT.JS

How does useState work and what actually triggers a re-render?

THEORY

Probes your understanding of state as the trigger for rendering and the immutability requirement. Interviewers watch for the mutation-vs-new-reference mistake.

STRONG ANSWER

useState returns the current value and a **setter**. Calling the setter schedules a re-render by marking the component as dirty, then React re-runs the function component to produce new output. A re-render only happens when you pass a **new reference** or value, React uses **Object.is** to bail out if the value is unchanged. That's why you must never mutate state in place, you create a new object or array. Updates are also **batched** within event handlers, and you should use the **functional updater** form when the next state depends on the previous one.

CODE · TypeScript / TSX

```tsx
const [count, setCount] = useState(0);

// Wrong: relies on stale closure value
setCount(count + 1);
setCount(count + 1); // still ends at 1

// Right: functional updater, batches correctly
setCount(c => c + 1);
setCount(c => c + 1); // ends at 2

// Objects: new reference, never mutate
setUser(prev => ({ ...prev, name: 'Ada' }));
```

ARCHITECTURE / FLOW

```
setState(value)
     |
     v
  Object.is(old, new)? --yes--> bail out, no render
     |
     no
     v
  mark dirty -> schedule render -> re-run component
```

Q18 REACT & NEXT.JS

Explain the useEffect dependency array and cleanup function.

THEORY

Checks that you understand when effects run and how to avoid leaks and stale state. The dependency array and cleanup are the two biggest sources of bugs.

STRONG ANSWER

useEffect runs after the render is committed to the screen. The **dependency array** controls when it re-runs, an empty array means once on mount, no array means every render, and listed deps mean it runs whenever any of them changes. The **cleanup function** you return runs before the next effect and on unmount, which is where you cancel subscriptions, timers, or fetches to prevent **memory leaks**. The most common bug is omitting a dependency, which gives you a **stale closure** over old values. In React 18 Strict Mode effects run twice in development to surface missing cleanup.

CODE · TypeScript / TSX

```tsx
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/user/${id}`, { signal: controller.signal })
    .then(r => r.json())
    .then(setUser);

  // cleanup: abort if id changes or component unmounts
```

CODE · TypeScript / TSX (continued)

```
  return () => controller.abort();
}, [id]); // re-runs only when id changes
```

ARCHITECTURE / FLOW

```
Mount --> run effect
  |
  v
dep changes --> cleanup(old) --> run effect(new)
  |
  v
Unmount --> cleanup() (cancel timers / subs / fetch)
```

Q19 REACT & NEXT.JS

What is the difference between useMemo and useCallback?

THEORY

Tests memoization fundamentals and when each tool applies. Interviewers want to hear that one memoizes a value and the other a function, plus that they're optimizations, not correctness tools.

STRONG ANSWER

useMemo caches the **result** of an expensive computation and only recomputes when its dependencies change. **useCallback** caches a **function reference** so it stays stable across renders, useCallback(fn, deps) is essentially useMemo(() => fn, deps). You reach for them to avoid recomputing heavy work or to keep a **stable reference** that won't break a **React.memo** child or a downstream useEffect dependency. They aren't free, they have their own memory and comparison cost, so overusing them can hurt. With the **React Compiler** much of this manual memoization becomes unnecessary.

CODE · TypeScript / TSX

```
// useMemo: cache a computed value
const sorted = useMemo(
  () => items.slice().sort((a, b) => a.price - b.price),
  [items]
);

// useCallback: cache a function identity
const onSelect = useCallback(
  (id: string) => setSelected(id),
  []
);
// <MemoChild onSelect={onSelect} /> won't re-render needlessly
```

ARCHITECTURE / FLOW

```
useMemo  -> caches a VALUE      (heavy compute)
useCallback -> caches a FUNCTION (stable identity)

   deps unchanged --> return cached
   deps changed   --> recompute / new fn
```

Q20 REACT & NEXT.JS

Why do keys matter when rendering lists in React?

THEORY

Probes reconciliation at the list level. The interviewer wants to know you understand keys as identity, not just a way to silence a warning.

STRONG ANSWER

Keys give React a stable **identity** for each list item so it can match elements between renders during **reconciliation**. With good keys React can move, insert, or remove just the changed items instead of re-rendering the whole list. Using the array **index** as a key is dangerous when the list can reorder, insert, or delete, because the identity shifts and React reuses the wrong DOM node, which corrupts component state like input values. You should use a **stable unique id** from your data. Keys must be unique among siblings, not globally.

CODE · TypeScript / TSX

```tsx
// Bad: index breaks on reorder/insert, state attaches to wrong row
{todos.map((t, i) => <Todo key={i} todo={t} />)}

// Good: stable id survives reordering
{todos.map(t => <Todo key={t.id} todo={t} />)}

// Reordering with stable keys: React moves nodes,
// preserving each row's internal state (e.g. an open menu).
```

ARCHITECTURE / FLOW

```
Items: [A, B, C]  keys: id-1 id-2 id-3
Insert X at front: [X, A, B, C]

index keys: every row shifts -> 4 re-renders, state misaligns
id keys:    only X is new   -> 1 insert, A/B/C untouched
```

Q21 REACT & NEXT.JS

What is the difference between controlled and uncontrolled components?

THEORY

Tests form-handling knowledge. Interviewers want to know you can choose the right approach and understand where the source of truth lives.

STRONG ANSWER

In a **controlled component** React state is the single **source of truth**, the input's value comes from state and every keystroke flows through an onChange handler. In an **uncontrolled component** the DOM holds the value and you read it when needed, typically through a **ref** or on submit. Controlled inputs give you instant validation, formatting, and conditional logic, at the cost of a render per keystroke. Uncontrolled inputs are lighter and play well with native forms or libraries like react-hook-form. I default to controlled for interactive forms and uncontrolled for simple or performance-sensitive cases.

CODE · TypeScript / TSX

```tsx
// Controlled: state drives the input
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />

// Uncontrolled: DOM owns the value, read via ref
const ref = useRef<HTMLInputElement>(null);
<input ref={ref} defaultValue="" />
<button onClick={() => console.log(ref.current?.value)}>Save</button>
```

ARCHITECTURE / FLOW

```
Controlled:
  state --value--> input
  input --onChange--> setState  (React = source of truth)

Uncontrolled:
  DOM holds value
  ref.current.value  (read on demand; DOM = source of truth)
```

Q22 REACT & NEXT.JS

How do you build a custom hook, for example useDebounce?

THEORY

Tests whether you can extract and reuse stateful logic cleanly. The interviewer is checking your grasp of hook composition and effect cleanup.

STRONG ANSWER

A **custom hook** is just a function starting with **use** that calls other hooks to package reusable stateful logic. For **useDebounce** I hold the debounced value in state and use a **useEffect** with a timer that updates it after a delay. The crucial part is the **cleanup**, returning a function that clears the previous timeout so rapid changes reset the timer instead of firing many updates. This keeps the logic testable and shareable across components, like debouncing a search input. The hook returns the debounced value, and the component re-renders only when it settles.

CODE · TypeScript

```ts
function useDebounce<T>(value: T, delay = 300): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id); // reset on each change
  }, [value, delay]);
  return debounced;
}

// Usage
const q = useDebounce(search, 400);
useEffect(() => { fetchResults(q); }, [q]);
```

ARCHITECTURE / FLOW

```
type fast:  s -> se -> sea -> sear -> searc -> search
              (each keystroke resets the timer)
                          |
                  400ms quiet window
                          v
debounced value updates once -> one fetch('search')
```

Q23 REACT & NEXT.JS

What is the Context API and when should you NOT use it?

THEORY

Probes state-management judgment. Interviewers want to know you understand Context's re-render behavior and that it isn't a Redux replacement for everything.

STRONG ANSWER

Context lets you pass values down the tree without **prop drilling**, ideal for low-frequency global data like theme, locale, or the current user. The catch is that when a context value changes, every **consumer** re-renders, so it's a poor fit for high-frequency or rapidly changing state. I avoid it for server state, that belongs in a data layer like **React Query**, and for complex client state with frequent updates, where a store like **Zustand** with selective subscriptions performs better. A common pitfall is passing a new object literal as the value each render, which forces all consumers to re-render. Split contexts by concern and memoize the value.

CODE · TypeScript / TSX

```tsx
const ThemeContext = createContext<Theme>('light');

function App() {
  const [theme, setTheme] = useState<Theme>('light');
  // memoize so consumers don't re-render on unrelated updates
  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return (
    <ThemeContext.Provider value={value}>
      <Page />
    </ThemeContext.Provider>
  );
}
```

ARCHITECTURE / FLOW

```
Good fit: theme, locale, auth user  (changes rarely)
Bad fit:  form keystrokes, server data, cursor pos

Provider value changes
     |
     v
ALL consumers re-render  <-- the cost to watch
```

Q24 REACT & NEXT.JS

How does React.memo work and how do you prevent unnecessary re-renders?

THEORY

Tests performance optimization knowledge. The interviewer wants to confirm you know memo does a shallow props comparison and how to make it actually stick.

STRONG ANSWER

React.memo wraps a component so it skips re-rendering when its props are **shallowly equal** to the previous render. It only helps if the props are stable, so it pairs with **useCallback** for function props and **useMemo** for object or array props, otherwise a fresh reference each render defeats it. A child also re-renders whenever its parent does unless memoized, so memo is the tool to cut that chain. For custom comparison you can pass a second **areEqual** function. I treat it as a targeted fix after profiling with the **React DevTools Profiler**, not a blanket wrap on every component.

CODE · TypeScript / TSX

```tsx
const Row = React.memo(function Row({ item, onPick }: Props) {
  return <li onClick={() => onPick(item.id)}>{item.name}</li>;
});

function List({ items }: { items: Item[] }) {
  // stable identity so React.memo can actually skip
  const onPick = useCallback((id: string) => select(id), []);
  return <ul>{items.map(i => <Row key={i.id} item={i} onPick={onPick} />)}</ul>;
}
```

ARCHITECTURE / FLOW

```
Parent re-renders
     |
     v
Child memo? --no--> child re-renders
     |
     yes
     v
props shallow-equal? --yes--> SKIP
                     --no---> re-render
(stable props need useCallback / useMemo)
```

Q25 REACT & NEXT.JS

What is useRef and what are its main use cases?

THEORY

Checks that you understand a mutable container that survives renders without triggering them. Interviewers watch for confusion between refs and state.

STRONG ANSWER

useRef returns a mutable object with a **.current** property that persists across renders and, crucially, mutating it does **not** trigger a re-render. The classic use is grabbing a **DOM node**, for focusing an input, measuring size, or integrating a non-React library. It's also great for holding mutable values you don't want to render on, like a **timer id**, a previous value, or a flag to skip the first effect run. The mental rule is, use state when the UI should update, use a ref when you need to remember something without re-rendering. Reading or writing a ref during render is discouraged.

CODE · TypeScript / TSX

```tsx
function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);
  const renders = useRef(0);
  renders.current++; // persists, but does NOT cause a render

  useEffect(() => { inputRef.current?.focus(); }, []);
  return <input ref={inputRef} placeholder="Search..." />;
}
```

ARCHITECTURE / FLOW

```
useState  -> change triggers re-render (UI value)
useRef    -> change is silent, no re-render (escape hatch)

Use cases:
  - DOM access  (focus, measure, scroll)
  - timer / interval ids
  - previous value / skip-first-render flag
```

Q26 REACT & NEXT.JS

What is the difference between the Next.js App Router and Pages Router?

THEORY

Tests current Next.js knowledge. Interviewers want to see you understand the architectural shift to Server Components and nested layouts, not just the folder name.

STRONG ANSWER

The **Pages Router** lives in /pages, is client-component based, and fetches data with **getServerSideProps** or **getStaticProps** at the page level. The **App Router** lives in /app and is built on **React Server Components** by default, with file conventions like layout, page, loading, and error. It brings **nested layouts** that preserve state, **streaming** with Suspense, and colocated data fetching directly in async server components. The App Router is the recommended default for new projects and unlocks things like Server Actions. The Pages Router is still supported, and the two can coexist during migration.

CODE · TypeScript / TSX

```tsx
// App Router: app/dashboard/page.tsx (Server Component by default)
export default async function Page() {
  const data = await getData(); // fetch right in the component
  return <Dashboard data={data} />;
}

// Pages Router: pages/dashboard.tsx
export async function getServerSideProps() {
  return { props: { data: await getData() } };
}
export default function Page({ data }) { return <Dashboard data={data} />; }
```

ARCHITECTURE / FLOW

```
Pages Router (/pages)        App Router (/app)
  client components            server components default
  getServerSideProps           async/await in component
  _app / _document             layout.tsx (nested)
  page-level data              streaming + Suspense
```

Q27 REACT & NEXT.JS

What is the difference between React Server Components and Client Components?

THEORY

Probes the core mental model of the App Router. The interviewer wants the boundary rules and the trade-offs, including what each can and cannot do.

STRONG ANSWER

Server Components render on the server, can be **async**, talk directly to databases or secrets, and ship **zero JavaScript** to the client, which shrinks the bundle. **Client Components**, marked with the **'use client'** directive, run in the browser and are the only ones that can use **state**, **effects**, or browser APIs and event handlers. By default everything in the App Router is a Server Component, and you opt into client behavior only at the interactive leaves. A key rule is that you can pass server-rendered components as **children** into client components, but props crossing the boundary must be **serializable**. The pattern is to keep client components small and push them to the edges of the tree.

CODE · TypeScript / TSX

```tsx
// Server Component (default) - runs on server, no JS shipped
export default async function Page() {
  const posts = await db.posts.findMany();
  return <LikeButton>{posts.length}</LikeButton>; // pass server UI in
}

// Client Component - interactivity lives here
'use client';
export function LikeButton({ children }: { children: React.ReactNode }) {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>{children} {n}</button>;
}
```

ARCHITECTURE / FLOW

```
          Server Component (default)
          - async, DB/secret access
          - ships 0 JS
              |
   'use client' boundary
              v
          Client Component (leaf)
          - useState/useEffect, onClick
          - browser APIs
```

Q28 REACT & NEXT.JS

Explain the difference between SSR, SSG, and ISR.

THEORY

Tests rendering-strategy judgment. Interviewers want to know you can match a strategy to content freshness and traffic needs.

STRONG ANSWER

SSG, static generation, renders pages at **build time** into static HTML, fastest to serve and ideal for content that rarely changes like docs or marketing pages. **SSR**, server-side rendering, builds the HTML **per request**, which guarantees fresh data and supports personalization but adds server cost and latency. **ISR**, incremental static regeneration, is the hybrid, you serve static pages but **revalidate** them on a time interval or on demand, so a product page can refresh every sixty seconds without a full rebuild. In the App Router these map to caching and **revalidate** options on fetch rather than separate functions. I pick SSG for static content, ISR for semi-dynamic content at scale, and SSR for truly per-request or user-specific data.

CODE · TypeScript

```typescript
// App Router controls strategy via fetch caching / revalidate

// SSG-like: cached at build, served static
await fetch(url, { cache: 'force-cache' });

// ISR: static but regenerates every 60s
await fetch(url, { next: { revalidate: 60 } });

// SSR: fresh on every request
await fetch(url, { cache: 'no-store' });
```

ARCHITECTURE / FLOW

```
SSG : build time ----> HTML          (fastest, may be stale)
ISR : build + revalidate(60s) ---> refresh in background
SSR : per request ----> HTML         (always fresh, slower)

   freshness  -->   SSG < ISR < SSR
   speed/cost -->   SSG > ISR > SSR
```

Q29 REACT & NEXT.JS

How does data fetching and fetch caching work in the Next.js App Router?

THEORY

Tests practical knowledge of the App Router's extended fetch and caching layers. The interviewer wants to hear about request memoization, the data cache, and revalidation.

STRONG ANSWER

In the App Router you fetch data directly inside async **Server Components** using the extended **fetch** API. Next adds **request memoization**, so the same fetch in one render pass is deduplicated automatically, and a persistent **Data Cache** across requests controlled by the **cache** and **next.revalidate** options. You revalidate by time with **revalidate**, or on demand with **revalidateTag** and **revalidatePath** after a mutation, which is great with **Server Actions**. Opting out with **no-store** makes a route dynamic and always fresh. Note that in Next 15 fetch defaults moved toward being uncached, so I'm explicit about caching intent rather than relying on defaults.

CODE · TypeScript

```typescript
// Tagged fetch -> can be invalidated on demand
async function getProducts() {
  const res = await fetch('https://api/products', {
    next: { tags: ['products'], revalidate: 3600 },
  });
  return res.json();
}

// In a Server Action after a mutation:
'use server';
import { revalidateTag } from 'next/cache';
export async function addProduct(/* ... */) {
  // ...write...
  revalidateTag('products'); // refresh only what's affected
}
```

ARCHITECTURE / FLOW

```
fetch() in Server Component
   |
   +-- Request Memoization  (dedupe within one render)
   |
   +-- Data Cache  (persist across requests)
         |
         revalidate: 60   -> time-based
         revalidateTag()  -> on-demand after mutation
         cache:'no-store' -> always fresh (dynamic)
```

Q30 REACT & NEXT.JS

How do code-splitting and lazy loading work, and how did you raise a Lighthouse score from 62 to 89?

THEORY

Ties optimization theory to real results. The interviewer wants concrete techniques and the metrics they moved, not vague claims.

STRONG ANSWER

Code-splitting breaks the bundle into chunks so the browser only downloads what a route or interaction needs, and **lazy loading** defers non-critical components with **dynamic import** and Suspense. On a project I took **Lighthouse from 62 to 89** by first profiling the bundle, then dynamically importing heavy below-the-fold widgets like charts and a rich text editor, which cut the initial JavaScript and improved **TBT** and **LCP**. I moved interactive pieces to small client components so most of the tree stayed as zero-JS **Server Components**, added **next/image** for responsive images and **next/font** to kill layout shift, and prefetched routes with the Link component. The biggest wins were trimming **TBT** by deferring third-party scripts and reducing **CLS** with reserved image dimensions.

CODE · TypeScript / TSX

```tsx
import dynamic from 'next/dynamic';

// Heavy, below-the-fold: split out and skip SSR
const Chart = dynamic(() => import('./Chart'), {
  loading: () => <Skeleton />,
  ssr: false,
});

// next/image: responsive + reserved space (better LCP & CLS)
import Image from 'next/image';
<Image src="/hero.jpg" width={1200} height={600} priority alt="Hero" />;
```

ARCHITECTURE / FLOW

```
Before: one big bundle -> slow TBT/LCP   Lighthouse 62

  split routes + dynamic import (charts, editor)
  next/image (LCP, CLS)  +  next/font (CLS)
  defer 3rd-party scripts (TBT)
              |
              v
After: small initial JS, lazy chunks       Lighthouse 89
```

3 State Management & Data Fetching

Context, Redux Toolkit, thunks, selectors, Zustand, TanStack Query caching.

Q31–Q45

Q31 STATE MANAGEMENT & DATA FETCHING

When should you use local component state vs global state?

THEORY

Local state lives in a single component via useState/useReducer; global state is shared across many components through a store or context. Reach for global only when state is genuinely shared.

STRONG ANSWER

Default to **local state** and only lift it up when a sibling or distant component needs the same data. Reach for **global state** when many unrelated components read or mutate the same value, like the **authenticated user**, **theme**, or a shopping cart. A common mistake is putting everything in a global store, which causes unnecessary re-renders and tight coupling. Another key distinction is **server state** (data from an API) which is better handled by a query library than a global store. Keep transient UI state like input values, toggles, and hover local.

CODE · TypeScript / TSX

```tsx
// Local: only this component cares
function SearchBox() {
  const [query, setQuery] = useState('');
  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}

// Global: shared auth used across the app
const user = useAuthStore(s => s.user);
if (!user) return <Login />;
```

ARCHITECTURE / FLOW

```
Decision: where does state belong?

  Used by ONE component? ......... useState (local)
  Shared by a few nearby? ........ lift up / props
  Shared app-wide (auth/theme)? .. global store
  Comes from a server/API? ....... React Query
```

Q32 STATE MANAGEMENT & DATA FETCHING

Explain Redux core concepts: store, action, reducer, dispatch.

THEORY

Redux is a predictable state container built on a single source of truth and pure functions. State changes only through dispatched actions handled by reducers.

STRONG ANSWER

The **store** holds the entire application state as a single immutable object tree. An **action** is a plain object with a `type` describing what happened, plus optional payload. A **reducer** is a pure function `(state, action) => newState` that computes the next state without mutating the old one. You trigger changes by calling **dispatch(action)**, which runs the action through the reducers. The strict unidirectional flow makes state changes traceable and enables time-travel debugging.

CODE · TypeScript

```
const initial = { count: 0 };

function reducer(state = initial, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    default:
      return state;
  }
}

const store = createStore(reducer);
store.dispatch({ type: 'INCREMENT' }); // count -> 1
store.getState(); // { count: 1 }
```

ARCHITECTURE / FLOW

```
Unidirectional data flow:

  UI event
     |
  dispatch(action) --> reducer(state, action)
     ^                        |
     |                    new state
  re-render <---------- store updates
     |
  selectors read state
```

Q33 STATE MANAGEMENT & DATA FETCHING

How does Redux Toolkit's createSlice reduce boilerplate?

THEORY

Redux Toolkit (RTK) is the official, recommended way to write Redux. createSlice generates action creators and the reducer together from one config object.

STRONG ANSWER

createSlice takes a name, initial state, and a `reducers` map, and auto-generates the **action creators** and **action types** for you, so you no longer write switch statements or constants by hand. It uses **Immer** under the hood, letting you write "mutating" code like `state.value++` that is actually applied immutably. RTK's **configureStore** sets up the store with good defaults including the Redux DevTools and thunk middleware. This collapses the classic actions/constants/reducers file trio into a single concise slice.

CODE · TypeScript

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },        // Immer
```

CODE · TypeScript (continued)

```
    addBy: (state, action) => { state.value += action.payload; },
  },
});

export const { increment, addBy } = counterSlice.actions;
export default counterSlice.reducer;

const store = configureStore({ reducer: { counter: counterSlice.reducer } });
```

ARCHITECTURE / FLOW

```
Classic Redux            vs   createSlice

  actions.ts                      one slice file:
  constants.ts          ->          - name
  reducers.ts                       - initialState
  (manual switch)                   - reducers {}
                                  (actions auto-made)
```

Q34 STATE MANAGEMENT & DATA FETCHING

How do you handle async logic with createAsyncThunk?

THEORY

createAsyncThunk wraps an async function and auto-dispatches lifecycle actions for the pending, fulfilled, and rejected states of a promise.

STRONG ANSWER

createAsyncThunk takes a type prefix and an async payload creator, returning a thunk you can dispatch. It automatically dispatches three actions: **pending** when it starts, **fulfilled** with the resolved value, and **rejected** with the error. You handle these in the slice's **extraReducers** using the builder callback to update loading flags and data. This gives you a consistent pattern for tracking request status without manually dispatching start/success/failure actions.

CODE · TypeScript

```
export const fetchUser = createAsyncThunk('user/fetch', async (id: string) => {
  const res = await fetch(`/api/users/${id}`);
  return res.json();
});

const userSlice = createSlice({
  name: 'user',
  initialState: { data: null, status: 'idle' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUser.pending,   (s) => { s.status = 'loading'; })
      .addCase(fetchUser.fulfilled, (s, a) => { s.status = 'done'; s.data = a.payload; })
      .addCase(fetchUser.rejected,  (s) => { s.status = 'error'; });
  },
});
```

ARCHITECTURE / FLOW

```
dispatch(fetchUser(id))
        |
        v
  [pending]  -> status = loading
        |
```

ARCHITECTURE / FLOW (continued)

```
   promise resolves?
     /        \
   yes         no
    |           |
[fulfilled] [rejected]
 data set    error set
```

Q35 STATE MANAGEMENT & DATA FETCHING

What are selectors and memoized selectors (reselect / createSelector)?

THEORY

A selector is a function that reads a slice of state. Memoized selectors cache derived results so they only recompute when their inputs change.

STRONG ANSWER

A plain **selector** is just `(state) => state.cart.items`, keeping store-shape knowledge in one place. When you **derive** data, like filtering or summing, recomputing on every render and returning a new array reference causes wasted re-renders. **createSelector** from Reselect memoizes: it takes input selectors plus a combiner, and only re-runs the combiner when an input's reference changes. This keeps expensive computations and stable references, which is important because new object references break React's referential equality checks.

CODE · TypeScript

```ts
import { createSelector } from '@reduxjs/toolkit';

const selectItems = (state: RootState) => state.cart.items;
const selectFilter = (state: RootState) => state.cart.filter;

export const selectVisible = createSelector(
  [selectItems, selectFilter],
  (items, filter) => items.filter(i => i.category === filter) // memoized
);

// Usage: recomputes only when items or filter change
const visible = useSelector(selectVisible);
```

ARCHITECTURE / FLOW

```
createSelector([inputA, inputB], combiner)

  inputs same ref? --> yes --> return CACHED result
         |
        no
         |
     run combiner --> cache new result + inputs
```

Q36 STATE MANAGEMENT & DATA FETCHING

What is Redux middleware and how does it work?

THEORY

Middleware sits between dispatching an action and the moment it reaches the reducer, letting you intercept, delay, transform, or log actions.

STRONG ANSWER

Middleware is a function with the curried signature **store => next => action**. Each middleware can inspect the action, then call **next(action)** to pass it along the chain, or stop it entirely. This is how async tools like **thunk** work: they let you dispatch a function instead of a plain object, which the middleware executes. Middleware is composed into a pipeline, so logging, crash reporting, and async handling can stack cleanly. RTK adds the thunk middleware by default via configureStore.

CODE · TypeScript

```
const logger = (store) => (next) => (action) => {
  console.log('dispatching', action.type);
  const result = next(action); // pass to next middleware / reducer
  console.log('next state', store.getState());
  return result;
};

const store = configureStore({
  reducer,
  middleware: (getDefault) => getDefault().concat(logger),
});
```

ARCHITECTURE / FLOW

```
dispatch(action)
      |
      v
  middleware A -> next -> middleware B -> next
                                            |
                                            v
                                        reducer
                                            |
                                       store update

  Each can log / delay / block / transform.
```

Q37 STATE MANAGEMENT & DATA FETCHING

Show the basics of creating a Zustand store.

THEORY

Zustand is a minimal hook-based state library. You call create with a function that returns state and the actions that update it via set.

STRONG ANSWER

create returns a hook that you call to read state, passing a **selector** so the component only re-renders when that slice changes. Inside the store function you receive **set** (and optionally **get**) to update state; set merges shallowly by default. There's no provider, no reducers, and no action types, so it's far less boilerplate than Redux. You define actions right alongside state, colocating logic with the data it touches.

CODE · TypeScript

```
import { create } from 'zustand';

interface CounterState {
  count: number;
  increment: () => void;
  reset: () => void;
}
```

CODE · TypeScript (continued)

```ts
const useCounter = create<CounterState>((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
  reset: () => set({ count: 0 }),
}));

// In a component (selector avoids extra re-renders):
const count = useCounter((s) => s.count);
const increment = useCounter((s) => s.increment);
```

ARCHITECTURE / FLOW

```
create((set, get) => ({ ...state, ...actions }))
        |
   returns useStore hook
        |
  useStore(s => s.count)  // subscribe to a slice
        |
  set(...) --> notifies only subscribers of changed slice
```

Q38 STATE MANAGEMENT & DATA FETCHING

Zustand vs Redux Toolkit — what are the tradeoffs?

THEORY

Both manage global client state, but differ in ceremony, structure, and ecosystem. The choice often comes down to scale and team conventions.

STRONG ANSWER

Zustand is minimal: no providers, tiny API, and very little boilerplate, which makes it fast to adopt for small-to-medium apps. **Redux Toolkit** brings more structure, a powerful **DevTools** time-travel experience, a large middleware ecosystem, and well-established patterns that scale across large teams. Redux's strict action/reducer flow gives auditability, while Zustand's freedom can lead to inconsistent patterns if undisciplined. For most new projects Zustand is enough, but RTK shines in large, long-lived codebases needing strict conventions and traceability.

CODE · TypeScript

```ts
// Zustand: action colocated, no dispatch
const useStore = create((set) => ({
  todos: [],
  add: (t) => set((s) => ({ todos: [...s.todos, t] })),
}));
useStore.getState().add({ id: 1 });

// RTK: explicit slice + dispatch
const slice = createSlice({
  name: 'todos', initialState: [],
  reducers: { add: (s, a) => { s.push(a.payload); } },
});
dispatch(slice.actions.add({ id: 1 }));
```

ARCHITECTURE / FLOW

```
              Zustand        Redux Toolkit
  Boilerplate  very low       moderate
  Provider     none           required
  DevTools     basic          excellent (time-travel)
  Structure    freeform       enforced patterns
  Best for     small/medium   large teams/apps
```

Q39 STATE MANAGEMENT & DATA FETCHING

How do you choose between Context, Redux, and Zustand?

THEORY

Each tool fits a different need: Context for low-frequency shared values, Zustand/Redux for frequently updated app state. They are not mutually exclusive.

STRONG ANSWER

Context is best for low-frequency, rarely-changing values like **theme**, **locale**, or the current user, since every consumer re-renders when the context value changes. For frequently updated or complex client state, a dedicated store like **Zustand** or **Redux** avoids that re-render problem through selector-based subscriptions. Pick **Zustand** for minimal ceremony and **Redux Toolkit** when you need strict structure, middleware, and time-travel DevTools at scale. Crucially, for **server state** none of these is ideal; reach for React Query instead.

CODE · TypeScript / TSX

```tsx
// Context: fine for a rarely-changing theme
const ThemeContext = createContext('light');
<ThemeContext.Provider value={theme}>{children}</ThemeContext.Provider>;

// Zustand/Redux: frequently-updated state with fine-grained subscriptions
const price = useCartStore((s) => s.totalPrice); // only re-renders on price change

// React Query: server data (not the job of the above)
const { data } = useQuery({ queryKey: ['products'], queryFn: getProducts });
```

ARCHITECTURE / FLOW

```
Pick by use case:

  Rarely changes, app-wide?   -> Context
  Frequent client state, min? -> Zustand
  Frequent + strict + large?  -> Redux Toolkit
  Data from a server/API?     -> React Query
```

Q40 STATE MANAGEMENT & DATA FETCHING

Explain TanStack Query: queries and the cache.

THEORY

TanStack (React) Query manages asynchronous server state. useQuery fetches and caches data keyed by a query key, handling loading and error states for you.

STRONG ANSWER

useQuery takes a **queryKey** (a serializable array identifying the data) and a **queryFn** that returns a promise. The result exposes **data**, **isLoading**, **isError**, and more, so you don't manage those flags by hand. Results are stored in a normalized **cache** keyed by the query key, so multiple components requesting the same key share one fetch and one cached entry. It also handles background **refetching**, retries, and **deduping** of identical in-flight requests automatically.

CODE · TypeScript / TSX

```tsx
function Products() {
  const { data, isLoading, isError } = useQuery({
    queryKey: ['products', { category: 'books' }],
    queryFn: () => fetch('/api/products?cat=books').then(r => r.json()),
  });

  if (isLoading) return <Spinner />;
  if (isError) return <Error />;
  return <List items={data} />;
}
```

ARCHITECTURE / FLOW

```
useQuery({ queryKey, queryFn })
        |
  key in cache & fresh? --yes--> return cached data
        |
       no
        |
  run queryFn --> store in cache[key]
        |
  shared by all components using same key (deduped)
```

Q41 STATE MANAGEMENT & DATA FETCHING

What is the difference between staleTime and gcTime (cacheTime)?

THEORY

These two timers control freshness and retention. staleTime governs when data is refetched; gcTime governs how long unused data stays in memory.

STRONG ANSWER

staleTime is how long fetched data is considered **fresh**; while fresh, React Query serves it from cache without refetching on mount or window focus. Once it becomes **stale** (the default staleTime is 0), the next trigger refetches in the background. **gcTime** (formerly cacheTime, default 5 minutes) is how long an **inactive** query, one with no mounted observers, stays in the cache before garbage collection. So staleTime is about freshness for refetching, while gcTime is about memory cleanup of unused entries.

CODE · TypeScript

```ts
useQuery({
  queryKey: ['profile'],
  queryFn: getProfile,
  staleTime: 60_000,   // fresh for 1 min: no refetch on remount/focus
  gcTime: 5 * 60_000,  // kept 5 min after last observer unmounts
});

// fresh  -> served from cache, no network
// stale  -> served from cache, then background refetch
// no observers + gcTime elapsed -> removed from cache
```

ARCHITECTURE / FLOW

```
Timeline of a query entry:

  fetch -> [ FRESH ]---staleTime---> [ STALE ]
             served as-is              refetch on trigger

  unmount (no observers) -> [ INACTIVE ]---gcTime---> garbage collected
```

Q42 STATE MANAGEMENT & DATA FETCHING

How do mutations and query invalidation work?

THEORY

useMutation handles create/update/delete side effects. After a mutation succeeds, you invalidate related queries so the cache refetches up-to-date data.

STRONG ANSWER

useMutation takes a **mutationFn** and returns a **mutate** function plus status flags; unlike queries it doesn't run automatically. After a successful write you typically call **queryClient.invalidateQueries** with the affected key, which marks those queries stale and triggers a refetch to sync the cache with the server. This **invalidation** pattern keeps client cache and server truth consistent without manually editing cached data. You can also seed the cache directly with **setQueryData** when you already have the fresh value.

CODE · TypeScript

```
const queryClient = useQueryClient();

const addTodo = useMutation({
  mutationFn: (todo) => fetch('/api/todos', {
    method: 'POST', body: JSON.stringify(todo),
  }).then(r => r.json()),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['todos'] }); // refetch list
  },
});

addTodo.mutate({ title: 'Ship it' });
```

ARCHITECTURE / FLOW

```
mutate(newTodo) --> POST /api/todos
        |
     success
        |
  invalidateQueries(['todos'])
        |
  ['todos'] marked stale --> background refetch
        |
  UI shows up-to-date server list
```

Q43 STATE MANAGEMENT & DATA FETCHING

How do you implement optimistic updates with rollback?

THEORY

Optimistic updates apply a change to the cache immediately, before the server confirms, then roll back if the request fails. React Query supports this via mutation lifecycle callbacks.

STRONG ANSWER

In **onMutate** you cancel in-flight queries, snapshot the current cache, then optimistically write the expected new value with **setQueryData**, returning the snapshot as context. If the mutation fails, **onError** uses that snapshot to **roll back** the cache to its previous state. In **onSettled** you invalidate the query to reconcile with the real server response. This gives instant UI feedback while guaranteeing the cache self-corrects on failure.

CODE · TypeScript

```
useMutation({
  mutationFn: toggleTodo,
  onMutate: async (next) => {
    await queryClient.cancelQueries({ queryKey: ['todos'] });
    const prev = queryClient.getQueryData(['todos']);
    queryClient.setQueryData(['todos'], (old) => applyToggle(old, next));
    return { prev }; // context for rollback
  },
  onError: (_e, _next, ctx) => {
    queryClient.setQueryData(['todos'], ctx.prev); // rollback
  },
  onSettled: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),
});
```

ARCHITECTURE / FLOW

```
onMutate:  snapshot prev -> write optimistic value (UI updates now)
      |
  request sent
     /        \
  success      error
    |            |
 onSettled   onError: restore prev (rollback)
    |            |
 invalidate  invalidate (onSettled) -> reconcile w/ server
```

Q44 STATE MANAGEMENT & DATA FETCHING

How do infinite queries / pagination work in React Query?

THEORY

useInfiniteQuery loads pages incrementally, appending results. It tracks page params and exposes helpers to fetch the next page for infinite scroll or 'load more'.

STRONG ANSWER

useInfiniteQuery uses an **initialPageParam** and a **getNextPageParam** function that derives the next cursor or page number from the last page, returning undefined when there's no more data. The data is shaped as `{ pages, pageParams }`, so you flatten `data.pages` to render the combined list. You call **fetchNextPage** to load more, and **hasNextPage** tells you whether to show a load-more button or trigger on scroll. This is ideal for feeds and cursor-based APIs.

CODE · TypeScript / TSX

```
const { data, fetchNextPage, hasNextPage, isFetchingNextPage } = useInfiniteQuery({
  queryKey: ['feed'],
  queryFn: ({ pageParam }) => getFeed(pageParam),
  initialPageParam: 0,
  getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
});

const items = data?.pages.flatMap((p) => p.items) ?? [];
```

CODE · TypeScript / TSX (continued)

```
<button onClick={() => fetchNextPage()} disabled={!hasNextPage || isFetchingNextPage}>
  {isFetchingNextPage ? 'Loading...' : 'Load more'}
</button>
```

ARCHITECTURE / FLOW

```
data = { pages: [P1, P2, P3], pageParams: [0, c1, c2] }

  fetchNextPage()
        |
  getNextPageParam(lastPage) -> next cursor
        |
  undefined? -> hasNextPage = false (stop)
        |
  flatMap(pages) -> single rendered list
```

Q45 STATE MANAGEMENT & DATA FETCHING

Server state vs client state — what is React Query's core philosophy?

THEORY

React Query's central idea is that server state is fundamentally different from client state and shouldn't be forced into a global store. It is a cache, not a source of truth.

STRONG ANSWER

Client state is owned by your app and fully synchronous: form inputs, toggles, theme. **Server state** is owned elsewhere, fetched asynchronously, can become **stale**, and may be changed by other users, so you don't really own it, you just cache a snapshot. React Query's philosophy is to treat server data as a **cache** with freshness rules, deduping, background refetching, and invalidation, rather than copying it into Redux and manually keeping it in sync. This removes huge amounts of boilerplate (loading flags, cache logic) and leaves your global store for true client state only.

CODE · TypeScript

```
// Anti-pattern: server data manually mirrored in a global store
dispatch(setUsers(await fetchUsers())); // you now own sync, staleness, errors

// React Query: treat it as a managed cache
const { data: users } = useQuery({ queryKey: ['users'], queryFn: fetchUsers });

// Client state stays where it belongs
const [isModalOpen, setModalOpen] = useState(false);
```

ARCHITECTURE / FLOW

```
               Client state         Server state
  Owner        your app             the backend
  Sync         synchronous          async (network)
  Staleness    never stale          can go stale
  Tool         useState/Zustand     React Query (cache)

  Rule: don't store server data in a global client store.
```

4 Node.js, Express & REST APIs

Event loop, middleware, JWT, validation, error handling, Sequelize, REST design.

Q46–Q60

Q46 NODE.JS, EXPRESS & REST APIS

Explain the Node.js event loop and non-blocking I/O.

THEORY

Node.js runs JavaScript on a single thread but offloads I/O to the libuv thread pool and the OS. The event loop coordinates callbacks so the main thread is never blocked waiting on I/O.

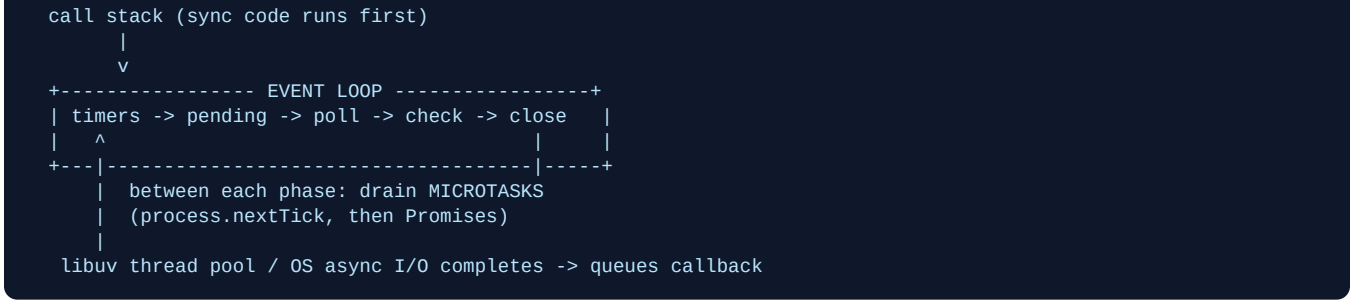
STRONG ANSWER

Node uses a **single-threaded event loop** with **non-blocking I/O**. When you start I/O like a DB query or file read, Node hands it off to **libuv** (a thread pool or OS async primitives) and immediately returns, so the main thread keeps running other code. When the I/O finishes, its **callback** is queued and the event loop runs it on the next available tick. The loop processes work in **phases** (timers, pending callbacks, poll, check, close) and drains **microtasks** (Promises, process.nextTick) between phases. This design lets one thread handle thousands of concurrent connections efficiently, but **CPU-bound work** blocks everything, so it must be offloaded to worker threads.

CODE · JavaScript

```
const fs = require('fs');

console.log('1: start');

setTimeout(() => console.log('4: timer'), 0);

fs.readFile(__filename, () => {
  console.log('5: I/O callback (poll phase)');
  setImmediate(() => console.log('6: setImmediate (check phase)'));
});

Promise.resolve().then(() => console.log('3: microtask'));

process.nextTick(() => console.log('2: nextTick'));

console.log('1.5: end of sync');
// Order: 1 -> 1.5 -> 2 -> 3 -> 4 -> 5 -> 6
```

ARCHITECTURE / FLOW



Q47 NODE.JS, EXPRESS & REST APIS

CommonJS (require) vs ES Modules (import) in Node.

THEORY

CommonJS is Node's original synchronous module system using require/module.exports. ES Modules (ESM) are the standardized async system using import/export, enabled via .mjs or "type": "module".

STRONG ANSWER

CommonJS loads modules **synchronously** at runtime with `require()` and exports via `module.exports`; bindings are **copied values** and `require` can be called conditionally anywhere. **ES Modules** use static `import`/`export`, are resolved **asynchronously**, are **hoisted** to the top, and provide **live bindings** to the exported values. ESM also enables **tree-shaking** and top-level `await`, but you can't mix `require` and `import` freely. You opt into ESM with a `.mjs` extension or `"type": "module"` in package.json. A common gotcha is that ESM has no `__dirname`/`__filename`, so you derive them from `import.meta.url`.

CODE · JavaScript

```
// ----- CommonJS (math.cjs) -----
function add(a, b) { return a + b; }
module.exports = { add };
// consumer
const { add } = require('./math.cjs');

// ----- ES Modules (math.mjs) -----
export function add(a, b) { return a + b; }
export default add;
// consumer
import add, { add as named } from './math.mjs';

// __dirname replacement in ESM
import { fileURLToPath } from 'url';
import { dirname } from 'path';
const __dirname = dirname(fileURLToPath(import.meta.url));
```

ARCHITECTURE / FLOW

```
CommonJS                        ES Modules
--------                        ----------
require()  -> sync              import     -> async, hoisted
module.exports = {}             export / export default
copied values                   live bindings
conditional require ok          static, top-level only
__dirname builtin               import.meta.url
default in Node (.js)           .mjs or type:module
```

Q48 NODE.JS, EXPRESS & REST APIS

How does the Express middleware chain work with next()?

THEORY

Express processes a request through an ordered stack of middleware functions. Each receives (req, res, next) and either responds or calls next() to pass control to the next function.

STRONG ANSWER

Express middleware is a **stack of functions** executed in the order they are registered. Each function gets `(req, res, next)` and must either **end the cycle** by sending a response or call **`next()`** to hand off to the next middleware. If you call `next(err)` with an argument, Express **skips ahead** to error-handling middleware. Middleware can be **global** (`app.use`) or **route-specific**, and it's where you put cross-cutting concerns like logging, parsing, auth, and validation. Forgetting to call `next()` (or send a response) leaves the request **hanging** until it times out.

CODE · JavaScript

```
const express = require('express');
const app = express();

// global middleware
app.use(express.json());

// logging middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // pass control onward
});

// auth middleware on a specific route
function requireAuth(req, res, next) {
  if (!req.headers.authorization) return res.status(401).end();
  next();
}

app.get('/profile', requireAuth, (req, res) => {
  res.json({ ok: true });
});
```

ARCHITECTURE / FLOW

```
  request
     |
     v
  [express.json] --next()--> [logger] --next()--> [requireAuth]
                                                        |
                                              next()    | res.status(401)
                                                v       v
                                          [handler]   response (chain stops)
                                               |
                                               v
                                          res.json -> response
```

Q49 NODE.JS, EXPRESS & REST APIS

Explain routing, route params, and query params in Express.

THEORY

Routing maps an HTTP method and path to a handler. Route params are named path segments captured in req.params; query params come after ? and live in req.query.

STRONG ANSWER

A **route** binds an HTTP **method** plus a **path pattern** to a handler. **Route params** are dynamic path segments declared with a colon, like `/users/:id`, and they appear in **`req.params`**. **Query params** are the key/value pairs after the `?`, like `?page=2&limit=10`, and they appear in **`req.query`** as strings. You use route params to identify a specific resource and query params for filtering, sorting, and pagination. For modularity, related routes are grouped with **`express.Router()`** and mounted under a base path.

CODE · JavaScript

```
const router = require('express').Router();

// GET /users/42?fields=name,email
router.get('/users/:id', (req, res) => {
  const { id } = req.params;          // '42'
  const { fields } = req.query;       // 'name,email'
  res.json({ id, fields });
});

// multiple params: GET /users/42/posts/7
router.get('/users/:userId/posts/:postId', (req, res) => {
  res.json(req.params); // { userId: '42', postId: '7' }
});

module.exports = router;
// app.use('/api', router) -> mounts at /api/users/:id
```

ARCHITECTURE / FLOW

```
  GET  /api/users/42/posts/7?sort=desc&page=2
       \___/\_________________/\_____________/
       base   path + route params   query string

  req.params = { userId: '42', postId: '7' }
  req.query  = { sort: 'desc', page: '2' }
  method     = GET
```

Q50 NODE.JS, EXPRESS & REST APIS

What is centralized (4-arg) error-handling middleware in Express?

THEORY

Express identifies error-handling middleware by its four arguments (err, req, res, next). Registered last, it catches errors passed via next(err) and produces a consistent response.

STRONG ANSWER

Express recognizes **error-handling middleware** by its **four-argument signature** `(err, req, res, next)`. You register it **last**, after all routes, so any error passed with **`next(err)`** lands there instead of crashing the app. This gives you a **single place** to log the error, map known error types to **HTTP status codes**, and return a **consistent JSON shape**. It's best practice to attach a `statusCode` to custom errors and avoid leaking stack traces in production. Note that for **async** handlers you must catch rejections and forward them with `next(err)`, since Express doesn't auto-catch them.

CODE · JavaScript

```
// custom error
class ApiError extends Error {
  constructor(status, message) { super(message); this.statusCode = status; }
}

// somewhere in a route: throw new ApiError(404, 'User not found');

// centralized handler (must be 4 args, registered LAST)
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  if (status >= 500) console.error(err);
  res.status(status).json({
    error: {
      message: status >= 500 ? 'Internal Server Error' : err.message,
      ...(process.env.NODE_ENV !== 'production' && { stack: err.stack })
    }
```

CODE · JavaScript (continued)

```
  });
});
```

ARCHITECTURE / FLOW

```
  route handler
      |  throw / next(err)
      v
  ...skips remaining normal middleware...
      |
      v
  (err, req, res, next)  <-- 4 args => error handler
      |
      +--> log if 5xx
      +--> map err.statusCode
      +--> res.json({ error })  (consistent shape)
```

Q51 NODE.JS, EXPRESS & REST APIS

How do you implement input validation middleware (zod / joi)?

THEORY

Validation middleware parses and checks req.body/params/query against a schema before the handler runs. Invalid requests are rejected early with 400 and clear messages.

STRONG ANSWER

I put **validation at the edge** as middleware so handlers only ever see **well-formed data**. With **zod** or **joi** I define a **schema**, then a generic middleware parses `req.body` (or params/query) and either replaces it with the **typed, coerced** result or returns **400** with the validation errors. This keeps validation **declarative and reusable**, centralizes the error format, and with zod gives me **TypeScript types** for free via `z.infer`. Rejecting bad input early also reduces the attack surface and prevents malformed data from reaching the database.

CODE · TypeScript

```
import { z } from 'zod';
import { Request, Response, NextFunction } from 'express';

const createUser = z.object({
  email: z.string().email(),
  age: z.coerce.number().int().min(18)
});

export const validate = (schema: z.ZodTypeAny) =>
  (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ errors: result.error.flatten() });
    }
    req.body = result.data; // typed + coerced
    next();
  };

// app.post('/users', validate(createUser), createUserHandler);
```

ARCHITECTURE / FLOW

```
  request body (untrusted)
        |
        v
  validate(schema)
```

ARCHITECTURE / FLOW (continued)

```
       |
   safeParse
     /     \
  fail      success
   |           |
  400         req.body = typed data
  errors        |
                v
           handler (trusts input)
```

Q52 NODE.JS, EXPRESS & REST APIS

How does JWT authentication middleware work?

THEORY

A JWT is a signed token carrying claims. Auth middleware extracts the Bearer token, verifies the signature and expiry, and attaches the decoded user to req before protected handlers run.

STRONG ANSWER

A **JWT** is a signed, stateless token with three parts: **header**, **payload** (claims like userId and exp), and **signature**. The auth middleware pulls the token from the **`Authorization: Bearer`** header, calls `jwt.verify` with the **secret**, and on success attaches the decoded **claims** to `req.user`; on failure it returns **401**. Because the signature proves integrity, the server doesn't need to store sessions, which makes it **stateless and scalable**. The trade-offs are that tokens can't be easily revoked before expiry, so I keep **access tokens short-lived** and pair them with refresh tokens. I never put secrets in the payload since it's only **base64-encoded**, not encrypted.

CODE · TypeScript

```typescript
import jwt from 'jsonwebtoken';
import { Request, Response, NextFunction } from 'express';

export function authenticate(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' });
  }
  try {
    const payload = jwt.verify(header.slice(7), process.env.JWT_SECRET!);
    (req as any).user = payload; // { sub, role, exp }
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

ARCHITECTURE / FLOW

```
  Authorization: Bearer eyJ...   (header.payload.signature)
        |
        v
  authenticate middleware
        |
   jwt.verify(token, SECRET)
     /           \
  invalid/expired   valid
     |              |
    401          req.user = claims -> next() -> protected handler
```

Q53 NODE.JS, EXPRESS & REST APIS

Explain refresh-token rotation and why you used it.

THEORY

Refresh-token rotation issues a new refresh token on each use and invalidates the old one. Reuse of a consumed token signals theft and triggers session revocation.

STRONG ANSWER

In my Node/Express APIs I issue a **short-lived access token** and a **long-lived refresh token**. With **rotation**, every time the client redeems a refresh token I **invalidate the old one** and issue a brand-new refresh token, storing it (hashed) in the DB tied to the session. If a **previously-used token** is presented again, that's a strong sign of **theft**, so I revoke the entire **token family/session** and force re-login. I store refresh tokens in an **httpOnly, Secure** cookie to mitigate XSS, while the access token rides the `Authorization` header. This limits the blast radius of a leaked token and gives me **server-side revocation** despite JWTs being stateless.

CODE · JavaScript

```
// POST /auth/refresh
async function refresh(req, res) {
  const presented = req.cookies.refreshToken;
  const stored = await Token.findOne({ where: { hash: sha256(presented) } });

  if (!stored) return res.status(401).end();          // unknown token
  if (stored.used) {                                  // reuse => theft
    await Token.update({ revoked: true }, { where: { familyId: stored.familyId } });
    return res.status(401).json({ error: 'Token reuse detected' });
  }
  await stored.update({ used: true });                // consume old

  const next = issueRefreshToken(stored.userId, stored.familyId); // rotate
  res.cookie('refreshToken', next, { httpOnly: true, secure: true });
  res.json({ accessToken: issueAccessToken(stored.userId) });
}
```

ARCHITECTURE / FLOW

```
  login -> [access(15m)] + [refresh RT1, family F]

  /refresh with RT1 -> mark RT1 used -> issue RT2 (family F)
  /refresh with RT2 -> mark RT2 used -> issue RT3

  /refresh with RT1 again (already used)
        |
        v
  REUSE DETECTED -> revoke entire family F -> 401 -> re-login
```

Q54 NODE.JS, EXPRESS & REST APIS

What are REST principles, resource naming, and HTTP status codes?

THEORY

REST models the API around resources identified by URIs, manipulated with standard HTTP methods and stateless requests. Conventions cover plural nouns, verb-as-method, and meaningful status codes.

STRONG ANSWER

REST treats data as **resources** addressed by URIs, manipulated with **HTTP methods** that carry the verb: **GET** reads, **POST** creates, **PUT/PATCH** update, **DELETE** removes. Requests are **stateless**, so each carries everything needed, and resources are named with **plural nouns** (`/users`, `/users/42/orders`) rather than verbs in the path. I return meaningful **status codes**: 200/201/204 for success, 400/401/403/404/409/422 for client errors, and 5xx for server errors. Good APIs are also **consistent** in pagination, filtering, and error shape. Proper method semantics matter because GET/PUT/DELETE should be **idempotent** and GET should be safe (no side effects).

CODE · JavaScript

```
// Resource: users
// GET    /users           200  list (with ?page=&limit=)
// POST   /users           201  create (Location: /users/42)
// GET    /users/42        200  fetch one (404 if missing)
// PUT    /users/42        200  full replace (idempotent)
// PATCH  /users/42        200  partial update
// DELETE /users/42        204  no content
// Nested: GET /users/42/orders   200

app.post('/users', (req, res) => {
  const user = create(req.body);
  res.status(201).location(`/users/${user.id}`).json(user);
});
```

ARCHITECTURE / FLOW

```
  Method   Path           Success   Common errors
  ------   ----           -------   -------------
  GET      /users         200       -
  POST     /users         201       400, 409
  GET      /users/:id     200       404
  PUT      /users/:id     200       400, 404 (idempotent)
  PATCH    /users/:id     200       400, 404
  DELETE   /users/:id     204       404 (idempotent)
  any (no auth)           ->        401 / 403
```

Q55 NODE.JS, EXPRESS & REST APIS

How do you handle async errors with an asyncHandler wrapper?

THEORY

Express doesn't catch rejected promises from async route handlers. A wrapper attaches a .catch(next) so async errors flow to the centralized error middleware.

STRONG ANSWER

Express **does not catch** rejections from `async` handlers, so an unhandled rejection would crash the process or hang the request. To avoid `try/catch` in every handler, I wrap each one in an **`asyncHandler`** higher-order function that invokes the handler and chains **`.catch(next)`**, forwarding any error to the **centralized error middleware**. This keeps controllers **clean** and guarantees consistent error handling. Newer Express 5 catches async errors automatically, but the wrapper is the standard pattern on Express 4.

CODE · TypeScript

```
// asyncHandler.ts
import { Request, Response, NextFunction, RequestHandler } from 'express';

export const asyncHandler =
  (fn: RequestHandler): RequestHandler =>
```

CODE · TypeScript (continued)

```
  (req: Request, res: Response, next: NextFunction) =>
    Promise.resolve(fn(req, res, next)).catch(next);

// usage - rejection is forwarded to error middleware automatically
app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await db.findUser(req.params.id); // may reject
  if (!user) throw new ApiError(404, 'Not found');
  res.json(user);
}));
```

ARCHITECTURE / FLOW

```
  async handler rejects
        |
   without wrapper:  rejection uncaught -> request hangs / crash
        |
   with asyncHandler:
        |
   Promise.resolve(fn(...)).catch(next)
        |
        v
   next(err) -> centralized (err,req,res,next) handler -> JSON error
```

Q56 NODE.JS, EXPRESS & REST APIS

Explain Node streams and backpressure.

THEORY

Streams process data in chunks instead of buffering it all in memory. Backpressure is the mechanism that slows a fast producer when a slower consumer can't keep up.

STRONG ANSWER

Streams let you process data **chunk by chunk** rather than loading everything into memory, which is essential for large files, network data, and DB cursors. There are four types: **Readable**, **Writable**, **Duplex**, and **Transform**. **Backpressure** is the flow-control signal: when you `write()` to a slower destination and its internal buffer fills, `write` returns **false**, telling the producer to **pause** until the `drain` event fires. The idiomatic way to handle this automatically is **`pipe`** (or `pipeline`), which manages backpressure and error propagation for you. Without it, a fast producer would balloon memory and risk crashing the process.

CODE · JavaScript

```
const fs = require('fs');
const { pipeline } = require('stream');
const zlib = require('zlib');

// pipeline handles backpressure + errors + cleanup
pipeline(
  fs.createReadStream('big.log'),     // Readable
  zlib.createGzip(),                   // Transform
  fs.createWriteStream('big.log.gz'),  // Writable
  (err) => {
    if (err) console.error('Failed:', err);
    else console.log('Done');
  }
);

// manual backpressure (what pipe does for you):
// if (!writable.write(chunk)) readable.pause();
// writable.on('drain', () => readable.resume());
```

ARCHITECTURE / FLOW

```
  Readable (fast)        Writable (slow)
     producer  --chunk-->  buffer
        |                     |
        |   write() === false (buffer full)
        |<-------- PAUSE -----+
        |
        |   'drain' emitted (buffer drained)
        +-------- RESUME ----->

  pipe()/pipeline() automate this pause/resume cycle
```

Q57 NODE.JS, EXPRESS & REST APIS

How do Sequelize models and associations (hasMany / belongsTo) work?

THEORY

A Sequelize model maps a class to a table. Associations declare relationships so Sequelize manages foreign keys and provides helper methods and eager-loading.

STRONG ANSWER

A **Sequelize model** maps a JS class to a **database table**, with attributes defining columns and types. **Associations** declare relationships: a `User.hasMany(Order)` paired with `Order.belongsTo(User)` sets up a **one-to-many** relation where the **foreign key** (`userId`) lives on the `Order` table. Declaring both sides gives you **mixin helper methods** like `user.getOrders()` and `user.createOrder()`, plus the ability to **eager-load** related rows with `include`. For many-to-many you use `belongsToMany` through a **join table**. Getting the FK ownership right matters: `belongsTo` always puts the foreign key on the **source/owning** model.

CODE · JavaScript

```javascript
const User = sequelize.define('User', { name: DataTypes.STRING });
const Order = sequelize.define('Order', {
  total: DataTypes.DECIMAL,
  userId: DataTypes.INTEGER
});

// one-to-many: FK userId on Order
User.hasMany(Order, { foreignKey: 'userId' });
Order.belongsTo(User, { foreignKey: 'userId' });

// helper methods generated by the association:
const user = await User.create({ name: 'Ada' });
await user.createOrder({ total: 99.5 });
const orders = await user.getOrders();

// eager load
const withOrders = await User.findByPk(user.id, { include: Order });
```

ARCHITECTURE / FLOW

```
  User (1) --------- hasMany --------> (N) Order
   id  <----------- belongsTo (FK userId) ---- userId
   name                                       total

  generated helpers:
    user.getOrders() / user.createOrder() / user.setOrders()
    User.findByPk(id, { include: Order })  -> eager load
```

Q58 NODE.JS, EXPRESS & REST APIS

Sequelize migrations vs sync: which and why?

THEORY

sync() auto-creates/alters tables from model definitions. Migrations are versioned, ordered scripts that evolve the schema deterministically and reversibly.

STRONG ANSWER

`sequelize.sync()` generates the schema directly from your **model definitions**, which is convenient for **prototyping and tests** but dangerous in production because `alter`/`force` can silently change or drop data. **Migrations** are **versioned scripts** with `up`/`down` functions run by the CLI; they give you an **ordered, reversible, reviewable** history of schema changes that you commit to source control and run identically in every environment. The rule of thumb is **sync for local/test, migrations for production**. Migrations also let you do data backfills and coordinate schema changes with deploys safely.

CODE · JavaScript

```
// DEV/TEST only - never in prod with real data
await sequelize.sync({ alter: true });

// Production: versioned migration (run via sequelize-cli)
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.addColumn('Orders', 'status', {
      type: Sequelize.STRING,
      allowNull: false,
      defaultValue: 'pending'
    });
  },
  async down(queryInterface) {
    await queryInterface.removeColumn('Orders', 'status'); // reversible
  }
};
// npx sequelize-cli db:migrate / db:migrate:undo
```

ARCHITECTURE / FLOW

```
  sync()                        migrations
  ------                        ----------
  derived from models           explicit up/down scripts
  no history                    versioned + ordered
  not reversible (force drops)  reversible (down)
  ok: local / tests             required: staging / prod

  flow: write migration -> commit -> db:migrate on deploy
```

Q59 NODE.JS, EXPRESS & REST APIS

What is the N+1 query problem and how does eager loading fix it?

THEORY

N+1 happens when you fetch N parent rows then run one extra query per parent for its relation. Eager loading with include fetches everything in a single joined query.

STRONG ANSWER

The **N+1 problem** occurs when you load **N parent records** and then trigger **one additional query per record** to fetch a relation, giving you **N+1 total queries** and terrible performance under load. In Sequelize this happens with **lazy loading** (calling `user.getOrders()` inside a loop). The fix is **eager loading** via the **`include`** option, which makes Sequelize emit a **single query with a JOIN** to fetch parents and their relations together. For very large result sets you can instead use **`separate: true`** (one query per association, so 2 total) to avoid row multiplication from the join. The key is to never query inside a loop.

CODE · JavaScript

```
// BAD: N+1 -> 1 query for users + 1 per user for orders
const users = await User.findAll();
for (const u of users) {
  u.orders = await u.getOrders(); // extra query each iteration!
}

// GOOD: eager load -> a single JOIN query
const users2 = await User.findAll({
  include: [{ model: Order, as: 'orders' }]
});

// For big children, 2 queries instead of a huge join:
const users3 = await User.findAll({
  include: [{ model: Order, separate: true }]
});
```

ARCHITECTURE / FLOW

```
N+1 (lazy):
  SELECT * FROM users;            -> N users
  SELECT * FROM orders WHERE userId=1;
  SELECT * FROM orders WHERE userId=2;
  ... (N more queries)            => 1 + N queries

Eager (include):
  SELECT ... FROM users
    LEFT JOIN orders ON orders.userId = users.id;  => 1 query
```

Q60 NODE.JS, EXPRESS & REST APIS

How do you secure an API: rate limiting, helmet, and CORS?

THEORY

Defense in depth combines abuse prevention (rate limiting), secure HTTP headers (helmet), and controlled cross-origin access (CORS), alongside auth and validation.

STRONG ANSWER

I layer several middlewares for **defense in depth**. **Rate limiting** (e.g. express-rate-limit, ideally backed by Redis) caps requests per IP/key to blunt **brute-force and DoS** abuse, returning **429** when exceeded. **Helmet** sets protective **HTTP headers** like HSTS, X-Content-Type-Options, and a Content-Security-Policy to reduce XSS/clickjacking risk. **CORS** controls which **origins** may call the API from a browser; I use an **allowlist** rather than `*`, especially when sending **credentials**. On top of these I keep **input validation**, **JWT auth**, parameterized queries via the ORM, and HTTPS. The principle is that no single control is enough, so they **stack**.

CODE · JavaScript

```
const helmet = require('helmet');
const cors = require('cors');
const rateLimit = require('express-rate-limit');

app.use(helmet()); // secure default headers

app.use(cors({
  origin: ['https://app.example.com'], // allowlist, not '*'
  credentials: true
}));

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 100,                 // per IP per window
  standardHeaders: true,
  message: { error: 'Too many requests' } // -> 429
});
app.use('/api', limiter);
```

ARCHITECTURE / FLOW

```
  incoming request
        |
        v
  [helmet]  set secure headers (HSTS, CSP, noSniff)
        |
  [cors]    origin allowed? --no--> blocked by browser
        |
  [rateLimit] under limit? --no--> 429 Too Many Requests
        |
  [validate] -> [authenticate] -> handler

  layered controls = defense in depth
```

5 Golang, Gin & GORM

Goroutines, channels, interfaces, Gin middleware, GORM, clean architecture.

Q61–Q75

Q61 GOLANG, GIN & GORM

What are goroutines and how do they differ from OS threads?

THEORY

A goroutine is a lightweight, function-level unit of concurrent execution managed by the Go runtime. The runtime multiplexes many goroutines onto a small pool of OS threads.

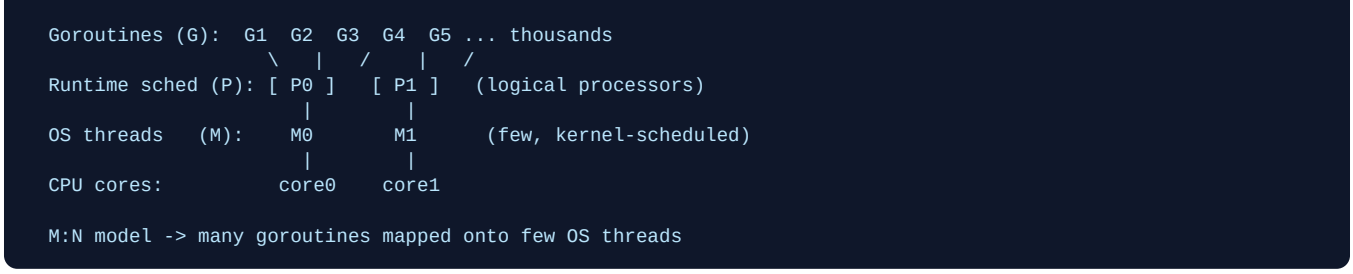
STRONG ANSWER

A **goroutine** is a function running concurrently, started with the **go** keyword. Unlike **OS threads** that cost ~1MB of stack and are scheduled by the kernel, goroutines start with a tiny ~2KB **growable stack** and are scheduled in user space by the Go runtime's **M:N scheduler** (the **GMP model**). This means I can spawn hundreds of thousands of goroutines cheaply, whereas OS threads would exhaust memory quickly. Context switches between goroutines are also far cheaper because they avoid kernel transitions. The runtime handles blocking calls by parking the goroutine and reusing the thread for others.

CODE · Go

```go
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(id int) {        // launch a goroutine
            defer wg.Done()
            fmt.Printf("worker %d running\n", id)
        }(i)
    }
    wg.Wait()                    // block until all finish
}
```

ARCHITECTURE / FLOW



Q62 GOLANG, GIN & GORM

What are channels, and what is the difference between buffered and unbuffered channels?

THEORY

Channels are typed conduits that let goroutines communicate and synchronize by sending and receiving values. They embody Go's motto: share memory by communicating.

STRONG ANSWER

A **channel** is created with **make(chan T)** and used to pass values safely between goroutines. An **unbuffered channel** has capacity zero, so a send blocks until another goroutine receives, giving a synchronous **handshake**. A **buffered channel**, made with **make(chan T, n)**, lets up to n values sit in the buffer, so sends only block when the buffer is full and receives only block when it is empty. I use unbuffered channels for tight synchronization and buffered ones to smooth out bursty producers. Closing a channel with **close()** signals no more values, and ranging over it exits cleanly.

CODE · Go

```go
unbuf := make(chan int)      // capacity 0, synchronous
buf := make(chan int, 2)     // capacity 2, async up to 2

go func() {
    buf <- 1                 // does not block
    buf <- 2                 // does not block (buffer full now)
    close(buf)               // signal completion
}()

for v := range buf {         // drains until closed
    fmt.Println(v)
}
_ = unbuf
```

ARCHITECTURE / FLOW

```
  Unbuffered (cap 0):
    sender ---X---> [   ] ---X---> receiver
    send blocks until receiver is ready (handshake)

  Buffered (cap 2):
    sender ---> [ v1 | v2 ] ---> receiver
    send blocks only when buffer full;
    recv blocks only when buffer empty
```

Q63 GOLANG, GIN & GORM

What does the select statement do in Go?

THEORY

select lets a goroutine wait on multiple channel operations at once, proceeding with whichever is ready. It is the core primitive for multiplexing concurrent communication.

STRONG ANSWER

select blocks until one of its **case** channel operations can proceed; if several are ready it picks one at random to avoid starvation. I use it to listen on multiple channels, implement **timeouts** with **time.After**, and respond to cancellation via **ctx.Done()**. Adding a **default** case makes the select **non-blocking**, returning immediately if no channel is ready. It is the idiomatic way to coordinate fan-in, fan-out, and graceful shutdown. Without a default and with all cases blocked, the goroutine simply waits.

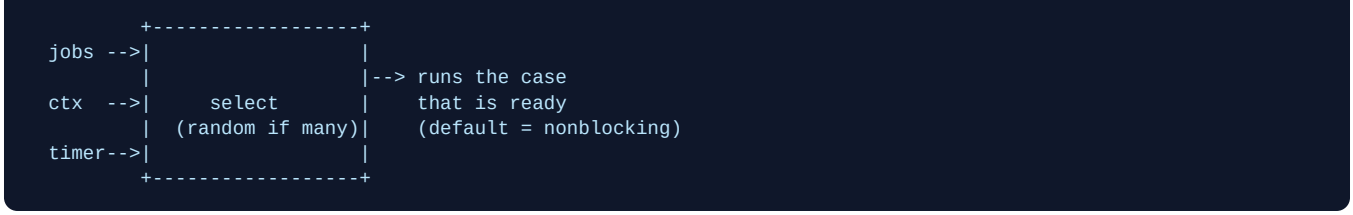
CODE · Go

```go
select {
case msg := <-jobs:
    fmt.Println("got job:", msg)
case <-ctx.Done():
    fmt.Println("cancelled:", ctx.Err())
    return
```

CODE · Go (continued)

```
case <-time.After(2 * time.Second):
    fmt.Println("timed out waiting")
default:
    fmt.Println("nothing ready, moving on")
}
```

ARCHITECTURE / FLOW



Q64 GOLANG, GIN & GORM

How do sync.Mutex and sync.WaitGroup work?

THEORY

sync.Mutex provides mutual exclusion to protect shared state, while sync.WaitGroup waits for a collection of goroutines to finish. Both live in the sync package.

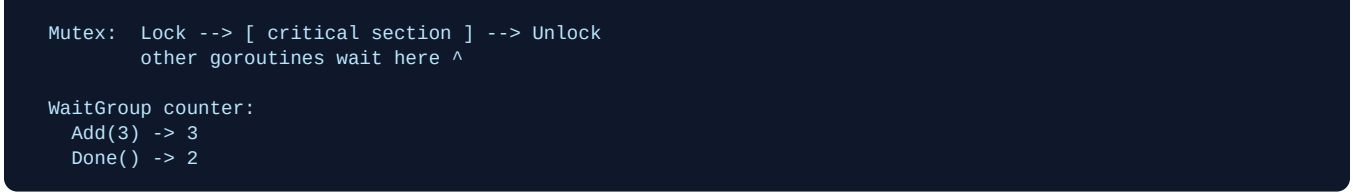
STRONG ANSWER

A **sync.Mutex** guards a **critical section**: I call **Lock()** before touching shared data and **Unlock()** after, usually with **defer**, so only one goroutine accesses it at a time and I avoid **data races**. A **sync.WaitGroup** is a counter for goroutine completion: **Add(n)** increments it, each goroutine calls **Done()** when finished, and **Wait()** blocks until the counter hits zero. The key rule is to call **Add** before launching the goroutine, not inside it, to avoid a race with Wait. For read-heavy workloads I might reach for **sync.RWMutex** instead.

CODE · Go

```
type Counter struct {
    mu sync.Mutex
    n  int
}
func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++                   // safe critical section
}

var wg sync.WaitGroup
c := &Counter{}
for i := 0; i < 100; i++ {
    wg.Add(1)
    go func() { defer wg.Done(); c.Inc() }()
}
wg.Wait()
fmt.Println(c.n)            // 100
```

ARCHITECTURE / FLOW



ARCHITECTURE / FLOW (continued)

```
    Done() -> 1
    Done() -> 0  ==> Wait() unblocks
```

Q65 GOLANG, GIN & GORM

How do interfaces work in Go, and what is implicit satisfaction?

THEORY

An interface is a set of method signatures; any type that implements those methods satisfies it automatically. There is no explicit implements keyword.

STRONG ANSWER

A Go **interface** declares behavior as a list of methods, and a type satisfies it simply by having those methods, known as **implicit satisfaction** or **structural typing**. This decouples packages: a consumer can define a small interface it needs without the implementer importing it. I follow the idiom of keeping interfaces small, like the single-method **io.Reader**, and accepting interfaces while returning concrete structs. The empty interface **interface{}** (or **any**) holds any value, and I use **type assertions** or a **type switch** to recover the concrete type. This design powers Go's testability through easy mocking.

CODE · Go

```go
type Notifier interface {
    Notify(msg string) error      // any type with this satisfies it
}

type EmailService struct{ from string }

func (e EmailService) Notify(msg string) error {
    fmt.Printf("email from %s: %s\n", e.from, msg)
    return nil                    // no "implements" needed
}

func send(n Notifier, m string) error { return n.Notify(m) }

func main() { _ = send(EmailService{from: "ops"}, "deploy ok") }
```

ARCHITECTURE / FLOW

```
  interface Notifier { Notify(string) error }
            ^                  ^
            | satisfies       | satisfies (implicitly)
   EmailService           SMSService
   (has Notify)           (has Notify)

  No "implements" keyword: matching methods = satisfaction
```

Q66 GOLANG, GIN & GORM

What is idiomatic error handling in Go (error as a value)?

THEORY

Go treats errors as ordinary values returned from functions rather than thrown exceptions. Callers inspect them explicitly, typically as the last return value.

STRONG ANSWER

In Go, **error** is a built-in interface with a single **Error() string** method, and functions return it as the last value so I handle it right where it occurs with **if err != nil**. I add context as the error propagates using **fmt.Errorf** with the **%w** verb to **wrap** it, preserving the chain. Callers can then inspect it with **errors.Is** for sentinel values or **errors.As** to extract a typed error. This explicit, value-based style makes control flow obvious and avoids hidden exception paths. The trade-off is verbosity, which I accept for clarity and reliability.

CODE · Go

```go
var ErrNotFound = errors.New("user not found")

func findUser(id int) (*User, error) {
    u, ok := store[id]
    if !ok {
        return nil, fmt.Errorf("findUser %d: %w", id, ErrNotFound)
    }
    return u, nil
}

func main() {
    _, err := findUser(42)
    if errors.Is(err, ErrNotFound) {   // unwraps the chain
        fmt.Println("handle missing user:", err)
    }
}
```

ARCHITECTURE / FLOW

```
  func -> (value, error)
            |       |
         use it   if err != nil { handle / wrap with %w }

  wrapping chain:
    ErrNotFound  <-- wrapped by --  "findUser 42: ..."
    errors.Is(err, ErrNotFound) walks chain -> true
```

Q67 GOLANG, GIN & GORM

What are structs and struct embedding (composition) in Go?

THEORY

A struct groups named fields into a composite type. Embedding places one type inside another to reuse its fields and methods, favoring composition over inheritance.

STRONG ANSWER

A **struct** is a typed collection of **fields**, and methods are attached via receivers. Go has no class inheritance; instead it uses **embedding**, where I declare a type without a field name and the outer struct **promotes** the embedded type's fields and methods as if they were its own. This is **composition over inheritance**: behavior is assembled, not inherited. Embedding an interface is also common to satisfy it partially or to wrap implementations. If names collide, the outer type's members take precedence and inner ones are still reachable by qualifying them.

CODE · Go

```go
type Auditable struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}
func (a Auditable) Age() time.Duration { return time.Since(a.CreatedAt) }
```

CODE · Go (continued)

```
type User struct {
    Auditable          // embedded: fields + Age() promoted
    ID   int
    Name string
}

func main() {
    u := User{Auditable: Auditable{CreatedAt: time.Now()}, Name: "Sam"}
    fmt.Println(u.CreatedAt, u.Age())   // promoted access
}
```

ARCHITECTURE / FLOW

```
type User struct {
    Auditable   <-- embedded (no field name)
    ID int
    Name string
}

User --promotes--> CreatedAt, UpdatedAt, Age()
composition, not inheritance:
u.Age()  ==  u.Auditable.Age()
```

Q68 GOLANG, GIN & GORM

How do defer, panic, and recover work?

THEORY

defer schedules a call to run when the surrounding function returns. panic unwinds the stack, and recover regains control inside a deferred function.

STRONG ANSWER

defer registers a function call that runs when the enclosing function exits, in **LIFO** order, which I use for cleanup like closing files or unlocking mutexes, even on early returns. **panic** stops normal flow and unwinds the stack, running deferred calls as it goes, typically for truly unrecoverable situations. **recover**, called only inside a deferred function, stops the unwinding and returns the panic value, letting me turn a panic into a normal error. The idiom is to use errors for expected failures and reserve panic/recover for things like a top-level handler that keeps a server from crashing. Deferred argument values are evaluated when defer runs, not when the function exits.

CODE · Go

```
func safeDivide(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {     // catch the panic
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    return a / b, nil                     // b==0 panics
}

func main() {
    res, err := safeDivide(10, 0)
    fmt.Println(res, err)                 // 0 recovered: ...
}
```

ARCHITECTURE / FLOW

```
normal:  call --> defer registered --> return --> defers run (LIFO)

panic:   panic(x)
           |
       stack unwinds, running defers
           |
       defer calls recover() --> stops unwind, returns x
           |
       function returns normally with an error
```

Q69 GOLANG, GIN & GORM

What is the context package used for (cancellation, timeouts)?

THEORY

The context package carries deadlines, cancellation signals, and request-scoped values across API boundaries and goroutines. It is the standard way to control goroutine lifetimes.

STRONG ANSWER

A **context.Context** propagates **cancellation**, **deadlines**, and **timeouts** down a call tree so I can stop work that is no longer needed, like an abandoned HTTP request. I derive child contexts with **WithCancel**, **WithTimeout**, or **WithDeadline**, and every derived call returns a **cancel** function I should **defer** to release resources. Functions watch **ctx.Done()** in a select and check **ctx.Err()** to know why they stopped. The convention is to pass ctx as the first parameter and never store it in a struct. I also use **context.WithValue** sparingly for request-scoped data like a trace ID.

CODE · Go

```go
func fetch(ctx context.Context) error {
    select {
    case <-time.After(3 * time.Second):  // simulated slow work
        return nil
    case <-ctx.Done():
        return ctx.Err()                 // canceled or deadline
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()
    fmt.Println(fetch(ctx))              // context deadline exceeded
}
```

ARCHITECTURE / FLOW

```
Background
    |
WithTimeout(1s) --> ctx (+ cancel func)
    |                  |
passed down -------> goroutines watch ctx.Done()
    |
deadline hit OR cancel() called
    |
ctx.Done() closed --> all watchers stop, ctx.Err() set
```

Q70 GOLANG, GIN & GORM

How do routing and handlers work in Gin?

THEORY

Gin is a fast HTTP web framework built on a radix-tree router. Handlers are functions taking a *gin.Context that read the request and write the response.

STRONG ANSWER

In **Gin** I create an engine with **gin.Default()**, then register routes by HTTP method, like **r.GET("/users/:id", handler)**, where the router uses a **radix tree** for fast matching. A handler has the signature **func(c *gin.Context)** and uses the **gin.Context** to read **path params** via **c.Param**, **query params** via **c.Query**, and JSON bodies via **c.ShouldBindJSON**. I write responses with helpers like **c.JSON(http.StatusOK, obj)**. Routes can be organized into **route groups** with shared prefixes and middleware, which keeps a versioned API like **/api/v1** clean. Binding plus struct validation tags handles most request parsing for me.

CODE · Go

```go
func main() {
    r := gin.Default()
    v1 := r.Group("/api/v1")
    {
        v1.GET("/users/:id", getUser)
        v1.POST("/users", createUser)
    }
    r.Run(":8080")
}

func getUser(c *gin.Context) {
    id := c.Param("id")             // path param
    q := c.DefaultQuery("fields", "all")
    c.JSON(http.StatusOK, gin.H{"id": id, "fields": q})
}
```

ARCHITECTURE / FLOW

```
  HTTP request
      |
  gin.Engine (radix-tree router)
      |  match method + path
  route group /api/v1
      |
  handler func(c *gin.Context)
      |  c.Param / c.Query / c.ShouldBindJSON
      v
  c.JSON(status, body) --> HTTP response
```

Q71 GOLANG, GIN & GORM

How do you write middleware in Gin?

THEORY

Gin middleware is a gin.HandlerFunc that runs before and/or after the main handler in a chain. It is used for cross-cutting concerns like auth, logging, and recovery.

STRONG ANSWER

Gin **middleware** is just a **func(c *gin.Context)** that I register with **r.Use(...)** globally, on a **route group**, or per route. Inside it I do pre-processing, then call **c.Next()** to run the rest of the chain, and code after **c.Next()** runs on the way back out, which is handy for timing or logging. To stop the chain, for example on a failed auth check, I call **c.AbortWithStatusJSON(...)** so downstream handlers never run. I can also stash request-scoped data with **c.Set** and read it later with **c.Get**. This is how I add JWT auth, request IDs, and metrics consistently.

CODE · Go

```go
func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        token := c.GetHeader("Authorization")
        if token == "" {
            c.AbortWithStatusJSON(401, gin.H{"error": "missing token"})
            return                       // stop the chain
        }
        c.Set("userID", parse(token))   // share with handlers
        c.Next()                         // run downstream handlers
    }
}

// usage: r.Use(AuthMiddleware())
```

ARCHITECTURE / FLOW

```
  request
     |
  [ Logger ] -- c.Next() -->
     |                      \
  [ Auth ] -- c.Next() -->    | chain
     |                      /
  [ handler ] -- writes response
     |
  <-- code after c.Next() runs (timing/logging) <--
  (c.Abort() short-circuits the chain)
```

Q72 GOLANG, GIN & GORM

How do you define models and perform CRUD with GORM?

THEORY

GORM is Go's most popular ORM, mapping structs to database tables. Models are plain structs, often embedding gorm.Model for common fields.

STRONG ANSWER

In **GORM** a **model** is a struct whose fields map to columns, and embedding **gorm.Model** adds **ID**, **CreatedAt**, **UpdatedAt**, and a soft-delete **DeletedAt**. I control mapping with **struct tags** like **gorm:"uniqueIndex"** and run **db.AutoMigrate(&User{})** to create or update the schema. CRUD is method-chained on the **DB** handle: **Create** to insert, **First** or **Find** to read, **Save** or **Updates** to update, and **Delete** to remove. Every call returns a result whose **.Error** I check, and conditions are built with **Where**. Soft deletes mean **Delete** just sets DeletedAt unless I use **Unscoped**.

CODE · Go

```go
type User struct {
    gorm.Model                // ID, timestamps, soft delete
    Name  string `gorm:"size:100"`
    Email string `gorm:"uniqueIndex"`
```

CODE · Go (continued)

```
}

db.AutoMigrate(&User{})

db.Create(&User{Name: "Sam", Email: "s@x.io"})   // C

var u User
db.First(&u, "email = ?", "s@x.io")              // R

db.Model(&u).Update("Name", "Samuel")            // U
db.Delete(&u)                                    // D (soft)
```

ARCHITECTURE / FLOW

```
  struct User { gorm.Model; Name; Email }
         |  AutoMigrate
         v
  table users (id, created_at, updated_at, deleted_at, name, email)

  Create -> INSERT
  First/Find -> SELECT (... WHERE)
  Update/Save -> UPDATE
  Delete -> soft delete (sets deleted_at)
```

Q73 GOLANG, GIN & GORM

How do associations and Preload work in GORM?

THEORY

GORM models relationships between tables as struct fields: has-one, has-many, belongs-to, and many-to-many. Preload eagerly loads those related records.

STRONG ANSWER

I define **associations** by adding related structs or slices as fields, with a **foreign key** field connecting them, for example a **User** with a slice of **Orders** (**has-many**) where each Order has a **UserID** (**belongs-to**). By default GORM does not load related data, so I call **Preload("Orders")** to eagerly fetch them in a separate query, avoiding the **N+1 problem** of querying inside a loop. Preload can be nested like **Preload("Orders.Items")** and constrained with a function for filtered loads. For **many-to-many** I use the **many2many** tag pointing at a join table. GORM can also auto-create associations on Create.

CODE · Go

```
type User struct {
    gorm.Model
    Name   string
    Orders []Order           // has-many
}
type Order struct {
    gorm.Model
    UserID uint              // belongs-to (foreign key)
    Total  float64
}

var u User
db.Preload("Orders").First(&u, 1)   // eager-load orders
fmt.Println(u.Orders)

// nested + filtered:
// db.Preload("Orders", "total > ?", 100).Find(&users)
```

ARCHITECTURE / FLOW

```
users 1 ------ * orders        (has-many / belongs-to)
 id <---------- user_id

Without Preload: load user, then 1 query per order (N+1)

db.Preload("Orders").First(&u, 1):
  Q1: SELECT * FROM users WHERE id=1
  Q2: SELECT * FROM orders WHERE user_id IN (1)
-> 2 queries, no N+1
```

Q74 GOLANG, GIN & GORM

How do you apply dependency injection and Clean Architecture in Go?

THEORY

Clean Architecture separates code into layers with dependencies pointing inward toward the domain. Dependency injection wires concrete implementations into those layers, usually via interfaces.

STRONG ANSWER

I structure services in layers: a **handler/transport** layer (Gin), a **service/use-case** layer holding business logic, and a **repository** layer for persistence (GORM), with the domain at the center. The key rule is the **dependency inversion principle**: inner layers define **interfaces**, and outer layers implement them, so the service depends on a **Repository interface**, not on GORM directly. I do **dependency injection** by **constructor injection**, passing implementations into constructors in **main.go**, which makes the service trivially testable with a mock repo. This matches my resume work building **repository** and **service** layers in Go. The payoff is swappable infrastructure and clean unit tests.

CODE · Go

```go
type UserRepository interface {        // defined in domain/service layer
    FindByID(ctx context.Context, id int) (*User, error)
}

type UserService struct{ repo UserRepository }   // depends on interface

func NewUserService(r UserRepository) *UserService {
    return &UserService{repo: r}        // constructor injection
}
func (s *UserService) Get(ctx context.Context, id int) (*User, error) {
    return s.repo.FindByID(ctx, id)
}

// main.go: svc := NewUserService(NewGormUserRepo(db))
```

ARCHITECTURE / FLOW

```
  +-----------------------------------------------+
  |  Handler (Gin)  -> Service -> Repository iface |
  |     transport       use-case      (interface) |
  +-----------------------------------------------+
                               ^ implemented by
                         GormUserRepo (DB detail)

  Dependencies point INWARD; DI wires them in main.go
```

Q75 GOLANG, GIN & GORM

How do Go modules work and what is a clean project structure?

THEORY

Go modules are the dependency management system, defined by a go.mod file at the project root. A clean layout separates entry points, private packages, and public APIs.

STRONG ANSWER

A **Go module** is a versioned collection of packages declared by **go.mod**, which records the module path and dependencies, while **go.sum** locks their checksums for reproducible builds. I add dependencies implicitly via **go get** or **go mod tidy**, which prunes unused ones, and Go uses **semantic versioning** with **minimal version selection**. For layout I follow the common community convention: **cmd/** for entry-point mains, **internal/** for packages that cannot be imported outside the module, and **pkg/** for reusable public code. Inside internal I split by layer, such as **handler**, **service**, and **repository**, matching Clean Architecture. This keeps imports honest and the dependency graph one-directional.

CODE · Go

```go
// go.mod
module github.com/me/orders-api

go 1.22

require (
    github.com/gin-gonic/gin v1.10.0
    gorm.io/gorm v1.25.10
)

// commands:
//   go mod init github.com/me/orders-api
//   go get gorm.io/gorm
//   go mod tidy   // add missing, remove unused deps
```

ARCHITECTURE / FLOW

```
orders-api/
|- go.mod / go.sum        module + locked deps
|- cmd/
|   '- api/main.go        entry point, wires DI
|- internal/             private to this module
|   |- handler/          Gin handlers
|   |- service/          business logic
|   '- repository/       GORM data access
|- pkg/                  reusable public packages
'- configs/
```

6 Databases & Caching

SQL vs NoSQL, indexing, transactions, Postgres, MongoDB, Redis, pgvector.

Q76–Q90

Q76 DATABASES & CACHING

SQL vs NoSQL — when to choose each

THEORY

SQL databases enforce a fixed schema with relational integrity and ACID guarantees, while NoSQL databases trade strict structure for flexible schemas and horizontal scale.

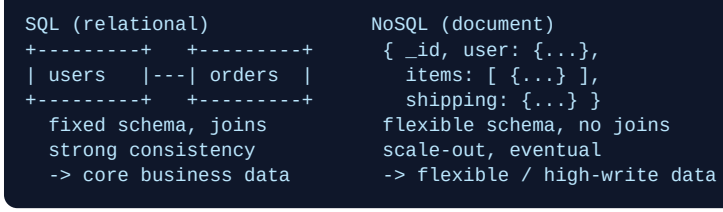
STRONG ANSWER

I reach for a **SQL** database like **PostgreSQL** when data is highly relational, I need **strong consistency**, complex **joins**, and multi-row **transactions** — most financial or core business data. I choose **NoSQL** like **MongoDB** when the schema is fluid, the access pattern is document-shaped, or I need to scale writes across many nodes. The honest answer is most systems are **polyglot**: I keep the source of truth in Postgres and add **Redis** for caching or Mongo for flexible documents. The decision is really about **access patterns and consistency needs**, not hype.

CODE · SQL

```sql
-- SQL: relational, ACID, joins across normalized tables
SELECT u.name, SUM(o.total) AS spend
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.created_at >= '2026-01-01'
GROUP BY u.name;

-- NoSQL (Mongo): self-contained document, flexible shape
-- db.orders.find({ userId: 42, status: 'paid' })
```

ARCHITECTURE / FLOW



Q77 DATABASES & CACHING

Indexing and how a B-tree index speeds lookups

THEORY

An index is an auxiliary sorted data structure that lets the database find rows without scanning the whole table. Most relational indexes are B-trees, which keep keys sorted in a shallow, balanced tree.

STRONG ANSWER

Without an index a lookup is a **sequential scan** — O(n) over every row. A **B-tree index** stores keys sorted across a balanced tree, so the engine narrows the search by comparing at each node, giving **O(log n)** lookups. Because the tree stays balanced and shallow (usually 3-4 levels), even millions of rows resolve in a handful of page reads. B-trees also support **range queries** and **ORDER BY** since keys are stored in order. The tradeoff is that every index adds **write and storage overhead**, so I index columns used in WHERE, JOIN, and ORDER BY — not everything.

CODE · SQL

```sql
-- Sequential scan: reads every row
SELECT * FROM users WHERE email = 'a@b.com';

-- Add a B-tree index -> O(log n) lookup
CREATE INDEX idx_users_email ON users (email);

-- Range queries also use the B-tree (sorted keys)
SELECT * FROM events WHERE created_at >= '2026-06-01'
ORDER BY created_at;
```

ARCHITECTURE / FLOW

```
B-tree index lookup for key = 42
           [ 30 | 60 ]            <- root
          /    |     \
     [10|20] [40|50]  [70|80]      <- internal
               |
         leaf: 40 41 [42] 43 ...   -> row pointer

  Each level halves the search space -> O(log n)
```

Q78 DATABASES & CACHING

Composite indexes and column order (leftmost prefix)

THEORY

A composite index covers multiple columns in a defined order. It can only be used efficiently for queries that filter on a leftmost prefix of those columns.

STRONG ANSWER

A **composite index** on (a, b, c) is sorted first by a, then b, then c — like a phone book sorted by last then first name. The **leftmost-prefix rule** means the index helps queries filtering on a, or a+b, or a+b+c, but **not** a query that filters only on b or c. So column order matters: I put the most **selective** or most **commonly filtered** equality columns first and range columns last. A query using a range on an early column stops the index from helping later columns. Designing order to match real query patterns is what makes the index pay off.

CODE · SQL

```sql
CREATE INDEX idx_orders ON orders (customer_id, status, created_at);

-- USES index (leftmost prefix)
SELECT * FROM orders WHERE customer_id = 7;
SELECT * FROM orders WHERE customer_id = 7 AND status = 'paid';

-- DOES NOT use it efficiently (skips leading column)
SELECT * FROM orders WHERE status = 'paid';
```

ARCHITECTURE / FLOW

```
Index (customer_id, status, created_at) sorted left-to-right

  cust=7 | paid    | 2026-01-02
  cust=7 | paid    | 2026-03-09
  cust=7 | shipped | 2026-02-01
  cust=8 | paid    | 2026-01-05

  filter cust       -> YES    filter cust+status -> YES
  filter status only-> NO (not a leftmost prefix)
```

Q79 DATABASES & CACHING

ACID properties and transactions

THEORY

ACID describes the guarantees a transaction provides: Atomicity, Consistency, Isolation, and Durability. A transaction groups statements so they succeed or fail as one unit.

STRONG ANSWER

Atomicity means all statements in a transaction commit together or none do — a failed transfer rolls back. **Consistency** means the database moves from one valid state to another, respecting constraints. **Isolation** means concurrent transactions don't see each other's uncommitted changes, governed by the isolation level. **Durability** means once committed, the data survives crashes, typically via a **write-ahead log**. The classic example is a bank transfer: debiting one account and crediting another must both happen or neither — I wrap them in **BEGIN/COMMIT** so a failure triggers **ROLLBACK**.

CODE · SQL

```sql
BEGIN;
  UPDATE accounts SET balance = balance - 100
    WHERE id = 1;
  UPDATE accounts SET balance = balance + 100
    WHERE id = 2;
  -- if either fails, neither change persists
COMMIT;
-- on error instead:
-- ROLLBACK;
```

ARCHITECTURE / FLOW

```
A transaction = all-or-nothing unit

  BEGIN ----> [ debit ] --> [ credit ] ----> COMMIT (durable)
                 |              |
                 +---- error ---+----> ROLLBACK (nothing kept)

  A: atomic   C: valid state   I: isolated   D: durable
```

Q80 DATABASES & CACHING

Transaction isolation levels (dirty/non-repeatable/phantom reads)

THEORY

Isolation levels trade consistency for concurrency by controlling which read anomalies are allowed. The four standard levels are Read Uncommitted, Read Committed, Repeatable Read, and Serializable.

STRONG ANSWER

A **dirty read** is reading another transaction's uncommitted data; a **non-repeatable read** is getting different values for the same row read twice; a **phantom read** is new rows appearing in a repeated range query. **Read Uncommitted** allows all three, **Read Committed** (Postgres default) blocks dirty reads, **Repeatable Read** also blocks non-repeatable reads, and **Serializable** blocks all anomalies as if transactions ran one at a time. Higher isolation costs **concurrency and locking**. I default to Read Committed and bump to Serializable only for logic that's sensitive to concurrent updates, like inventory reservations.

CODE · SQL

```sql
-- Set per-transaction isolation
BEGIN ISOLATION LEVEL SERIALIZABLE;
  SELECT qty FROM stock WHERE sku = 'A1';
  UPDATE stock SET qty = qty - 1 WHERE sku = 'A1';
COMMIT;

-- Postgres default is READ COMMITTED
SHOW transaction_isolation;
```

ARCHITECTURE / FLOW

```
Level            dirty  non-repeat  phantom
---------------- -----  ----------  -------
Read Uncommitted   yes      yes        yes
Read Committed      no      yes        yes
Repeatable Read     no       no        yes*
Serializable        no       no         no

  higher level = safer reads, lower concurrency
  (*Postgres RR also blocks phantoms)
```

Q81 DATABASES & CACHING

Normalization vs denormalization

THEORY

Normalization splits data into related tables to remove redundancy and update anomalies. Denormalization deliberately duplicates data to speed up reads.

STRONG ANSWER

Normalization organizes data so each fact lives in one place, reducing redundancy and preventing **update anomalies** — typically aiming for **3rd normal form**. The cost is that reads need **joins**, which can be slow at scale. **Denormalization** copies or precomputes data — like storing an order's total or a cached author name — to avoid joins and speed reads, at the cost of having to keep duplicates in sync. I normalize the write path for integrity, then selectively denormalize hot read paths, often via **materialized views** or duplicated columns. It's a classic read-vs-write tradeoff.

CODE · SQL

```sql
-- Normalized: author name lives once in authors
SELECT b.title, a.name
FROM books b JOIN authors a ON a.id = b.author_id;

-- Denormalized: cache author_name on books (no join)
ALTER TABLE books ADD COLUMN author_name text;
SELECT title, author_name FROM books;  -- fast read
```

CODE · SQL (continued)

```sql
-- Or precompute with a materialized view
CREATE MATERIALIZED VIEW book_list AS
SELECT b.title, a.name FROM books b JOIN authors a ON a.id = b.author_id;
```

ARCHITECTURE / FLOW

```
Normalized (3NF)              Denormalized
  books -> author_id            books { title, author_name }
  authors { id, name }

  no duplication                duplicated name
  joins on read                 no join, fast read
  easy updates                  must sync copies
  -> write integrity            -> read performance
```

Q82 DATABASES & CACHING

SQL joins (inner, left, right, full)

THEORY

Joins combine rows from two tables based on a matching condition. The join type determines what happens to rows that have no match on the other side.

STRONG ANSWER

An **INNER JOIN** returns only rows that match in both tables. A **LEFT JOIN** keeps all rows from the left table and fills NULLs where the right has no match — useful to find unmatched rows. A **RIGHT JOIN** is the mirror, keeping all right-table rows. A **FULL OUTER JOIN** keeps unmatched rows from both sides. In practice I use INNER and LEFT joins the most; a LEFT JOIN with a **WHERE right.id IS NULL** is the standard way to find records that lack a related row, like customers with no orders.

CODE · SQL

```sql
-- INNER: only matching pairs
SELECT u.name, o.id FROM users u
JOIN orders o ON o.user_id = u.id;

-- LEFT: all users, NULL where no order
SELECT u.name, o.id FROM users u
LEFT JOIN orders o ON o.user_id = u.id;

-- LEFT + IS NULL: users with no orders
SELECT u.name FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;
```

ARCHITECTURE / FLOW

```
  users A        orders B
   ( A only ) ( A & B ) ( B only )

  INNER  -> only the overlap (A & B)
  LEFT   -> A only + overlap
  RIGHT  -> overlap + B only
  FULL   -> A only + overlap + B only

  unmatched side filled with NULLs
```

Q83 DATABASES & CACHING

Finding slow queries with EXPLAIN / EXPLAIN ANALYZE

THEORY

EXPLAIN shows the planner's chosen execution plan and cost estimates. EXPLAIN ANALYZE actually runs the query and reports real timing and row counts.

STRONG ANSWER

EXPLAIN prints the query plan — join order, whether it uses an **index scan** or a **sequential scan**, and estimated cost — without running the query. **EXPLAIN ANALYZE** runs it and shows **actual time** and **actual rows** per node, which I compare against the estimates. The red flags I look for are a **Seq Scan** on a large table, a big gap between estimated and actual rows (stale statistics), and expensive **nested-loop joins** over many rows. The usual fixes are adding the right **index**, running **ANALYZE** to refresh stats, or rewriting the query. I use buffers too to spot heavy disk reads.

CODE · SQL

```sql
-- Plan only (no execution)
EXPLAIN SELECT * FROM orders WHERE customer_id = 7;

-- Run it and show real timings + buffers
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 7;

-- After adding an index, re-check the plan
CREATE INDEX idx_orders_cust ON orders (customer_id);
ANALYZE orders;  -- refresh planner statistics
```

ARCHITECTURE / FLOW

```
Reading a plan (bottom-up)

  Seq Scan on orders   (cost=0..18334  rows=1  actual=210ms)
     Filter: customer_id = 7        <- RED FLAG: full scan

  After index:
  Index Scan using idx_orders_cust  (actual=0.3ms)
     Index Cond: customer_id = 7    <- fast, targeted

  watch: Seq Scan, est vs actual rows, nested loops
```

Q84 DATABASES & CACHING

Connection pooling — why it matters

THEORY

Opening a database connection is expensive — TCP handshake, authentication, and backend process startup. A connection pool reuses a fixed set of open connections across many requests.

STRONG ANSWER

Each new database connection costs a **handshake, auth, and a backend process**, and databases like Postgres cap total connections. Under load, opening a connection per request **exhausts the server** and adds latency. A **connection pool** keeps a bounded set of warm connections that requests **borrow and return**, so you serve thousands of requests with maybe 20 connections. In serverless or high-concurrency setups I add an external pooler like **PgBouncer** in transaction mode. The key tuning knob is **pool size** — too small starves requests, too large overwhelms the database.

CODE · JavaScript

```
// Node: pooled Postgres client (pg)
import { Pool } from 'pg';
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,                 // bounded warm connections
  idleTimeoutMillis: 30000
});
// borrow + auto-return per query
const { rows } = await pool.query('SELECT 1');
// PgBouncer sits in front for serverless fan-out
```

ARCHITECTURE / FLOW

```
Without pool: each request opens+closes a connection
  req -> [connect][auth][query][close]  (slow, exhausts DB)

With pool (max=20):
  req1 --\
  req2 ---+--> [ pool of 20 warm conns ] --> Postgres
  reqN --/      borrow ----> use ----> return

  -> low latency, bounded load on the database
```

Q85 DATABASES & CACHING

MongoDB documents and when a document model fits

THEORY

MongoDB stores BSON documents — nested, schema-flexible JSON-like objects — grouped in collections. The document model fits data that is read and written as a single self-contained unit.

STRONG ANSWER

A **document** is a flexible, nested record, so related data that's always accessed together — like an order with its line items and shipping address — lives in **one document** instead of being spread across joined tables. This fits when the access pattern is **aggregate-oriented**, the schema varies between records, or you want to **embed** rather than join. The tradeoffs are no native multi-collection joins (you **$lookup** or denormalize) and watching out for the 16MB document limit and unbounded array growth. I embed data that's read together and **reference** data that's large, shared, or grows without bound.

CODE · JavaScript

```
// One self-contained order document (embed what's read together)
db.orders.insertOne({
  _id: 'o-1001',
  userId: 42,
  status: 'paid',
  items: [ { sku: 'A1', qty: 2 }, { sku: 'B7', qty: 1 } ],
  shipping: { city: 'Pune', zip: '411001' }
```

CODE · JavaScript (continued)

```
});

// Read the whole aggregate in one query (no join)
db.orders.findOne({ _id: 'o-1001' });
// Reference instead of embed when data is shared/unbounded
db.orders.createIndex({ userId: 1, status: 1 });
```

ARCHITECTURE / FLOW

```
Relational                Document (Mongo)
 orders --< items          { order, items: [...],
 orders -- shipping          shipping: {...} }

  joins to reassemble       one read = whole aggregate
  rigid schema              flexible per-document shape

  embed -> read together    reference -> shared/unbounded
```

Q86 DATABASES & CACHING

Redis use cases: caching, sessions, rate limiting

THEORY

Redis is an in-memory key-value store with sub-millisecond latency and rich data types. Its speed and atomic operations make it ideal for caching, session storage, and counters.

STRONG ANSWER

For **caching**, I store expensive query or computation results under a key with a **TTL** so reads hit memory instead of the database. For **sessions**, Redis holds session state with an expiry, giving stateless app servers a shared, fast store. For **rate limiting**, I use an **atomic INCR** on a per-user key with an **EXPIRE** window to count requests and reject over the limit. Redis also does pub/sub, queues, and leaderboards via sorted sets. The thing to remember is it's **in-memory**, so I plan for eviction policies and treat it as a cache or ephemeral store, not the sole source of truth.

CODE · JavaScript

```
# Cache with TTL (expire in 300s)
SET user:42:profile "{...json...}" EX 300
GET user:42:profile

# Session storage with expiry
SETEX session:abc123 1800 "{userId:42}"

# Rate limit: 100 requests / 60s window (atomic)
INCR rl:user:42
EXPIRE rl:user:42 60   # set window on first hit
# app rejects when INCR result > 100
```

ARCHITECTURE / FLOW

```
Redis (in-memory, sub-ms)

  Caching     key -> value (EX ttl)   miss -> load from DB
  Sessions    session:id -> state (SETEX expiry)
  Rate limit  INCR counter + EXPIRE window -> reject > N

  app --> Redis (hit, fast) --> DB only on miss
  ephemeral store: use eviction + TTLs, not source of truth
```

Q87 DATABASES & CACHING

Cache invalidation strategies (TTL, write-through, cache-aside)

THEORY

Caching speeds reads but risks serving stale data, so invalidation strategy is key. Common approaches are TTL expiry, cache-aside (lazy loading), and write-through.

STRONG ANSWER

Cache-aside (lazy loading) is the most common: the app checks the cache, and on a **miss** loads from the DB and populates the cache. **Write-through** writes to the cache and DB together so the cache stays fresh but writes are slower. **TTL** sets an expiry so entries self-evict, bounding staleness without explicit invalidation. On updates I either **delete the key** (let the next read repopulate) or write through. The hard part — Phil Karlton's joke that cache invalidation is one of the two hard problems — is avoiding **stale reads** and the **thundering herd** when a hot key expires.

CODE · JavaScript

```javascript
// Cache-aside (lazy load + delete on write)
async function getUser(id) {
  const hit = await redis.get(`user:${id}`);
  if (hit) return JSON.parse(hit);
  const row = await db.query('SELECT * FROM users WHERE id=$1', [id]);
  await redis.set(`user:${id}`, JSON.stringify(row), 'EX', 300);
  return row;
}
async function updateUser(id, data) {
  await db.query('UPDATE users SET ... WHERE id=$1', [id]);
  await redis.del(`user:${id}`);   // invalidate, next read reloads
}
```

ARCHITECTURE / FLOW

```
Cache-aside (read):
  app -> cache? --hit--> return
              \--miss--> DB -> fill cache -> return

Write-through (write):
  app -> write cache + DB together (fresh, slower write)

Invalidate-on-write:
  app -> write DB -> DEL key -> next read repopulates

  TTL bounds staleness; beware stale reads + thundering herd
```

Q88 DATABASES & CACHING

pgvector and HNSW indexing for semantic search (tie to resume)

THEORY

pgvector is a Postgres extension that stores embedding vectors and computes similarity between them. HNSW is a graph-based approximate-nearest-neighbor index that makes vector search fast at scale.

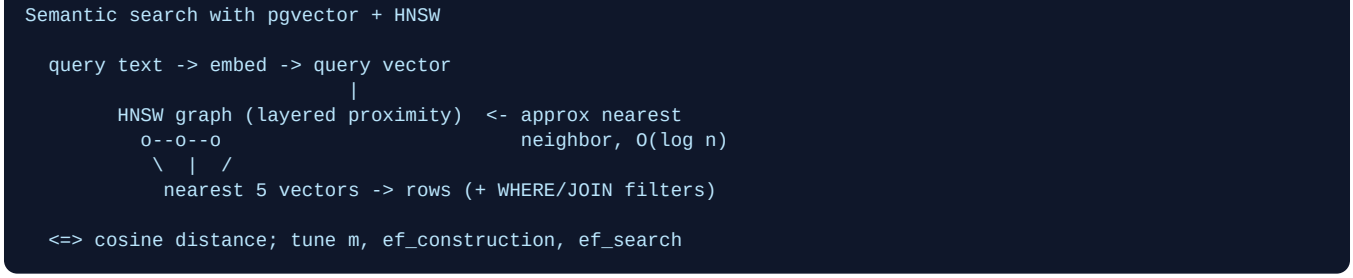
STRONG ANSWER

On my projects I used **pgvector** to keep embeddings right next to relational data, so semantic search and SQL filters live in one query. Each row stores a **vector** column, and I search with a distance operator like **cosine** (`<=>`) to find the nearest embeddings to a query vector. A flat scan is O(n), so I add an **HNSW index** — a layered proximity graph that gives fast **approximate nearest neighbor** lookups with tunable recall via `ef_search`. The big win is combining vector similarity with normal **WHERE** filters and joins in plain SQL, instead of running a separate vector database. I tune `m` and `ef_construction` to trade build cost for recall.

CODE · SQL

```sql
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE docs ADD COLUMN embedding vector(1536);

-- HNSW index for fast approximate nearest-neighbor search
CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Semantic search + relational filter in one query
SET hnsw.ef_search = 40;
SELECT id, title FROM docs
WHERE tenant_id = 7
ORDER BY embedding <=> '[0.12, ...]'::vector
LIMIT 5;
```

ARCHITECTURE / FLOW



Q89 DATABASES & CACHING

Database migrations and versioning

THEORY

Migrations are versioned, ordered scripts that evolve the database schema in a repeatable, reviewable way. They keep schema changes in source control alongside code.

STRONG ANSWER

A **migration** is a version-controlled script describing a schema change — add a column, create an index — applied in order and tracked in a **migrations table** so each runs once. Tools like **Flyway**, **Liquibase**, or framework migrators (Prisma, Knex, Rails) handle ordering and apply state. The discipline I follow is making migrations **forward-only and additive** where possible, with a **rollback/down** path, and never editing an already-applied migration. For zero-downtime I use **expand-then-contract**: add the new column, backfill, deploy code, then drop the old one. Big backfills run in **batches** to avoid long locks.

CODE · SQL

```sql
-- migrations/0007_add_phone_to_users.sql (forward)
ALTER TABLE users ADD COLUMN phone text;
CREATE INDEX CONCURRENTLY idx_users_phone ON users (phone);

-- down / rollback
-- DROP INDEX IF EXISTS idx_users_phone;
-- ALTER TABLE users DROP COLUMN phone;

-- Expand-contract for zero downtime:
-- 1) add new col  2) backfill in batches
-- 3) deploy code reading new col  4) drop old col
```

ARCHITECTURE / FLOW

```
Versioned migrations in source control

  0005_init.sql
  0006_add_orders.sql      applied -> recorded in
  0007_add_phone.sql  <--  schema_migrations table

  forward-only, ordered, run once each

  Zero downtime: expand -> backfill -> deploy -> contract
```

Q90 DATABASES & CACHING

Replication and sharding basics (scaling reads/writes)

THEORY

Replication copies data to multiple nodes to scale reads and provide failover. Sharding partitions data across nodes to scale writes and total dataset size.

STRONG ANSWER

Replication keeps one **primary** for writes and one or more **replicas** that copy its data, so you spread **read** traffic across replicas and fail over if the primary dies. Replicas can lag, giving **eventual consistency** on reads, so latency-sensitive reads go to the primary. **Sharding** splits data horizontally by a **shard key** — say user_id ranges or a hash — so each node owns a slice, scaling **writes** and storage beyond one machine. The tradeoffs are cross-shard queries and rebalancing, so the **shard key** choice is critical to avoid hot spots. In short, replicas scale reads and resilience, shards scale writes and size.

CODE · SQL

```sql
-- Replication: route reads to a replica, writes to primary
-- (app-level or via a proxy like pgpool / read-replica URL)
--   WRITE -> primary
--   READ  -> replica (may lag slightly)

-- Sharding: pick a shard by key (hash of user_id)
-- shard = hash(user_id) % N
-- e.g. SELECT * FROM users_shard_2 WHERE user_id = 105;

-- watch: cross-shard joins, rebalancing, hot shard keys
```

ARCHITECTURE / FLOW

```
Replication (scale reads / HA)
            writes
  client ---------> [ PRIMARY ]
  reads  <-------- [ replica ] [ replica ]   (may lag)
```

ARCHITECTURE / FLOW (continued)

```
Sharding (scale writes / size)
  key=hash(user_id) % N
  shard0: users 0-3M   shard1: 3-6M   shard2: 6-9M

  replicas -> read scale   shards -> write+storage scale
```

7 Auth, Security & Payments

JWT rotation, OAuth, rate limiting, Stripe / PayPal / Razorpay webhooks.

Q91–Q105

Q91 AUTH, SECURITY & PAYMENTS

What is the difference between authentication and authorization?

THEORY

Authentication verifies who you are; authorization decides what you are allowed to do. Authentication always comes first, then authorization gates access to resources.

STRONG ANSWER

Authentication answers 'who are you?' by validating credentials like a password, an **OAuth** token, or a **JWT**. **Authorization** answers 'what can you do?' by checking the authenticated identity against **roles**, **scopes**, or permissions. In a request pipeline I authenticate first to establish the user, then run authorization middleware to enforce **access control** on each route. A common shorthand is **AuthN** for authentication and **AuthZ** for authorization. Confusing the two leads to bugs like trusting a valid token without checking whether that user owns the resource.

CODE · JavaScript

```
// AuthN: verify identity, then AuthZ: verify permission
function authN(req, res, next) {
  const user = verifyJwt(req.headers.authorization);
  if (!user) return res.status(401).send('Unauthenticated');
  req.user = user;
  next();
}

function authZ(role) {
  return (req, res, next) => {
    if (!req.user.roles.includes(role))
      return res.status(403).send('Forbidden');
    next();
  };
}

app.delete('/orders/:id', authN, authZ('admin'), deleteOrder);
```

ARCHITECTURE / FLOW

```
Request --> [AuthN: who?] --401--> reject
                |
             identity
                v
            [AuthZ: allowed?] --403--> reject
                |
              yes
                v
            Handler / Resource
```

Q92 AUTH, SECURITY & PAYMENTS

Explain JWT structure (header.payload.signature) and how signing works.

THEORY

A JWT is three Base64URL-encoded parts joined by dots: header, payload, and signature. The signature is a keyed hash over the header and payload that lets the server detect tampering.

STRONG ANSWER

A **JWT** has three segments: a **header** declaring the algorithm (like HS256 or RS256), a **payload** holding **claims** such as sub, exp, and roles, and a **signature**. The signature is computed as HMAC or an RSA signature over `base64url(header) + '.' + base64url(payload)` using a **secret** or **private key**. On each request the server recomputes the signature and compares it; if it matches, the token is authentic and untampered. The payload is only encoded, not encrypted, so I never put secrets in it. With asymmetric algorithms (**RS256**) the private key signs and a public key verifies, which is ideal for distributed verification.

CODE · TypeScript

```typescript
import jwt from 'jsonwebtoken';

// header={alg:'HS256',typ:'JWT'} payload=claims, signed with secret
const token = jwt.sign(
  { sub: 'user_123', roles: ['user'] },
  process.env.JWT_SECRET,
  { algorithm: 'HS256', expiresIn: '15m' }
);

// verify recomputes the signature and checks exp
try {
  const claims = jwt.verify(token, process.env.JWT_SECRET);
  console.log(claims.sub); // user_123
} catch (e) {
  console.error('invalid or expired token');
}
```

ARCHITECTURE / FLOW

```
JWT = header . payload . signature

  header  : {alg:HS256, typ:JWT}  -> base64url
  payload : {sub, exp, roles}     -> base64url
  sig     : HMAC_SHA256(h.p, secret)

  verify: recompute HMAC(h.p, secret) == sig ?  AND exp not passed
```

Q93 AUTH, SECURITY & PAYMENTS

What is the difference between access tokens and refresh tokens, and why rotate them?

THEORY

Access tokens are short-lived credentials sent on every request; refresh tokens are long-lived credentials used only to mint new access tokens. Rotation issues a fresh refresh token on each use to limit theft damage.

STRONG ANSWER

An **access token** is short-lived (minutes) and presented on each API call, so if leaked it expires quickly. A **refresh token** is long-lived and used solely against a token endpoint to obtain a new access token without re-login. With **refresh token rotation**, every refresh returns a new refresh token and invalidates the old one, so a stolen token becomes useless after the legitimate client refreshes. I also add **reuse detection**: if an already-used refresh token appears again, I revoke the whole token family, signaling theft. Refresh tokens are stored server-side or as a hashed value so they can be revoked.

CODE · TypeScript

```
async function refresh(oldToken) {
  const row = await db.refreshTokens.find(hash(oldToken));
  if (!row) throw new Error('unknown token');
  if (row.used) {                // reuse detection
    await db.revokeFamily(row.familyId);
    throw new Error('token reuse -> family revoked');
  }
  await db.markUsed(row.id);
  const newRefresh = randomToken();
  await db.saveRefresh(hash(newRefresh), row.familyId);
  return { access: signAccess(row.userId), refresh: newRefresh };
}
```

ARCHITECTURE / FLOW

```
Login --> access(15m) + refresh(30d)

  access expires ---> POST /refresh (refresh token)
                          |
                      rotate: issue NEW refresh, revoke old
                          |
                      reused old token? --> revoke entire family
```

Q94 AUTH, SECURITY & PAYMENTS

Where should tokens be stored: httpOnly cookie vs localStorage?

THEORY

localStorage is readable by any JavaScript so it is exposed to XSS, while an httpOnly cookie is invisible to scripts. Cookies are the safer default but require CSRF protection.

STRONG ANSWER

localStorage is convenient but any injected script can read it, so an **XSS** flaw means full token theft. An **httpOnly** cookie cannot be read by JavaScript, which neutralizes XSS token exfiltration, and adding **Secure** and **SameSite** flags hardens it further. The tradeoff is that cookies are sent automatically, exposing you to **CSRF**, which I counter with **SameSite=Strict/Lax** plus a CSRF token or origin check. My usual choice is the refresh token in an httpOnly, Secure, SameSite cookie and the short-lived access token held in memory. I avoid persisting long-lived secrets in localStorage entirely.

CODE · TypeScript

```
// Server sets refresh token as a hardened cookie
res.cookie('refresh', token, {
  httpOnly: true,   // not visible to document.cookie / JS
  secure: true,     // HTTPS only
  sameSite: 'strict', // mitigates CSRF
  path: '/auth/refresh',
  maxAge: 30 * 24 * 60 * 60 * 1000
```

CODE · TypeScript (continued)

```
});

// Access token kept in memory (a module variable), never localStorage
let accessToken = null;
export const setAccess = (t) => { accessToken = t; };
export const getAccess = () => accessToken;
```

ARCHITECTURE / FLOW

```
                | XSS readable | CSRF risk | Best for
localStorage    |     YES      |    no     | avoid for tokens
JS-memory var   |  only if XSS |    no     | access token
httpOnly cookie |     NO       |   YES*    | refresh token

* mitigate CSRF with SameSite + token/origin check
```

Q95 AUTH, SECURITY & PAYMENTS

Explain the OAuth 2.0 authorization-code flow.

THEORY

The authorization-code flow redirects the user to the provider to log in, returns a short-lived code to the app, then the app's backend exchanges that code for tokens. It keeps tokens off the front channel and supports PKCE for public clients.

STRONG ANSWER

In the **authorization-code flow** the app redirects the user to the **authorization server** with client_id, redirect_uri, scope, and a random **state**. After login and consent the provider redirects back with a one-time **authorization code**. The app's **backend** then POSTs that code with its **client_secret** to the **token endpoint** and receives an access token and refresh token over the back channel. Keeping the exchange server-side means tokens never appear in the browser URL. For SPAs and mobile I add **PKCE**, sending a code_challenge up front and a code_verifier at exchange, so an intercepted code is useless. The **state** parameter protects against CSRF on the callback.

CODE · TypeScript

```
// Step 1: redirect user to authorize (with PKCE + state)
const url = `${AUTH}/authorize?response_type=code` +
  `&client_id=${id}&redirect_uri=${cb}&scope=openid%20email` +
  `&state=${state}&code_challenge=${challenge}&code_challenge_method=S256`;
res.redirect(url);

// Step 2: callback -> exchange code server-side
app.get('/callback', async (req, res) => {
  if (req.query.state !== savedState) throw new Error('bad state');
  const tokens = await fetch(`${AUTH}/token`, {
    method: 'POST',
    body: form({ grant_type: 'authorization_code', code: req.query.code,
      redirect_uri: cb, client_id: id, code_verifier: verifier })
  }).then(r => r.json());
  // tokens.access_token, tokens.refresh_token
});
```

ARCHITECTURE / FLOW

```
User      App(server)     Auth Server
 |--login?->|                 |
 |---------- redirect ------>|  (client_id,scope,state,PKCE)
 |<--------- consent ---------|
 |--- code + state --------->|(App callback)
```

ARCHITECTURE / FLOW (continued)

```
|          |--- code+secret+verifier -->| /token
|          |<--- access+refresh tokens --|
```

Q96 AUTH, SECURITY & PAYMENTS

How does password hashing with bcrypt work (salt, work factor)?

THEORY

bcrypt is a deliberately slow, adaptive hash that embeds a random salt and a cost factor. The salt defeats rainbow tables and the work factor makes brute force expensive.

STRONG ANSWER

I never store raw passwords; I store a **bcrypt** hash. bcrypt generates a unique random **salt** per password and folds it into the output, so identical passwords yield different hashes and **rainbow tables** are useless. The **work factor** (cost / rounds) sets how many iterations it runs, so I tune it (around 10 to 12) to take tens of milliseconds and raise it as hardware improves. The salt and cost are encoded in the resulting hash string, so verification just re-runs bcrypt and compares. Because it is intentionally **slow**, large-scale offline cracking becomes impractical. Modern alternatives like **argon2** are also good choices.

CODE · TypeScript

```typescript
import bcrypt from 'bcrypt';

const COST = 12; // work factor; ~2^12 iterations

async function hashPassword(plain) {
  // salt is generated and embedded in the returned hash
  return bcrypt.hash(plain, COST);
}

async function checkPassword(plain, stored) {
  return bcrypt.compare(plain, stored); // re-derives salt + cost
}

// stored hash format: $2b$12$<22-char-salt><31-char-hash>
//                     alg cost  salt        digest
```

ARCHITECTURE / FLOW

```
password + random salt --> bcrypt(cost=12) --> hash

 stored = $2b$12$ + salt + digest   (salt & cost embedded)

 login:  bcrypt(input, salt-from-stored, cost) == digest ?

 higher cost = slower = harder brute force
```

Q97 AUTH, SECURITY & PAYMENTS

How do you implement rate limiting with Redis (sliding window / token bucket)?

THEORY

Redis provides fast atomic counters shared across servers, ideal for distributed rate limiting. Sliding window and token bucket are two common algorithms that smooth bursts.

STRONG ANSWER

I use **Redis** because its atomic operations give a single source of truth across many app instances. A simple **fixed window** uses INCR with EXPIRE, but it allows bursts at window edges, so I prefer a **sliding window log** with a sorted set keyed by user or IP, trimming timestamps older than the window. A **token bucket** stores a token count and last-refill time, refilling at a steady rate and allowing controlled bursts up to the bucket size. To make the read-modify-write atomic I run it as a **Lua script** so concurrent requests cannot race. On exceeding the limit I return **429** with a Retry-After header.

CODE · TypeScript

```
// Sliding-window log with a Redis sorted set (run via Lua for atomicity)
async function allow(key, limit, windowMs) {
  const now = Date.now();
  const m = redis.multi();
  m.zremrangebyscore(key, 0, now - windowMs); // drop old entries
  m.zadd(key, now, `${now}-${Math.random()}`);
  m.zcard(key);                               // count in window
  m.pexpire(key, windowMs);
  const res = await m.exec();
  const count = res[2][1];
  return count <= limit; // false -> respond 429 Retry-After
}
```

ARCHITECTURE / FLOW

```
Sliding window (sorted set of timestamps):
  |---- window ----|
  x  x   x  x   x  now   count > limit ? -> 429

Token bucket:
  [ * * * * . . ]  refill r/sec, take 1 per request
  empty bucket -> reject
```

Q98 AUTH, SECURITY & PAYMENTS

What is CORS and how do you configure it?

THEORY

CORS is a browser security mechanism that controls which origins may read responses from your API via HTTP headers. The server opts specific origins in; the browser enforces the rules.

STRONG ANSWER

CORS (Cross-Origin Resource Sharing) relaxes the browser's **same-origin policy** so a page on one origin can call an API on another, but only if the server allows it. The server sets **Access-Control-Allow-Origin** and related headers; for non-simple requests the browser first sends a **preflight** OPTIONS request and checks Allow-Methods and Allow-Headers. To send cookies I set **Access-Control-Allow-Credentials: true**, which forbids the wildcard origin, so I echo a specific allowlisted origin. It is enforced by the **browser**, not the server, so it is not a substitute for real authorization. I keep a strict origin allowlist rather than reflecting any origin.

CODE · TypeScript

```
import cors from 'cors';

const allow = ['https://app.example.com', 'https://admin.example.com'];

app.use(cors({
  origin: (origin, cb) =>
    !origin || allow.includes(origin)
```

CODE · TypeScript (continued)

```
      ? cb(null, true)
      : cb(new Error('CORS blocked')),
  credentials: true,            // allow cookies; no wildcard origin
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  maxAge: 600                   // cache preflight 10 min
}));
```

ARCHITECTURE / FLOW

```
Browser (app.example.com) --> API (api.example.com)

 preflight: OPTIONS --> server
   <-- Allow-Origin: app.example.com
       Allow-Methods: GET,POST...  Allow-Credentials: true
 actual request proceeds only if headers permit
 (enforced by browser, not the server)
```

Q99 AUTH, SECURITY & PAYMENTS

Explain common web attacks: XSS, CSRF, SQL injection and their defenses.

THEORY

XSS injects malicious scripts into pages, CSRF tricks a logged-in browser into making unwanted requests, and SQL injection smuggles SQL through inputs. Each has a well-known structural defense.

STRONG ANSWER

XSS runs attacker JavaScript in a victim's page; I defend with **output encoding**, a strict **Content-Security-Policy**, and framework escaping rather than dangerouslySetInnerHTML. **CSRF** abuses a user's existing session to forge state-changing requests; defenses are **SameSite** cookies, **anti-CSRF tokens**, and verifying the Origin header. **SQL injection** lets input alter a query; I always use **parameterized queries** or an ORM and never concatenate strings. Cross-cutting habits help too: validate and allowlist inputs, apply least privilege to the DB user, and set security headers. The theme is to separate **data from code** at every boundary.

CODE · TypeScript

```
// SQL injection: parameterized query, never string concat
await db.query('SELECT * FROM users WHERE email = $1', [email]); // safe
// BAD: `SELECT * FROM users WHERE email = '${email}'`

// XSS: escape output / set CSP
res.setHeader('Content-Security-Policy', "default-src 'self'");
// React auto-escapes {userInput}; avoid dangerouslySetInnerHTML

// CSRF: SameSite cookie + token check
res.cookie('sid', id, { httpOnly: true, sameSite: 'strict' });
if (req.headers['x-csrf-token'] !== session.csrf)
  return res.status(403).send('CSRF');
```

ARCHITECTURE / FLOW

```
Attack       | Vector                  | Defense
XSS          | injected <script>       | encode output + CSP
CSRF         | forged cross-site POST  | SameSite + CSRF token
SQLi         | input alters query      | parameterized queries

 principle: keep DATA separate from CODE
```

Q100 AUTH, SECURITY & PAYMENTS

What are HTTPS/TLS basics and why use them?

THEORY

HTTPS is HTTP over TLS, which encrypts traffic, authenticates the server, and protects integrity. It relies on certificates and an asymmetric handshake that establishes a fast symmetric session key.

STRONG ANSWER

HTTPS wraps HTTP in **TLS** to provide **confidentiality**, **integrity**, and server **authentication**. During the **handshake** the server presents a **certificate** signed by a trusted **CA**; the client verifies the chain, then they agree on a shared **session key** (modern TLS 1.3 uses ephemeral Diffie-Hellman for **forward secrecy**). After that, bulk data uses fast **symmetric encryption** like AES. Without TLS, credentials and tokens travel in plaintext and are vulnerable to **man-in-the-middle** interception. I enforce it with **HSTS**, redirect HTTP to HTTPS, and automate certificates via Let's Encrypt.

CODE · TypeScript

```typescript
// Force HTTPS + HSTS in Express
app.use((req, res, next) => {
  if (req.headers['x-forwarded-proto'] !== 'https')
    return res.redirect(`https://${req.headers.host}${req.url}`);
  res.setHeader('Strict-Transport-Security',
    'max-age=31536000; includeSubDomains; preload');
  next();
});

// TLS 1.3 preferred; certs auto-renewed (e.g. Let's Encrypt / certbot)
```

ARCHITECTURE / FLOW

```
Client                        Server
  |-- ClientHello ------------>|
  |<-- ServerHello + cert -----|  (cert signed by trusted CA)
  |-- verify chain, key exch ->|  (ephemeral DH = forward secrecy)
  |== symmetric session key ===|
  |== AES-encrypted HTTP ======|  confidentiality + integrity
```

Q101 AUTH, SECURITY & PAYMENTS

Walk through the Stripe PaymentIntent flow (client and server).

THEORY

A PaymentIntent is a server-created object tracking a payment's lifecycle. The server creates it and returns a client_secret; the client confirms the payment with that secret without exposing the API key.

STRONG ANSWER

On the server I create a **PaymentIntent** with the amount, currency, and metadata, which returns a **client_secret**. The browser uses **Stripe.js** / Elements with that secret to **confirm the payment**, so raw card data goes straight to Stripe and never touches my server, keeping me out of strict **PCI** scope. Confirmation may trigger **3D Secure** (SCA) which Stripe handles. I treat the client response as a hint only and rely on the **webhook** `payment_intent.succeeded` as the source of truth before fulfilling the order. I pass an **idempotency key** on creation so retries do not create duplicate intents.

CODE · TypeScript

```typescript
// Server: create the PaymentIntent
const intent = await stripe.paymentIntents.create(
  { amount: 4999, currency: 'usd', metadata: { orderId } },
  { idempotencyKey: `pi_${orderId}` }
);
res.json({ clientSecret: intent.client_secret });

// Client: confirm with Stripe.js (card never hits our server)
const { error, paymentIntent } = await stripe.confirmCardPayment(
  clientSecret,
  { payment_method: { card: cardElement } }
);
if (error) showError(error.message);
// fulfillment happens on webhook, not here
```

ARCHITECTURE / FLOW

```
Server                Browser                Stripe
  |-- create PI ------------------------------>|
  |<-- client_secret ----|                      |
  |                      |-- confirm(card) ----->|
  |                      |<-- 3DS / result ------|
  |<====== webhook payment_intent.succeeded =====|
  |-- fulfill order (source of truth) -->|
```

Q102 AUTH, SECURITY & PAYMENTS

How do you verify webhook signatures, and why does it matter?

THEORY

Webhook endpoints are public URLs, so any actor could POST fake events. Signature verification proves the payload came from the provider and was not altered, using a shared signing secret.

STRONG ANSWER

In my **webhook-driven order lifecycle** the endpoint is publicly reachable, so I must prove each event is genuinely from the provider. Stripe signs the request with a **signing secret** and sends a **Stripe-Signature** header containing a timestamp and HMAC; I verify it against the **raw request body** using `stripe.webhooks.constructEvent`. Verification must run on the **raw bytes** before any JSON middleware parses them, or the signature will not match. The included timestamp lets me reject **replayed** events outside a tolerance window. Only after a verified `payment_intent.succeeded` do I mark the order paid, and I make the handler **idempotent** since providers retry.

CODE · TypeScript

```typescript
// Use raw body for this route, NOT express.json()
app.post('/webhooks/stripe',
  express.raw({ type: 'application/json' }),
  (req, res) => {
    let event;
    try {
      event = stripe.webhooks.constructEvent(
        req.body,                        // raw bytes
        req.headers['stripe-signature'],
        process.env.STRIPE_WEBHOOK_SECRET // signing secret
      );
    } catch (err) {
      return res.status(400).send(`bad signature: ${err.message}`);
    }
    if (event.type === 'payment_intent.succeeded') markPaid(event);
    res.json({ received: true });
  });
```

ARCHITECTURE / FLOW

```
Stripe --POST event + Stripe-Signature(t,v1=HMAC)--> /webhooks

  server: HMAC_SHA256(t + '.' + rawBody, signingSecret)
          == v1 ?  AND |now - t| < tolerance ?
              |
          yes-> trust event -> update order (idempotent)
          no -> 400 reject (forged or replayed)
```

Q103 AUTH, SECURITY & PAYMENTS

How do idempotency keys prevent double charges in payments?

THEORY

Network retries and double-clicks can submit the same payment twice. An idempotency key lets the server (or provider) recognize a retry and return the original result instead of charging again.

STRONG ANSWER

Payments are not naturally **idempotent**, so a retried POST after a timeout could charge a customer twice. I attach a unique **idempotency key** per logical operation; Stripe stores the first response under that key and replays it for any retry, so at most one charge happens. On my own side I persist the key with a unique constraint and, on a duplicate, return the stored result rather than re-executing. The key must be **stable across retries** of the same intent but **unique across distinct payments**, often derived from the order id plus an attempt token. This makes the operation safe to retry under network failures and **at-least-once** webhook delivery.

CODE · TypeScript

```typescript
// Provider-level idempotency: Stripe dedupes by this key
await stripe.paymentIntents.create(
  { amount, currency: 'usd' },
  { idempotencyKey: key } // same key on retry -> same result
);

// App-level: unique constraint guards the DB
async function charge(orderId, key) {
  try {
    await db.payments.insert({ orderId, key, status: 'pending' });
  } catch (e) {
    if (e.code === 'UNIQUE_VIOLATION')        // retry detected
      return db.payments.findByKey(key);      // return original
    throw e;
  }
  return doCharge(orderId, key);
}
```

ARCHITECTURE / FLOW

```
Client click x2 / network retry
   req(key=abc) --> server
   req(key=abc) --> server (retry)

  first key=abc  -> execute charge, store result
  later key=abc  -> found -> return SAME result (no 2nd charge)
  key=xyz        -> new operation -> new charge
```

Q104 AUTH, SECURITY & PAYMENTS

How do you model the order lifecycle as a state machine?

THEORY

Modeling orders as a state machine makes only valid transitions possible and centralizes side effects. Each state change is explicit, auditable, and guarded against illegal jumps.

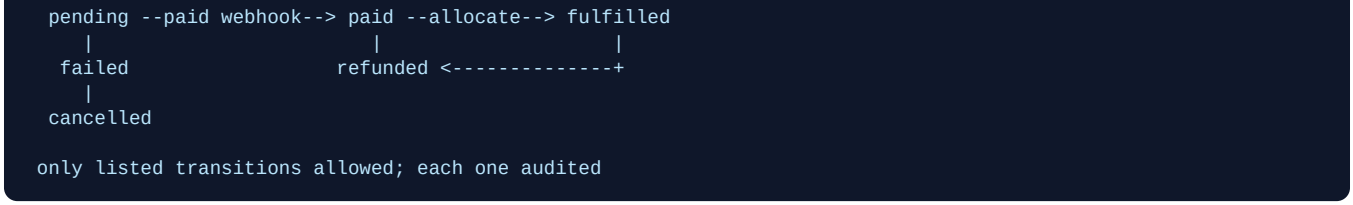
STRONG ANSWER

I model the order as a **finite state machine** with states like **pending**, **paid**, **fulfilled**, plus **failed**, **refunded**, and **cancelled**. Each transition has explicit **guards**: only the verified `payment_intent.succeeded` webhook moves pending to paid, and only inventory allocation moves paid to fulfilled. Encoding the allowed transitions prevents illegal jumps such as fulfilling an unpaid order, and I persist every change to an **audit log**. Because webhooks are **idempotent** and may arrive out of order, transitions are written conditionally (only if the current state allows it). This gives a clear, recoverable lifecycle that is easy to reason about and to drive UI from.

CODE · TypeScript

```
const TRANSITIONS = {
  pending:   ['paid', 'failed', 'cancelled'],
  paid:      ['fulfilled', 'refunded'],
  fulfilled: ['refunded'],
  failed: [], refunded: [], cancelled: []
};

async function transition(orderId, to) {
  const order = await db.orders.get(orderId);
  if (!TRANSITIONS[order.status].includes(to))
    throw new Error(`illegal ${order.status} -> ${to}`);
  // conditional update = idempotent against duplicate webhooks
  await db.orders.update(orderId, { status: to }, { where: { status: order.status } });
  await db.audit.log(orderId, order.status, to);
}
```

ARCHITECTURE / FLOW



Q105 AUTH, SECURITY & PAYMENTS

What is PCI compliance, and how do you avoid storing raw card data?

THEORY

PCI DSS is the security standard for handling cardholder data. The cheapest path to compliance is to never let raw card numbers touch your servers by delegating capture to a tokenizing provider.

STRONG ANSWER

PCI DSS governs how cardholder data is handled, and the scope of audit grows enormously if raw card numbers ever touch my systems. So I never store the **PAN**, CVV, or full track data; instead I let **Stripe.js** / Elements collect the card directly and return a **token** (a payment_method or PaymentIntent reference). I store only that opaque **token** and safe metadata like the last four digits and brand, which keeps me on the minimal **SAQ A** self-assessment. This **tokenization** means a breach of my database exposes no usable card data. I also enforce TLS everywhere, restrict access, and rely on the provider's vault as the system of record for card details.

CODE · TypeScript

```typescript
// Card is tokenized client-side; server stores only the token + safe meta
const pm = await stripe.paymentMethods.create({
  type: 'card',
  card: cardElement   // raw PAN never leaves Stripe.js
});

// Server persists a reference, NOT the card number
await db.customers.update(userId, {
  stripePaymentMethodId: pm.id,  // opaque token
  cardLast4: pm.card.last4,      // safe to store
  cardBrand: pm.card.brand
});
// NEVER store: full PAN, CVV, magnetic stripe / track data
```

ARCHITECTURE / FLOW

```
Browser (card) --> Stripe.js --> token (pm_xxx)
                                    |
     our server stores: pm_xxx, last4, brand   (safe)
     our server NEVER stores: full PAN, CVV, track data

  card data lives in Stripe's PCI vault -> minimal SAQ A scope
```

8 AI, LLM & RAG

Embeddings, semantic search, RAG, streaming, tool calling, vector indexing.

Q106–Q120

Q106 AI, LLM & RAG

What is an LLM and what are tokens?

THEORY

A Large Language Model is a transformer-based neural network trained to predict the next token in a sequence. Tokens are sub-word units, not whole words or characters.

STRONG ANSWER

An **LLM** is a transformer model trained on huge text corpora to predict the **next token** given prior context, which is what lets it generate fluent text. Internally, text is broken into **tokens** by a **tokenizer** (often BPE), where a token is roughly 3-4 characters or about 0.75 words in English. Both pricing and the **context window** are measured in tokens, so understanding tokenization matters for cost and limits. The model is fundamentally a probability distribution over the next token, and **sampling** from that distribution produces the output. As a rule of thumb, 1000 tokens is around 750 words.

CODE · TypeScript

```typescript
// Estimating tokens (rough) and exact counts via a tokenizer lib
import { encoding_for_model } from 'tiktoken';

function roughTokenEstimate(text: string): number {
  return Math.ceil(text.length / 4); // ~4 chars per token
}

function exactTokens(text: string): number {
  const enc = encoding_for_model('gpt-4o');
  const tokens = enc.encode(text);
  enc.free();
  return tokens.length;
}

console.log(roughTokenEstimate('Hello world')); // ~3
console.log(exactTokens('Hello world'));         // exact
```

ARCHITECTURE / FLOW

```
  "unbelievable" --> tokenizer --> ["un", "believ", "able"]

  text -> [tokens] -> LLM -> P(next token | context) -> sample -> token
                                                                   |
                            append to context, repeat <------------+
```

Q107 AI, LLM & RAG

What are embeddings and how do they capture meaning?

THEORY

An embedding is a fixed-length vector of floats that represents text in a high-dimensional space. Semantically similar text maps to nearby vectors.

STRONG ANSWER

An **embedding** is a dense vector (e.g. 1536 dimensions) produced by an embedding model that encodes the **semantic meaning** of a piece of text. The key property is that texts with similar meaning land **close together** in vector space, even if they share no keywords, because the model learned meaning from context during training. This is what powers **semantic search**: we embed both documents and the query, then find the nearest vectors. Embeddings are deterministic for a given model, so you must use the **same model** for indexing and querying. You store these vectors in a vector database like **pgvector** for fast retrieval.

CODE · TypeScript

```typescript
import OpenAI from 'openai';
const openai = new OpenAI();

async function embed(text: string): Promise<number[]> {
  const res = await openai.embeddings.create({
    model: 'text-embedding-3-small', // 1536 dims
    input: text,
  });
  return res.data[0].embedding;
}

const v = await embed('How do I reset my password?');
console.log(v.length); // 1536
// 'reset password' and 'forgot my login' will be near each other
```

ARCHITECTURE / FLOW

```
  high-dim space (projected to 2D):

      "dog"  "puppy"            "car"  "automobile"
        *   *                     *    *

              "bank (river)"          "bank (money)"
                  *                       *

  similar meaning -> small distance between vectors
```

Q108 AI, LLM & RAG

Semantic search vs keyword search?

THEORY

Keyword search matches literal terms (lexical overlap); semantic search matches meaning via embeddings. Hybrid search combines both for the best of each.

STRONG ANSWER

Keyword search (e.g. BM25, Postgres full-text, or LIKE) matches on the literal words, so it is precise for exact terms, IDs, and codes but fails on synonyms or paraphrases. **Semantic search** embeds the query and documents and ranks by vector similarity, so it finds relevant content even with zero shared words, but it can miss exact-match needs like a specific SKU. In practice the strongest approach is **hybrid search**, which fuses keyword and vector scores, often with **Reciprocal Rank Fusion** or a weighted blend. A common pattern is to retrieve a broad candidate set with both methods, then **re-rank** with a cross-encoder for final ordering. Choosing depends on the query type and the cost you can afford.

CODE · TypeScript

```
-- Hybrid: combine pgvector semantic + Postgres full-text keyword
WITH semantic AS (
  SELECT id, 1 - (embedding <=> $1) AS score
  FROM docs ORDER BY embedding <=> $1 LIMIT 20
),
keyword AS (
  SELECT id, ts_rank(tsv, plainto_tsquery($2)) AS score
  FROM docs WHERE tsv @@ plainto_tsquery($2) LIMIT 20
)
SELECT id, COALESCE(s.score,0)*0.6 + COALESCE(k.score,0)*0.4 AS final
FROM semantic s FULL OUTER JOIN keyword k USING (id)
ORDER BY final DESC LIMIT 10;
```

ARCHITECTURE / FLOW

```
  query: "how to cancel subscription"

  keyword  --> matches docs containing "cancel" "subscription"
  semantic --> also matches "end my plan", "stop billing"

  hybrid = fuse(keyword_rank, semantic_rank) -> best coverage
```

Q109 AI, LLM & RAG

Vector similarity (cosine) and vector databases?

THEORY

Cosine similarity measures the angle between two vectors, ignoring magnitude. Vector databases store embeddings and answer nearest-neighbor queries efficiently.

STRONG ANSWER

Cosine similarity measures the cosine of the angle between two vectors, giving a score from -1 to 1 where 1 means identical direction (most similar). It ignores **magnitude** and focuses on direction, which is ideal for text embeddings; on normalized vectors it is equivalent to the **dot product**. A **vector database** like **pgvector**, Pinecone, or Qdrant stores embeddings and supports fast **nearest-neighbor** queries, usually via an approximate index like **HNSW**. In pgvector the `<=>` operator is cosine distance, `<#>` is negative inner product, and `<->` is L2 distance. You pick the distance metric to match how your embedding model was trained.

CODE · TypeScript

```
// Cosine similarity from scratch
function cosine(a: number[], b: number[]): number {
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    na += a[i] * a[i];
    nb += b[i] * b[i];
  }
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
}

// In pgvector (cosine distance = 1 - cosine similarity):
// SELECT id FROM docs ORDER BY embedding <=> $1 LIMIT 5;
```

ARCHITECTURE / FLOW

```
  cosine = cos(theta) = (A . B) / (|A| * |B|)

        A
        ^
       /  theta
      / ).........> B      small theta -> cosine ~ 1 (similar)
     /                     90 degrees   -> cosine = 0 (unrelated)

  pgvector: <=> cosine,  <#> inner product,  <-> L2
```

Q110 AI, LLM & RAG

Describe the RAG pipeline end-to-end.

THEORY

Retrieval-Augmented Generation grounds an LLM in your data by retrieving relevant chunks at query time and injecting them into the prompt. It has an offline ingest phase and an online query phase.

STRONG ANSWER

RAG has two phases. Offline **ingest**: load documents, **chunk** them, **embed** each chunk, and store the vectors plus metadata in a vector store. Online **query**: embed the user's question, **retrieve** the top-k most similar chunks, build a prompt that injects those chunks as context, and have the LLM **generate** a grounded answer with citations. The point is to give the model fresh, private, or domain-specific knowledge it never saw in training, while reducing **hallucinations** by anchoring it to retrieved text. Good systems also add **re-ranking**, metadata filtering, and citation tracking. The quality ceiling is set by retrieval, so if retrieval is wrong the answer will be wrong.

CODE · TypeScript

```typescript
// Online query path (Vercel AI SDK style)
async function rag(question: string) {
  const qVec = await embed(question);
  const chunks = await db.query(
    `SELECT content, source FROM docs
     ORDER BY embedding <=> $1 LIMIT 5`, [toVector(qVec)]);
  const context = chunks.map((c, i) =>
    `[${i + 1}] (${c.source}) ${c.content}`).join('\n\n');
  return streamText({
    model: openai('gpt-4o'),
    system: 'Answer ONLY from context. Cite [n]. Say "I don\'t know" if absent.',
    prompt: `Context:\n${context}\n\nQuestion: ${question}`,
  });
}
```

ARCHITECTURE / FLOW

```
  INGEST (offline):
   docs -> chunk -> embed -> [vector store + metadata]

  QUERY (online):
   question -> embed -> retrieve top-k --> rerank
                                            |
              prompt(context + question) -> LLM -> answer + citations
```

Q111 AI, LLM & RAG

What chunking strategies do you use for documents?

THEORY

Chunking splits documents into retrieval units small enough to be precise but large enough to carry meaning. The strategy should respect document structure and add overlap.

STRONG ANSWER

Chunking splits a document into pieces that get embedded independently, and the size is a key tradeoff: too large dilutes relevance and wastes context, too small loses surrounding meaning. I prefer **structure-aware** splitting, breaking on headings, paragraphs, or Markdown sections rather than arbitrary character counts. I add **overlap** (e.g. 10-20%) so a sentence spanning a boundary isn't lost, and I attach **metadata** like source, title, and section for filtering and citations. Typical sizes are 200-500 tokens for precise QA. For some use cases I store small chunks but retrieve their **parent** larger context, known as parent-document retrieval.

CODE · TypeScript

```typescript
function chunkText(text: string, size = 400, overlap = 60): string[] {
  const words = text.split(/\s+/);
  const chunks: string[] = [];
  let i = 0;
  while (i < words.length) {
    chunks.push(words.slice(i, i + size).join(' '));
    i += size - overlap; // step back to create overlap
  }
  return chunks;
}

// Prefer splitting on headings/paragraphs first, then size-limit:
// const sections = md.split(/\n#{1,3}\s/);  // structure-aware
```

ARCHITECTURE / FLOW

```
  fixed-size with overlap:

  [---- chunk 1 ----]
             [---- chunk 2 ----]
                        [---- chunk 3 ----]
   ^overlap^   ^overlap^   <- boundary sentences preserved

  structure-aware: split on # headings / paragraphs, then cap size
```

Q112 AI, LLM & RAG

Explain HNSW indexing for fast approximate nearest neighbor.

THEORY

HNSW (Hierarchical Navigable Small World) is a graph-based ANN index. It trades a tiny bit of recall for massive speed by navigating a multi-layer graph instead of scanning every vector.

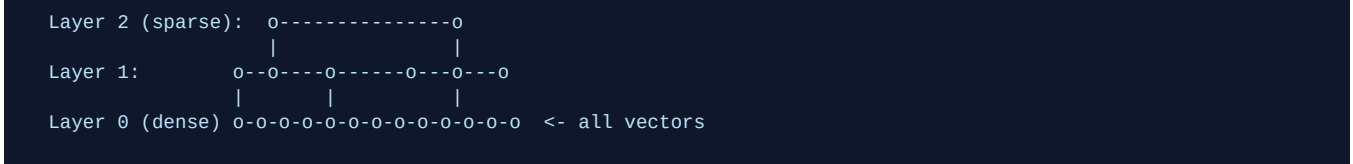
STRONG ANSWER

Brute-force nearest-neighbor over millions of vectors is too slow, so we use **approximate nearest neighbor** (ANN). **HNSW** builds a multi-layer proximity **graph** where upper layers are sparse for long hops and lower layers are dense for fine search. A query enters at the top, greedily walks toward closer neighbors, and descends layer by layer, giving roughly **logarithmic** search time. Key tuning knobs are **m** (neighbors per node) and **ef_construction** at build time, plus **ef_search** at query time, which trades **recall** for latency. In pgvector you create an HNSW index and raise `ef_search` when you need higher accuracy.

CODE · TypeScript

```
-- pgvector HNSW index for cosine distance
CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Per-query recall/latency tradeoff:
SET hnsw.ef_search = 100;  -- higher = better recall, slower

SELECT id, content
FROM docs
ORDER BY embedding <=> $1   -- uses the HNSW index
LIMIT 5;
```

ARCHITECTURE / FLOW



Q113 AI, LLM & RAG

Prompt engineering basics: system prompt and few-shot?

THEORY

The system prompt sets the model's role, rules, and constraints. Few-shot prompting provides example input/output pairs to steer format and behavior.

STRONG ANSWER

The **system prompt** establishes the model's persona, guardrails, and output rules, and it has strong influence over every response, so I keep it explicit and constrained. **Few-shot** prompting includes a handful of example input/output pairs in the prompt to demonstrate the exact **format** and reasoning style I want, which is far more reliable than just describing it. I also use techniques like asking for **structured output** (JSON schema), telling the model to say "I don't know" when unsure, and **chain-of-thought** for reasoning tasks. Clear delimiters separating instructions from data prevent **prompt injection** confusion. Good prompts are specific, give examples, and state what to do when information is missing.

CODE · TypeScript

```
const messages = [
  { role: 'system',
    content: 'You classify support tickets. Reply with one word: ' +
             'billing, technical, or other.' },
  // few-shot examples
  { role: 'user', content: 'My card was charged twice.' },
```

CODE · TypeScript (continued)

```
  { role: 'assistant', content: 'billing' },
  { role: 'user', content: 'The app crashes on launch.' },
  { role: 'assistant', content: 'technical' },
  // real query
  { role: 'user', content: 'I cannot log in after the update.' },
];
// model now outputs in the demonstrated single-word format
```

ARCHITECTURE / FLOW

```
  prompt structure:

  [ system ]  role + rules + constraints  (highest authority)
  [ few-shot ] input1 -> output1
               input2 -> output2          (teaches format)
  [ user ]    actual query
  ---------------------------------------
  delimiters separate INSTRUCTIONS from untrusted DATA
```

Q114 AI, LLM & RAG

Context window limits and how do you manage them?

THEORY

The context window is the maximum number of tokens (input plus output) a model can process at once. Exceeding it causes errors or truncation, so you must budget tokens.

STRONG ANSWER

The **context window** is the hard cap on tokens the model can see, covering the system prompt, conversation history, retrieved context, and the generated output together. When content grows past it, I manage tokens with **retrieval** (only inject the top-k relevant chunks instead of everything), **summarization** of older conversation turns, and **truncation** or sliding windows on history. I budget explicitly: reserve room for the answer, then fill the rest with the highest-value context. For long chats I keep a running **summary** plus the last few verbatim turns. Bigger windows aren't a free pass either, since cost and latency rise with tokens and models can lose info in the middle, so being selective still wins.

CODE · TypeScript

```
function buildPrompt(history: Msg[], chunks: string[], budget = 120_000) {
  const reserveForAnswer = 4_000;
  let used = countTokens(SYSTEM_PROMPT) + reserveForAnswer;
  const ctx: string[] = [];
  for (const c of chunks) {                // add top chunks until full
    const t = countTokens(c);
    if (used + t > budget) break;
    ctx.push(c); used += t;
  }
  const recent = history.slice(-6);        // keep last turns verbatim
  const summary = summarizeOlder(history.slice(0, -6)); // compress rest
  return { ctx, recent, summary };
}
```

ARCHITECTURE / FLOW

```
  context window (e.g. 128k tokens) =

  [ system | summary | recent turns | retrieved chunks | <- answer -> ]
   <----------------- input budget ----------------->  reserved
```

ARCHITECTURE / FLOW (continued)

```
  manage: retrieve (not stuff), summarize old turns, truncate history
```

Q115 AI, LLM & RAG

How do you stream responses (SSE) to the UI?

THEORY

Streaming sends tokens to the client as they are generated rather than waiting for the full response. Server-Sent Events (SSE) is the common transport for this one-way stream.

STRONG ANSWER

Instead of blocking until the whole answer is ready, **streaming** pushes tokens to the browser incrementally, which dramatically improves perceived latency and lets users read as the model writes. The common transport is **Server-Sent Events**, a one-way HTTP stream of `text/event-stream` chunks, though WebSockets also work. On the server I set the LLM call to stream and pipe its token deltas into the response; on the client I consume the stream and append to the UI. The **Vercel AI SDK** abstracts this with `streamText` and the `useChat` hook, handling backpressure, parsing, and reconnection. I make sure to flush chunks and handle aborts when the user cancels.

CODE · TypeScript

```typescript
// Next.js route handler streaming with the Vercel AI SDK
import { openai } from '@ai-sdk/openai';
import { streamText } from 'ai';

export async function POST(req: Request) {
  const { messages } = await req.json();
  const result = streamText({
    model: openai('gpt-4o'),
    messages,
  });
  return result.toDataStreamResponse(); // SSE to the client
}

// Client:
// const { messages, input, handleSubmit } = useChat();
// tokens render as they arrive, no full-response wait
```

ARCHITECTURE / FLOW

```
  client                          server                    LLM
    |  POST /api/chat  ----------->  |                        |
    |                                | --- stream request --->  |
    |  <== data: "Hel" ==            | <-- token deltas ------- |
    |  <== data: "lo" ==             |                          |
    |  <== data: " wor" ==           |   (SSE text/event-stream)
    |  <== data: [DONE] ==           |
```

Q116 AI, LLM & RAG

Explain tool / function calling.

THEORY

Tool calling lets an LLM request a structured call to an external function instead of answering directly. The model emits a JSON arguments object, your code runs the tool, and you feed the result back.

STRONG ANSWER

Tool calling (a.k.a. function calling) gives the LLM a set of typed functions it can invoke; based on the user's request it returns a structured **JSON** call with the function name and arguments rather than free text. Your application **executes** that function (DB query, API, calculator), then returns the result to the model, which uses it to compose the final answer. This is the foundation of **agents** and lets the model take real actions and fetch live data it can't know on its own. I define each tool with a JSON schema and validate the arguments before running anything, since the model can hallucinate parameters. The loop can repeat for multi-step tasks until the model stops requesting tools.

CODE · TypeScript

```
import { generateText, tool } from 'ai';
import { z } from 'zod';
import { openai } from '@ai-sdk/openai';

const result = await generateText({
  model: openai('gpt-4o'),
  tools: {
    getWeather: tool({
      description: 'Get current weather for a city',
      parameters: z.object({ city: z.string() }),
      execute: async ({ city }) => fetchWeather(city), // your code runs
    }),
  },
  maxSteps: 5, // allow multi-step tool loop
  prompt: 'What should I wear in Tokyo today?',
});
```

ARCHITECTURE / FLOW

```
  user -> LLM
            |  decides it needs data
            v
     tool_call { name: getWeather, args: { city: "Tokyo" } }
            |
   your code executes ------> returns result
            |
            v
          LLM <- result -> final natural-language answer
```

Q117 AI, LLM & RAG

Why do hallucinations happen and how does RAG mitigate them?

THEORY

Hallucinations are confident but false outputs. They occur because the model generates plausible text from learned patterns, not from a verified knowledge store.

STRONG ANSWER

Hallucinations happen because an LLM is a next-token predictor optimized for plausibility, so when it lacks knowledge it still produces fluent, confident-sounding text rather than admitting uncertainty. Causes include gaps in training data, outdated knowledge, ambiguous prompts, and pressure to always answer. **RAG** mitigates this by injecting **retrieved, authoritative context** into the prompt and instructing the model to answer only from that context and cite sources, which grounds the output in real data. I reinforce it by telling the model to say "I don't know" when the context lacks the answer, and by showing **citations** so users can verify. RAG reduces but doesn't eliminate hallucination, so evaluation and guardrails still matter.

CODE · TypeScript

```ts
const system = `You are a support assistant.
Rules:
- Answer ONLY using the provided <context>.
- If the answer is not in the context, reply: "I don't know."
- Cite the source number like [1] after each claim.
- Never invent facts, URLs, or numbers.`;

const prompt = `<context>\n${retrievedChunks}\n</context>\n\n` +
               `Question: ${question}`;
// grounding + "I don't know" + citations sharply cut hallucinations
```

ARCHITECTURE / FLOW

```
  WITHOUT RAG:
   question -> LLM (guesses from memory) -> confident wrong answer

  WITH RAG:
   question -> retrieve facts -> LLM grounded in context
                                   |
        "answer only from context, cite [n], else say I don't know"
                                   v
                  verifiable, source-cited answer
```

Q118 AI, LLM & RAG

Explain temperature, top_p and other generation parameters.

THEORY

These parameters control how the model samples the next token from its probability distribution, trading determinism for creativity. They shape the randomness and length of output.

STRONG ANSWER

Temperature scales the probability distribution before sampling: near 0 is nearly deterministic and picks the most likely token (good for extraction, classification, RAG), while higher values like 0.8 add randomness and creativity. **top_p** (nucleus sampling) instead keeps only the smallest set of tokens whose cumulative probability reaches p, sampling from that pool; you typically tune temperature or top_p, not both aggressively. **top_k** limits choices to the k most likely tokens. **max_tokens** caps output length, **stop** sequences end generation, and **frequency/presence penalties** discourage repetition. For RAG and structured tasks I set a **low temperature** for consistency and reproducibility.

CODE · TypeScript

```ts
const res = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages,
  temperature: 0.2,     // low = focused/deterministic (RAG, extraction)
  top_p: 1,             // nucleus sampling; usually tune one of the two
  max_tokens: 500,      // cap response length
  frequency_penalty: 0, // >0 discourages repeating tokens
  presence_penalty: 0,  // >0 encourages new topics
  stop: ['\n\nUser:'],  // hard stop sequence
});
// creative writing -> temperature ~0.8; factual QA -> ~0.0-0.3
```

ARCHITECTURE / FLOW

```
  next-token probabilities:  the(.5) a(.3) one(.1) ...

  temp -> 0:  always pick "the"          (deterministic)
  temp high:  flatter dist, more variety (creative)

  top_p 0.8:  keep tokens until cumulative prob >= 0.8, sample those
  top_k 5  :  keep only the 5 most likely tokens
```

Q119 AI, LLM & RAG

How do you evaluate a RAG system?

THEORY

RAG evaluation splits into retrieval quality and answer quality. You measure whether the right context was fetched and whether the final answer is faithful and relevant.

STRONG ANSWER

I evaluate **RAG** in two layers. **Retrieval** quality uses information-retrieval metrics on a labeled set: **recall@k** and **precision@k**, **MRR**, and context relevance, asking whether the correct chunks appeared in the top-k. **Answer** quality measures **faithfulness** (is the answer grounded in the retrieved context, no hallucination), **answer relevance** to the question, and correctness against ground truth. I use a curated eval dataset of question/answer/source triples and often an **LLM-as-judge** to score faithfulness and relevance at scale, with frameworks like RAGAS. I track these in CI so changes to chunking, the embedding model, or top_k can be compared objectively. The headline insight is that bad retrieval caps everything, so I debug retrieval first.

CODE · TypeScript

```typescript
// LLM-as-judge faithfulness check
const judge = await generateText({
  model: openai('gpt-4o'),
  prompt: `Question: ${q}\nContext: ${context}\nAnswer: ${answer}\n\n` +
    `Is every claim in the Answer supported by the Context?\n` +
    `Reply JSON: {"faithful": boolean, "unsupported": string[]}`,
});

// Retrieval metric: recall@k over a labeled set
function recallAtK(retrieved: string[], relevant: Set<string>, k: number) {
  const hit = retrieved.slice(0, k).filter(id => relevant.has(id)).length;
  return hit / relevant.size;
}
```

ARCHITECTURE / FLOW

```
  RAG eval = two layers

  RETRIEVAL: recall@k, precision@k, MRR, context relevance
       |  (did the right chunks get fetched?)
       v
  ANSWER:   faithfulness (grounded?), relevance, correctness
            measured via labeled set + LLM-as-judge (RAGAS)

  rule: fix retrieval first -- it caps answer quality
```

Q120 AI, LLM & RAG

How do you optimize cost and latency (caching, batching embeddings)?

THEORY

LLM and embedding calls dominate cost and latency. Caching, batching, smaller models, and prompt caching are the main levers.

STRONG ANSWER

I attack cost and latency on several fronts. For **embeddings**, I **batch** many texts into one API call and **cache** vectors keyed by content hash so unchanged documents are never re-embedded. For generation, I use **semantic caching** to reuse answers for similar queries, and I **route** simple requests to a smaller, cheaper model while reserving the large model for hard cases. **Prompt caching** (provider-side) reuses a fixed system prompt or large context across calls at a steep discount. I shrink prompts by retrieving only the top-k relevant chunks instead of stuffing context, and I **stream** to cut perceived latency. Finally I set sensible **max_tokens** and monitor token usage to catch regressions.

CODE · TypeScript

```typescript
import crypto from 'crypto';

// Batch + cache embeddings by content hash
async function embedMany(texts: string[]): Promise<number[][]> {
  const miss: string[] = [], out: (number[] | null)[] = texts.map(t => {
    const v = cache.get(hash(t)); if (!v) miss.push(t); return v ?? null;
  });
  if (miss.length) {
    const res = await openai.embeddings.create({
      model: 'text-embedding-3-small', input: miss }); // one batched call
    res.data.forEach((d, i) => cache.set(hash(miss[i]), d.embedding));
  }
  return texts.map(t => cache.get(hash(t))!);
}
function hash(s: string){ return crypto.createHash('sha256').update(s).digest('hex'); }
```

ARCHITECTURE / FLOW

```
levers:
 embeddings: BATCH many-per-call + CACHE by content hash
 generation: semantic cache + route easy->small model
 prompt caching: reuse fixed system/context at a discount
 prompts: retrieve top-k only (fewer input tokens)
 latency: STREAM tokens + set max_tokens

 measure tokens -> catch cost/latency regressions
```

9 Real-Time Systems & WebSockets

WebSockets, reconnection, Redis pub/sub, Yjs CRDT, presence and scaling.

Q121–Q135

Q121 REAL-TIME SYSTEMS & WEBSOCKETS

WebSockets vs HTTP polling vs long-polling

THEORY

HTTP polling repeatedly asks the server for updates; long-polling holds a request open until data arrives; WebSockets keep a single full-duplex TCP connection open.

STRONG ANSWER

HTTP polling sends a request on a fixed interval, wasting bandwidth on empty responses and adding latency equal to the poll interval. **Long-polling** improves this by holding the request open until the server has data, then immediately reopening, which reduces empty responses but still pays per-request HTTP overhead. **WebSockets** upgrade a single connection to a persistent, **full-duplex** channel so both sides push messages with minimal framing overhead and near-zero latency. For my real-time collaboration work I use WebSockets because presence and Yjs updates need **low-latency bidirectional** push, not request-driven pulls.

CODE · TypeScript

```typescript
// Polling
setInterval(async () => {
  const r = await fetch('/api/updates?since=' + cursor);
  apply(await r.json());
}, 3000); // latency up to 3s, many empty 200s

// Long-polling: server holds request until data
async function poll() {
  const r = await fetch('/api/updates/long'); // blocks server-side
  apply(await r.json());
  poll(); // reopen immediately
}

// WebSocket: persistent full-duplex
const ws = new WebSocket('wss://api.example.com/rt');
ws.onmessage = (e) => apply(JSON.parse(e.data));
ws.send(JSON.stringify({ type: 'edit', op })); // push anytime
```

ARCHITECTURE / FLOW

```
Polling:      C--req-->S  C--req-->S  C--req-->S   (mostly empty)
Long-poll:    C--req--------(held)------->S--data-->C  reopen
WebSocket:    C===handshake===> [ open full-duplex ]
              C <----msg----> S   <----msg----> C   (both push)
```

Q122 REAL-TIME SYSTEMS & WEBSOCKETS

The WebSocket handshake and connection lifecycle

THEORY

A WebSocket starts as an HTTP GET with an Upgrade header; the server responds 101 Switching Protocols, after which the socket carries WebSocket frames until close.

STRONG ANSWER

The client sends an HTTP **GET with `Upgrade: websocket`** and a random **`Sec-WebSocket-Key`**; the server replies **`101 Switching Protocols`** with a hashed **`Sec-WebSocket-Accept`** proving it understood the protocol. After the upgrade the connection leaves HTTP semantics and exchanges **frames** (text, binary, ping, pong, close) over the same TCP socket. The lifecycle is **CONNECTING -> OPEN -> CLOSING -> CLOSED**, exposed in the browser via `onopen`, `onmessage`, `onerror`, and `onclose`. A clean shutdown uses a **close frame** with a status code so both peers know the reason rather than seeing an abrupt TCP reset.

CODE · TypeScript

```
// Client request headers (sent by browser automatically)
// GET /rt HTTP/1.1
// Upgrade: websocket
// Connection: Upgrade
// Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
// Sec-WebSocket-Version: 13

// Server 101 response
// HTTP/1.1 101 Switching Protocols
// Upgrade: websocket
// Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

const ws = new WebSocket('wss://api.example.com/rt');
ws.onopen   = () => ws.send('hello');      // state OPEN
ws.onmessage = (e) => console.log(e.data);
ws.onclose  = (e) => console.log('closed', e.code, e.reason);
ws.close(1000, 'done'); // graceful close frame
```

ARCHITECTURE / FLOW

```
Client                          Server
  | -- GET Upgrade: websocket --> |
  | <-- 101 Switching Protocols-- |
  |        [ OPEN ]               |
  | <----- frames both ways ----> |
  | -- close frame (1000) ------> |
  | <-- close frame ------------- |
  |        [ CLOSED ]             |
States: CONNECTING -> OPEN -> CLOSING -> CLOSED
```

Q123 REAL-TIME SYSTEMS & WEBSOCKETS

Reconnection with exponential backoff

THEORY

After a drop the client should retry, but increasing the delay exponentially with random jitter prevents thundering-herd reconnect storms.

STRONG ANSWER

On `onclose` you reconnect, but retrying instantly hammers a recovering server, so you use **exponential backoff**: delay doubles each attempt up to a cap. You add **jitter** (randomization) so thousands of clients that dropped together don't reconnect in synchronized waves and cause a **thundering herd**. I reset the backoff to its base once a connection stays open past a stability threshold, so transient blips don't permanently inflate the delay. After reconnecting you also **resubscribe to channels and resync state** (e.g. request missed Yjs updates) because the server has no memory of the old socket.

CODE · TypeScript

```typescript
function connect() {
  const ws = new WebSocket(URL);
  let attempt = 0;
  ws.onopen = () => { attempt = 0; resubscribe(ws); };
  ws.onclose = () => {
    attempt++;
    const base = Math.min(30000, 1000 * 2 ** attempt); // cap 30s
    const jitter = Math.random() * base;               // full jitter
    const delay = base / 2 + jitter / 2;
    setTimeout(connect, delay);
  };
  return ws;
}
connect();
```

ARCHITECTURE / FLOW

```
attempt:  1     2     3     4     5
base:    1s    2s    4s    8s    16s ... cap 30s
+jitter: ~1s  ~1.5s ~3s   ~6s   ~12s   (spread, no herd)

[drop] -> wait(backoff+jitter) -> connect -> open? reset attempt=0
```

Q124 REAL-TIME SYSTEMS & WEBSOCKETS

Heartbeat / ping-pong to detect dead connections

THEORY

TCP can silently die (NAT timeout, sleep, network loss) without a close frame, so periodic ping/pong probes detect half-open connections.

STRONG ANSWER

A WebSocket can become a **half-open** connection where one side thinks it's connected but no data flows, because TCP doesn't always signal the break. The protocol has built-in **ping/pong control frames**: the server sends a `ping` on an interval and expects a `pong` before a timeout. If the pong is missing the server **terminates the socket** and frees its resources; the client meanwhile uses its own timer to detect a dead server and trigger reconnection. This is essential at scale so you don't leak thousands of zombie connections and so presence accurately reflects who is really online.

CODE · TypeScript

```typescript
// Server (ws library)
function heartbeat() { this.isAlive = true; }
wss.on('connection', (ws) => {
  ws.isAlive = true;
  ws.on('pong', heartbeat);
});
const interval = setInterval(() => {
  wss.clients.forEach((ws) => {
    if (!ws.isAlive) return ws.terminate(); // no pong -> kill
    ws.isAlive = false;
    ws.ping();                              // expect pong before next tick
  });
}, 30000);
wss.on('close', () => clearInterval(interval));
```

ARCHITECTURE / FLOW

```
Server                        Client
  | -- ping ------------------> |
  | <------------------ pong -- |   alive, isAlive=true
  | -- ping ------------------> |
  |        (no pong)            |   client dead / NATed
  | -- ping ------------------> |
  |   timeout -> terminate()    |   socket reclaimed
```

Q125 REAL-TIME SYSTEMS & WEBSOCKETS

Scaling WebSockets horizontally (stateless servers + shared bus)

THEORY

Connections are sticky to one server, so to scale you keep servers stateless and route cross-server messages through a shared message bus.

STRONG ANSWER

A WebSocket is **pinned to one server instance** for its lifetime, so client A on server 1 and client B on server 2 can't talk directly. The fix is to keep the WS servers **stateless** (no in-memory authority over shared data) and put a **shared bus** like Redis pub/sub or NATS between them. When server 1 receives a message it **publishes to the bus**, every server **subscribes**, and each delivers to its own locally connected sockets. Session and document state live in a shared store (Redis/DB), so any server can handle any client and you scale by simply **adding instances behind a load balancer**.

CODE · TypeScript

```typescript
// Each stateless WS node both publishes and subscribes
import Redis from 'ioredis';
const pub = new Redis(), sub = new Redis();
sub.subscribe('room:42');

sub.on('message', (_ch, payload) => {
  // fan out to THIS node's local sockets only
  for (const ws of localSockets.get('room:42') ?? [])
    ws.send(payload);
});

function onClientMessage(roomId: string, payload: string) {
  pub.publish(`room:${roomId}`, payload); // reaches all nodes
}
```

ARCHITECTURE / FLOW

```
        Load Balancer (sticky)
        /        |          \
   WS node1   WS node2   WS node3   (stateless)
      |  \       |  /        |
      |   \      | /         |
      +---- Shared Bus (Redis pub/sub) -----+
   publish on receive, every node delivers to its local sockets
```

Q126 REAL-TIME SYSTEMS & WEBSOCKETS

Redis pub/sub for fan-out across server instances

THEORY

Redis pub/sub broadcasts a published message to every subscriber of a channel, giving cheap cross-instance fan-out for real-time systems.

STRONG ANSWER

Redis pub/sub lets a publisher send to a named **channel** and Redis pushes that message to all current **subscribers**, decoupling senders from receivers. In a multi-node WS cluster each node subscribes to room channels, so one node's publish reaches every node that holds sockets for that room. It is **fire-and-forget with no persistence**: messages sent while a node is disconnected are lost, so durability needs the underlying CRDT/DB state, not the bus. For pattern routing you can `PSUBSCRIBE room:*`, and for very high throughput teams move to **Redis Streams** or a broker when at-least-once delivery and replay matter.

CODE · TypeScript

```typescript
// Publisher and subscriber MUST be separate connections
const pub = new Redis();
const sub = new Redis();

// Subscribe with a pattern to catch all rooms on this node
await sub.psubscribe('room:*');
sub.on('pmessage', (_pattern, channel, message) => {
  const roomId = channel.split(':')[1];
  broadcastLocal(roomId, message); // send to local sockets
});

// Publish an edit; every subscribed node receives it
await pub.publish('room:42', JSON.stringify({ type: 'yjs-update', update }));
```

ARCHITECTURE / FLOW

Q127 REAL-TIME SYSTEMS & WEBSOCKETS

Presence and typing indicators

THEORY

Presence tracks who is currently connected to a room; typing indicators are short-lived ephemeral signals that should expire automatically.

STRONG ANSWER

Presence answers who is online in a room and is derived from live connections, so it must update on connect, disconnect, and missed heartbeats. I store it as **ephemeral state with a TTL** (e.g. a Redis key per user that auto-expires) so a crashed client naturally drops off without a clean disconnect. **Typing indicators** are transient broadcasts that should **time out client-side** after a couple seconds of inactivity, never persisted, since stale 'typing...' is worse than none. In Yjs this maps cleanly onto the **awareness protocol**, which gossips cursor position, selection, and presence as ephemeral fields separate from the persisted document.

CODE · TypeScript

```typescript
// Presence with TTL so dead clients self-expire
async function setPresence(roomId: string, userId: string) {
  await redis.set(`presence:${roomId}:${userId}`, '1', 'EX', 30);
  await pub.publish(`room:${roomId}`, JSON.stringify({ t: 'join', userId }));
}
setInterval(() => refreshPresence(), 15000); // renew before TTL

// Typing: ephemeral, auto-clears client-side
let typingTimer: any;
function onKeystroke(roomId: string) {
  send({ t: 'typing', roomId, on: true });
  clearTimeout(typingTimer);
  typingTimer = setTimeout(() => send({ t: 'typing', on: false }), 2000);
}
```

ARCHITECTURE / FLOW

```
Presence (TTL=30s, renewed every 15s):
  user online --> SET key EX30 --renew--> still online
  user crashes --> no renew --> key expires --> dropped

Typing indicator:
  keypress -> broadcast typing=true -> peers show '...'
         (2s silent) -> auto typing=false -> indicator clears
```

Q128 REAL-TIME SYSTEMS & WEBSOCKETS

What a CRDT is and why it enables conflict-free merging

THEORY

A CRDT (Conflict-free Replicated Data Type) is a data structure whose concurrent updates merge deterministically without coordination or a central authority.

STRONG ANSWER

A **CRDT** is a replicated data type where every replica can be updated independently and all replicas **converge to the same state** once they've seen the same updates, with no locks or central server. This works because the merge operation is **commutative, associative, and idempotent**, so update order and duplicates don't change the result, satisfying **Strong Eventual Consistency**. There are two flavors: **state-based (CvRDT)** that merge whole states via a least-upper-bound, and **operation-based (CmRDT)** that broadcast operations. For collaborative text, sequence CRDTs assign each character a unique, totally-ordered identity so concurrent inserts interleave deterministically instead of clobbering each other.

CODE · TypeScript

```ts
// Tiny grow-only counter CRDT (state-based)
type GCounter = Record<string, number>; // nodeId -> count

function inc(c: GCounter, node: string): GCounter {
  return { ...c, [node]: (c[node] ?? 0) + 1 };
}

// merge = element-wise max -> commutative, idempotent
function merge(a: GCounter, b: GCounter): GCounter {
  const out: GCounter = { ...a };
  for (const k in b) out[k] = Math.max(out[k] ?? 0, b[k]);
  return out;
}
function value(c: GCounter) { return Object.values(c).reduce((s, n) => s + n, 0); }
// merge(merge(a,b),c) === merge(a,merge(b,c)); order-independent
```

ARCHITECTURE / FLOW

```
Replica A          Replica B
 {A:2}              {B:3}
   \                  /
    \   merge(max)   /
     v              v
   {A:2, B:3}   ==  {A:2, B:3}   (converge, any order)

Merge props: commutative + associative + idempotent
  => duplicates/out-of-order updates are safe
```

Q129 REAL-TIME SYSTEMS & WEBSOCKETS

Yjs basics (shared document, updates)

THEORY

Yjs is a high-performance CRDT framework; a Y.Doc holds shared types and emits compact binary update messages you broadcast to peers.

STRONG ANSWER

Yjs implements an optimized sequence CRDT exposed as a **`Y.Doc`** containing **shared types** like `Y.Text`, `Y.Array`, and `Y.Map`. Every local mutation produces a small **binary update** emitted via the doc's `update` event; you send that update over any transport (WebSocket, WebRTC) and apply it on peers with **`Y.applyUpdate`**. Because it's a CRDT, updates are **commutative and idempotent**, so they can arrive out of order or be re-applied safely and all docs converge. State vectors let a reconnecting client request only the **diff it's missing** rather than the whole document, and a **provider** like y-websocket wraps the transport, sync handshake, and awareness for you.

CODE · TypeScript

```ts
import * as Y from 'yjs';
const doc = new Y.Doc();
const ytext = doc.getText('content');

// Emit binary updates to the network
doc.on('update', (update: Uint8Array, origin) => {
  if (origin !== 'remote') ws.send(update);
});

// Apply remote updates (tag origin to avoid echo loops)
ws.onmessage = (e) =>
  Y.applyUpdate(doc, new Uint8Array(e.data), 'remote');

// Reconnect: send only the diff the peer lacks
```

CODE · TypeScript (continued)

```
const sv = Y.encodeStateVector(doc);
const diff = Y.encodeStateAsUpdate(doc, remoteStateVector);
ytext.insert(0, 'Hello'); // local edit -> triggers update event
```

ARCHITECTURE / FLOW

```
  Client A Y.Doc            Client B Y.Doc
  ytext.insert() ---update(bin)---> applyUpdate()
        ^                                |
        |<-------- update(bin) -----------+
  Both converge regardless of arrival order

  Reconnect sync:
    A: stateVector --> B computes diff --> sends only missing ops
```

Q130 REAL-TIME SYSTEMS & WEBSOCKETS

OT vs CRDT for collaborative editing

THEORY

Operational Transformation transforms operations against concurrent ones via a central server; CRDTs make operations natively commutative so no central transform is needed.

STRONG ANSWER

Operational Transformation (OT) represents edits as operations (insert/delete at an index) and **transforms** each against concurrent operations so indices stay correct; it's proven (Google Docs) but typically needs a **central server** to order and transform, and the transform functions are notoriously hard to get right. **CRDTs** instead give every element a stable unique identity so operations are **inherently commutative** and merge without a server-side transform, which suits **peer-to-peer and offline-first** editing. The trade-off is that CRDTs can carry **metadata/tombstone overhead** per element, while OT keeps payloads small but couples you to a coordinating server. For my Yjs-based work I chose CRDT because it tolerates reconnects, out-of-order delivery, and multi-node fan-out without a single ordering authority.

CODE · TypeScript

```
// OT: transform a local op against a concurrent remote op
function transform(local: Op, remote: Op): Op {
  if (remote.type === 'insert' && remote.pos <= local.pos)
    return { ...local, pos: local.pos + remote.text.length }; // shift index
  return local;
} // requires central server to sequence + transform all ops

// CRDT (Yjs): no index transform; identities make ops commute
import * as Y from 'yjs';
const doc = new Y.Doc();
Y.applyUpdate(doc, remoteUpdateA); // order-independent
Y.applyUpdate(doc, remoteUpdateB); // converges either way
```

ARCHITECTURE / FLOW

```
OT:   op --> [ central server: transform vs concurrent ops ] --> peers
      small payloads, but needs ordering authority

CRDT: op carries unique id --> broadcast --> each peer merges locally
      no central transform; P2P/offline OK; more per-elem metadata

          | central server | offline | merge logic
OT        |   required     |  hard   | transform fns
```

ARCHITECTURE / FLOW (continued)

```
CRDT      |   optional     |  easy   | commutative merge
```

Q131 REAL-TIME SYSTEMS & WEBSOCKETS

Message ordering and delivery guarantees (at-least-once etc.)

THEORY

Delivery semantics range from at-most-once (may lose) to at-least-once (may duplicate) to exactly-once (hard); ordering may be per-channel or global.

STRONG ANSWER

At-most-once sends without retry, so messages can be lost but never duplicated; **at-least-once** retries until acknowledged, guaranteeing delivery but risking **duplicates**; **exactly-once** is the hardest and is usually faked with at-least-once plus **idempotent handlers or dedup**. Ordering is separate: a single TCP/WebSocket connection preserves order, but across a fan-out bus or after reconnects messages can arrive **out of order or twice**. The practical pattern is at-least-once delivery with **per-message IDs and idempotent processing**, which is exactly why CRDTs shine, their merges are idempotent and order-independent so duplicates and reordering are harmless. For strict ordering I attach **monotonic sequence numbers** and have the client request gaps it detects.

CODE · TypeScript

```typescript
// At-least-once with idempotent dedup on the client
const seen = new Set<string>();
function onMessage(msg: { id: string; seq: number; data: unknown }) {
  if (seen.has(msg.id)) return;        // drop duplicate
  seen.add(msg.id);
  if (msg.seq > expectedSeq) requestGap(expectedSeq, msg.seq); // detect reorder
  expectedSeq = msg.seq + 1;
  apply(msg.data);
}

// Producer retries until ack -> guarantees at-least-once
async function sendReliable(m: Msg) {
  while (!(await sendAndAwaitAck(m))) await delay(backoff());
}
```

ARCHITECTURE / FLOW

```
at-most-once : send, no retry        -> may LOSE, no dup
at-least-once: retry until ack        -> no loss, may DUP
exactly-once : at-least-once + dedup  -> no loss, no dup (hard)

Reorder/dup handling:
  msg{id, seq} -> seen? drop : apply
  seq gap? -> request missing range
  CRDT merge: idempotent + commutative -> dup/reorder safe
```

Q132 REAL-TIME SYSTEMS & WEBSOCKETS

Backpressure when a client is slow

THEORY

If a server sends faster than a client can consume, the socket's send buffer grows; backpressure means pausing or shedding load to avoid unbounded memory growth.

STRONG ANSWER

When a consumer is slower than the producer, unsent data accumulates in the socket's **send buffer** (`ws.bufferedAmount`), and ignoring it leads to **unbounded memory growth** and eventually an OOM crash. The fix is **backpressure**: check `bufferedAmount` before sending and, when it exceeds a high-water mark, **pause, coalesce, drop low-value messages, or disconnect** the lagging client. For state-syncing systems you can **collapse intermediate updates** into the latest snapshot since a slow client doesn't need every keystroke, only convergence. Node streams handle this automatically via the `write()` return value and `drain` event, and at the protocol level **per-client queues with limits** keep one slow consumer from harming the whole node.

CODE · TypeScript

```ts
const HIGH_WATER = 1 << 20; // 1 MB

function safeSend(ws: WebSocket, payload: string) {
  if (ws.bufferedAmount > HIGH_WATER) {
    // slow client: coalesce to latest snapshot instead of queuing all
    pendingSnapshot.set(ws, latestState());
    return; // skip this incremental update
  }
  ws.send(payload);
}

// Flush coalesced snapshot once the buffer drains
setInterval(() => {
  for (const [ws, snap] of pendingSnapshot)
    if (ws.bufferedAmount < HIGH_WATER) { ws.send(snap); pendingSnapshot.delete(ws); }
}, 200);
```

ARCHITECTURE / FLOW

```
Fast producer --> [ send buffer ] --> slow consumer
                        |
            bufferedAmount grows
                        |
      < HIGH_WATER : send normally
      > HIGH_WATER : pause / coalesce-to-snapshot / drop / disconnect

Goal: bound memory; converge slow client to LATEST, skip stale deltas
```

Q133 REAL-TIME SYSTEMS & WEBSOCKETS

SSE vs WebSocket — when to use each

THEORY

Server-Sent Events are a one-way server-to-client stream over plain HTTP; WebSockets are full-duplex. Choose by directionality and infrastructure.

STRONG ANSWER

Server-Sent Events (SSE) are **unidirectional** (server to client only) over a long-lived HTTP response, with built-in **auto-reconnect and `Last-Event-ID`** resume, and they ride normal HTTP so proxies and load balancers handle them easily. **WebSockets** are **full-duplex**, support binary frames, and are the right choice when the client also pushes frequently, like collaborative editing or chat. I reach for SSE when updates are mostly **server-driven** (notifications, live feeds, dashboards) because it's simpler and HTTP-friendly; I use WebSockets for **interactive bidirectional** real-time like Yjs sync and presence. A caveat: SSE over HTTP/1.1 is limited by the **6-connection-per-domain** cap, which HTTP/2 multiplexing mitigates.

CODE · TypeScript

```ts
// SSE: server -> client only, plain HTTP, auto-reconnect
const es = new EventSource('/api/stream');
es.onmessage = (e) => render(e.data);
es.addEventListener('price', (e) => updatePrice(e.data));
// browser auto-reconnects and resends Last-Event-ID

// Server (Express) sends an SSE frame
res.writeHead(200, { 'Content-Type': 'text/event-stream',
  'Cache-Control': 'no-cache', Connection: 'keep-alive' });
res.write(`id: 42\nevent: price\ndata: ${JSON.stringify(p)}\n\n`);

// WebSocket: use when client must push too
const ws = new WebSocket('wss://api.example.com/rt');
ws.send(JSON.stringify({ type: 'edit', op })); // bidirectional
```

ARCHITECTURE / FLOW

```
SSE  (HTTP, 1-way):   Server ===events===> Client   (auto-reconnect)
WebSocket (2-way):    Server <===frames===> Client

Pick SSE when:        server-push feeds, notifications, simple infra
Pick WebSocket when:  client also pushes often (chat, Yjs, presence)
Note: SSE/HTTP1.1 capped ~6 conns/domain; HTTP/2 fixes via multiplexing
```

Q134 REAL-TIME SYSTEMS & WEBSOCKETS

WebRTC basics (peer-to-peer, signaling)

THEORY

WebRTC enables direct browser-to-browser media and data channels; peers exchange SDP and ICE candidates via a separate signaling channel, with STUN/TURN for NAT traversal.

STRONG ANSWER

WebRTC establishes a **peer-to-peer** connection for audio, video, and arbitrary **data channels**, keeping media off your servers for low latency. Peers can't find each other directly, so a **signaling channel** (often a WebSocket) exchanges **SDP offer/answer** describing codecs and an **ICE** list of candidate network paths. **STUN** servers discover a peer's public address behind NAT, and when direct connection fails a **TURN** server relays traffic as a fallback. For collaboration you can run Yjs over a WebRTC **data channel** (y-webrtc) to sync CRDT updates peer-to-peer, though you still need signaling and usually TURN for reliability across restrictive networks.

CODE · TypeScript

```ts
const pc = new RTCPeerConnection({
  iceServers: [{ urls: 'stun:stun.l.google.com:19302' },
               { urls: 'turn:turn.example.com', username: 'u', credential: 'p' }],
});

// Data channel for Yjs/CRDT sync
const dc = pc.createDataChannel('yjs');
dc.onmessage = (e) => Y.applyUpdate(doc, new Uint8Array(e.data));

// Signaling over WebSocket
pc.onicecandidate = (e) => e.candidate && signal.send({ ice: e.candidate });
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
signal.send({ sdp: offer }); // remote does setRemoteDescription + answer
```

ARCHITECTURE / FLOW

```
Peer A               Signaling (WS)               Peer B
  | --- SDP offer ------->  server  ----- offer ----> |
  | <-- SDP answer -------  server  <---- answer ----- |
  | --- ICE candidates -->  server  <--- ICE -------- |
          (STUN finds public IP; TURN relays if blocked)
  A <============ direct P2P data/media channel ==========> B
```

Q135 REAL-TIME SYSTEMS & WEBSOCKETS

Channel-based fan-out architecture (stateless WS server + Redis pub/sub)

THEORY

Organize real-time delivery around named channels (rooms); stateless WS nodes subscribe to channels on a shared bus so any node can serve any client.

STRONG ANSWER

I model the system around **channels** (one per room/document); a client connecting to any **stateless WS node** causes that node to **subscribe** to the room's Redis channel and join the socket to a local set. When an edit arrives, the node **publishes to the channel** and Redis fans it out to every node subscribed, each delivering only to its **locally connected sockets** for that room. Because nodes hold no authoritative state, the **load balancer can route any client to any node**, and durable state lives in the **CRDT/DB** so reconnects resync via state vectors. To avoid leaks I **unsubscribe** a channel when its last local socket leaves, use **presence keys with TTL**, and rely on the CRDT's idempotent merges to make at-least-once fan-out safe. This is the exact pattern from my resume: **stateless WS + Redis pub/sub** scaling presence and Yjs collaboration horizontally.

CODE · TypeScript

```typescript
const rooms = new Map<string, Set<WebSocket>>();

function join(ws: WebSocket, room: string) {
  if (!rooms.has(room)) { rooms.set(room, new Set()); sub.subscribe(`room:${room}`); }
  rooms.get(room)!.add(ws);
}
function leave(ws: WebSocket, room: string) {
  const set = rooms.get(room); set?.delete(ws);
  if (set && set.size === 0) { rooms.delete(room); sub.unsubscribe(`room:${room}`); }
}
sub.on('message', (ch, payload) => {
  for (const ws of rooms.get(ch.slice(5)) ?? []) ws.send(payload);
});
function publish(room: string, payload: string) { pub.publish(`room:${room}`, payload); }
```

ARCHITECTURE / FLOW

```
client --LB--> WS node (stateless)
   join -> SUB room:42 on Redis, add to local set rooms['42']

   edit -> PUBLISH room:42 -----> [ Redis ] -----> all subbed nodes
                                                   each: send to local rooms['42']

   last client leaves -> UNSUBSCRIBE room:42 (no leaks)
   durable state: CRDT/DB; presence: TTL keys; merges: idempotent
```

10 Architecture, Docker & DevOps

Clean architecture, monorepo, Docker, CI/CD, Nginx, PM2, testing, monitoring.

Q136–Q150

Q136 ARCHITECTURE, DOCKER & DEVOPS

Explain the layers of Clean Architecture.

THEORY

Clean Architecture organizes code into concentric layers where dependencies point inward, isolating business rules from frameworks and I/O.

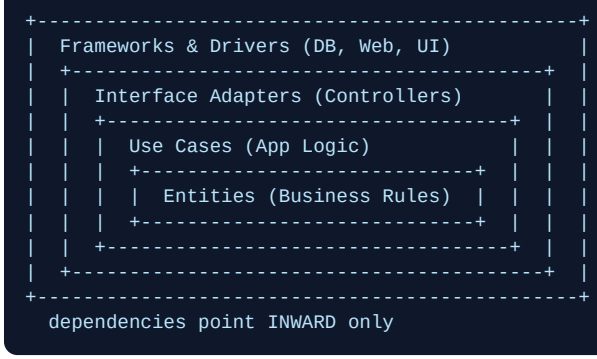
STRONG ANSWER

At the center are **Entities**, the enterprise-wide business objects and rules that change least often. Around them sit **Use Cases**, which orchestrate entities to fulfill application-specific operations. The **Interface Adapters** layer converts data between use cases and the outside world: controllers, presenters, and gateways. The outermost ring is **Frameworks & Drivers**, the database, web framework, and UI. The key rule is the **Dependency Rule**: source code dependencies point only inward, so inner layers never know about outer ones.

CODE · TypeScript

```typescript
// use case depends on an abstraction, not on the DB framework
interface UserRepository {
  findById(id: string): Promise<User | null>;
}

class GetUserProfile {
  constructor(private readonly repo: UserRepository) {}
  async exec(id: string): Promise<User> {
    const user = await this.repo.findById(id);
    if (!user) throw new Error('not found');
    return user;
  }
}
```

ARCHITECTURE / FLOW



Q137 ARCHITECTURE, DOCKER & DEVOPS

What is the Repository pattern and why use it?

THEORY

The Repository pattern abstracts data access behind a collection-like interface, decoupling business logic from the persistence mechanism.

STRONG ANSWER

A **repository** exposes methods like `findById` or `save` that read like an in-memory collection, hiding whether data comes from SQL, a REST API, or a cache. This lets use cases depend on an **interface** rather than a concrete ORM, which keeps the domain testable and swappable. In tests I substitute an **in-memory implementation**, avoiding a real database entirely. It also centralizes query logic so persistence concerns don't leak across the codebase. The tradeoff is a layer of indirection, so I avoid over-abstracting trivial CRUD.

CODE · TypeScript

```typescript
interface OrderRepository {
  save(order: Order): Promise<void>;
  findById(id: string): Promise<Order | null>;
}

class PrismaOrderRepository implements OrderRepository {
  constructor(private db: PrismaClient) {}
  async save(o: Order) { await this.db.order.create({ data: o }); }
  async findById(id: string) { return this.db.order.findUnique({ where: { id } }); }
}

// tests use an in-memory map implementing the same interface
```

ARCHITECTURE / FLOW

```
Use Case ---> OrderRepository (interface)
                    ^
         +----------+----------+
         |                     |
  PrismaOrderRepo         InMemoryOrderRepo
   (production)              (unit tests)
```

Q138 ARCHITECTURE, DOCKER & DEVOPS

Explain dependency injection and inversion of control.

THEORY

Inversion of Control flips who controls object creation; dependency injection is one way to achieve it by passing dependencies in rather than constructing them internally.

STRONG ANSWER

With **Inversion of Control**, a class no longer instantiates its own collaborators; instead something external supplies them. **Dependency Injection** is the concrete technique of passing those collaborators through the **constructor**, a setter, or a parameter. This decouples high-level policy from low-level details, satisfying the **Dependency Inversion Principle**. The payoff is testability: I inject a mock or fake in tests and the real implementation in production. A **DI container** can wire the graph automatically, but constructor injection by hand is often clearest for small projects.

CODE · TypeScript

```typescript
// without DI: hard to test, tightly coupled
class Billing { private gw = new StripeGateway(); }

// with DI: dependency passed in, easy to swap
class Billing {
  constructor(private gw: PaymentGateway) {}
  charge(amt: number) { return this.gw.charge(amt); }
}

const billing = new Billing(new StripeGateway());   // prod
const test = new Billing(new FakeGateway());         // test
```

ARCHITECTURE / FLOW

```
WITHOUT IoC:  Billing --creates--> StripeGateway (locked in)

WITH IoC:     Composition Root
                    |
                    | injects
                    v
              Billing( PaymentGateway )
                          ^
              Stripe / Fake / PayPal
```

Q139 ARCHITECTURE, DOCKER & DEVOPS

Compare monorepo vs polyrepo and their tradeoffs.

THEORY

A monorepo stores many projects in one repository; polyrepo gives each project its own. The choice affects sharing, tooling, and team autonomy.

STRONG ANSWER

A **monorepo** makes code sharing and atomic cross-package changes easy: one commit can update a library and every consumer together, with a single source of truth for tooling. Tools like **Turborepo**, **Nx**, or **pnpm workspaces** add caching and affected-only builds so CI stays fast. The cost is heavier tooling, large clone sizes, and the need for **build graph** awareness. **Polyrepo** gives strong isolation and independent release cadence per team, but version drift and duplicated config become painful. I usually pick a monorepo when packages are tightly coupled and a polyrepo when teams ship truly independent services.

CODE · Shell / Bash

```bash
# pnpm monorepo workspace layout
apps/
  web/        # next.js frontend
  api/        # node backend
packages/
  ui/         # shared component library
  config/     # shared eslint / tsconfig
  types/      # shared domain types
pnpm-workspace.yaml
turbo.json    # task pipeline + remote caching
```

ARCHITECTURE / FLOW

```
MONOREPO                      POLYREPO
+---------------------+       +------+ +------+ +------+
| apps/  packages/    |       | web  | | api  | |  ui  |
|  web    ui  types   |       +------+ +------+ +------+
```

ARCHITECTURE / FLOW (continued)

```
|  api    config      |        separate repos, versioned
+---------------------+        via published packages
 one CI, atomic commits        independent releases
```

Q140 ARCHITECTURE, DOCKER & DEVOPS

Summarize the SOLID principles with a concrete example.

THEORY

SOLID is five object-oriented design principles that promote maintainable, loosely coupled code.

STRONG ANSWER

Single Responsibility: a class has one reason to change. **O**pen/Closed: open for extension, closed for modification. **L**iskov Substitution: subtypes must be usable wherever their base type is. **I**nterface Segregation: prefer small, focused interfaces over fat ones. **D**ependency Inversion: depend on abstractions, not concretions. A concrete **Open/Closed** example is a notification system that accepts a `Notifier` interface, so adding SMS or Slack means writing a new class, never editing the dispatcher.

CODE · TypeScript

```typescript
interface Notifier { send(msg: string): Promise<void>; }

class EmailNotifier implements Notifier { /* ... */ }
class SlackNotifier implements Notifier { /* ... */ }

// Open for extension, closed for modification:
class Dispatcher {
  constructor(private channels: Notifier[]) {}
  async broadcast(msg: string) {
    await Promise.all(this.channels.map(c => c.send(msg)));
  }
}
// add a new channel without touching Dispatcher
```

ARCHITECTURE / FLOW

```
S - one class, one reason to change
O - extend via new classes, don't edit existing
L - subclass swaps in without breaking callers
I - many small interfaces > one fat interface
D - high-level + low-level both depend on abstractions
```

Q141 ARCHITECTURE, DOCKER & DEVOPS

Distinguish a Docker image, container, and layer.

THEORY

Images are immutable templates, containers are running instances, and layers are the cached filesystem deltas that compose an image.

STRONG ANSWER

A Docker **image** is a read-only template built from a Dockerfile that bundles your app and its dependencies. A **container** is a running (or stopped) instance of that image with a thin writable layer on top, isolated by namespaces and cgroups. Each instruction in a Dockerfile produces a **layer**, a cached filesystem diff, and identical layers are shared across images to save space and speed builds. Because the writable container layer is ephemeral, persistent data must live in **volumes**. Ordering instructions so stable layers come first maximizes **build cache** reuse.

CODE · Shell / Bash

```
# build an image (template) from a Dockerfile
docker build -t myapp:1.0 .

# run a container (instance) from the image
docker run -d --name web -p 8080:3000 myapp:1.0

# inspect the layers that compose the image
docker history myapp:1.0

# images are immutable; containers add a writable layer
docker ps        # running containers
docker images    # available images
```

ARCHITECTURE / FLOW

```
IMAGE (read-only, layered)        CONTAINER
+-------------------------+   +-------------------------+
| L4 app code             |   | writable layer (rw)     |
| L3 npm install          |   +-------------------------+
| L2 apt packages         |-->| image layers (read-only) |
| L1 base FROM node       |   +-------------------------+
+-------------------------+    run = image + writable top
```

Q142 ARCHITECTURE, DOCKER & DEVOPS

What are Dockerfile best practices for small, fast images?

THEORY

Multi-stage builds, minimal base images, and good layer ordering produce smaller, more secure, cache-friendly images.

STRONG ANSWER

I use a **multi-stage build** so heavy build tooling stays in an early stage and only the compiled output is copied into a slim runtime image. I pick a minimal base like **alpine** or **distroless** and run as a **non-root** user for security. Copying `package.json` and installing dependencies before copying source maximizes **layer caching**, so code changes don't reinstall everything. A **.dockerignore** keeps `node_modules` and `.git` out of the build context. The result is a tiny, reproducible image that pulls and starts fast.

CODE · Dockerfile

```
# ---- build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- runtime stage ----
```

CODE · Dockerfile (continued)

```
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

ARCHITECTURE / FLOW

```
Stage 1: build           Stage 2: runtime
+------------------+     +------------------+
| node:20-alpine   |     | node:20-alpine   |
| dev deps + src   | --> | prod deps only   |
| npm run build    | dist| COPY --from=build|
| (heavy, ~1GB)    |     | (slim, ~150MB)   |
+------------------+     +------------------+
        discarded          shipped to registry
```

Q143 ARCHITECTURE, DOCKER & DEVOPS

How do you use Docker Compose for multi-service local dev?

THEORY

Docker Compose declares multiple services, networks, and volumes in one YAML file so a whole stack starts with a single command.

STRONG ANSWER

I define each service, the app, database, and cache, in a `compose.yaml`, and `docker compose up` brings the whole stack online on a shared network where services reach each other by name. **depends_on** with a **healthcheck** controls startup order so the API waits for a ready database. **Volumes** persist database data and bind-mount source for hot reload, while **environment** variables wire up connection strings. I keep secrets in an `.env` file referenced by the compose file. This gives every developer an identical, reproducible environment in seconds.

CODE · YAML

```yaml
services:
  api:
    build: ./apps/api
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 5
volumes:
  pgdata:
```

ARCHITECTURE / FLOW

```
docker compose up
  +----------------------------------+
  |  default bridge network          |
  |   [api:3000] ---> [db:5432]       |
  |       |              |           |
  |   bind mount     pgdata volume    |
  |   (hot reload)   (persisted)      |
  +----------------------------------+
  services resolve each other by name
```

Q144 ARCHITECTURE, DOCKER & DEVOPS

Describe a CI/CD pipeline built with GitHub Actions.

THEORY

GitHub Actions runs workflows triggered by repo events, chaining jobs for lint, test, build, and deploy.

STRONG ANSWER

A workflow triggers **on push** or pull request and runs **jobs** made of steps on hosted runners. I separate a `test` job (install, lint, unit and integration tests) from a `deploy` job that **needs** it, so deploys only run on green builds against `main`. I use **dependency caching** to speed installs and **GitHub Secrets** for registry and SSH credentials. The deploy job builds and pushes a Docker image, then SSHes to the VPS to pull and restart. **Matrix** builds let me test across Node versions in parallel.

CODE · YAML

```yaml
name: ci-cd
on:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint && npm test
  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t registry/myapp:${{ github.sha }} .
      - run: docker push registry/myapp:${{ github.sha }}
        env:
          TOKEN: ${{ secrets.REGISTRY_TOKEN }}
```

ARCHITECTURE / FLOW

```
push to main
     |
     v
+----------+   needs   +-----------+
|  test    |---------->|  deploy   |
| lint     |  (green)  | build img |
| unit     |           | push      |
| integ    |           | ssh+pull  |
+----------+           +-----------+
 fails -> pipeline stops, no deploy
```

Q145 ARCHITECTURE, DOCKER & DEVOPS

How does Nginx work as a reverse proxy and load balancer?

THEORY

Nginx sits in front of app servers, forwarding client requests upstream and distributing load across backends.

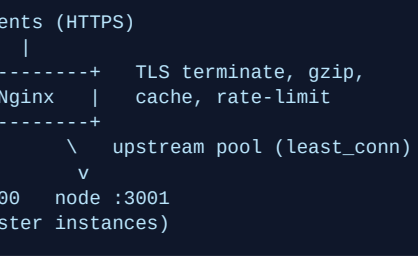
STRONG ANSWER

As a **reverse proxy**, Nginx terminates **TLS**, serves static assets, and forwards dynamic requests to app servers via `proxy_pass`, hiding the backend topology. As a **load balancer** it spreads traffic across an **upstream** pool using strategies like round-robin or least-connections, with **health checks** that drop failing nodes. It adds buffering, gzip, caching, and rate limiting at the edge, offloading work from Node. I also forward headers like `X-Forwarded-For` so the app sees the real client IP. This gives a single entry point with high availability and zero-downtime backend swaps.

CODE · Shell / Bash

```
upstream app {
    least_conn;
    server 127.0.0.1:3000;
    server 127.0.0.1:3001;
}

server {
    listen 443 ssl;
    server_name example.com;
    ssl_certificate     /etc/ssl/cert.pem;
    ssl_certificate_key /etc/ssl/key.pem;

    location / {
        proxy_pass http://app;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

ARCHITECTURE / FLOW



Q146 ARCHITECTURE, DOCKER & DEVOPS

How do you run Node in production with PM2 cluster mode?

THEORY

PM2 is a process manager that keeps Node apps alive, restarts on crashes, and scales across CPU cores via cluster mode.

STRONG ANSWER

PM2 in **cluster mode** forks one worker per CPU core using Node's cluster module, and the master round-robins incoming connections, so a single machine uses all cores. It provides **auto-restart** on crash, **zero-downtime reload** that recycles workers one at a time, and **log aggregation**. I declare everything in an **ecosystem file** for reproducible config and use `pm2 startup` plus `pm2 save` so processes survive reboots. Built-in metrics and memory-limit restarts help catch leaks. For containers I sometimes prefer one process per container, but on a bare VPS PM2 is ideal.

CODE · JavaScript

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'api',
    script: 'dist/server.js',
    instances: 'max',        // one worker per CPU core
    exec_mode: 'cluster',
    max_memory_restart: '500M',
    env: { NODE_ENV: 'production', PORT: 3000 }
  }]
};

// pm2 start ecosystem.config.js
// pm2 reload api   -> zero-downtime rolling restart
// pm2 startup && pm2 save  -> survive reboots
```

ARCHITECTURE / FLOW

```
        PM2 master (load balances)
        /     |      |      \
   worker  worker worker  worker
   :3000   :3000  :3000   :3000   (shared port)
   cpu0    cpu1   cpu2    cpu3

  pm2 reload -> restart workers one-by-one (no downtime)
```

Q147 ARCHITECTURE, DOCKER & DEVOPS

How do you manage environment config and secrets?

THEORY

Config should be externalized from code per the twelve-factor app, and secrets must never be committed to version control.

STRONG ANSWER

Following **twelve-factor** principles, I read config from **environment variables** so the same image runs in dev, staging, and prod with different values. Non-secret defaults live in a committed `.env.example`, while real `.env` files are **gitignored**. At startup I **validate** env vars with a schema (zod or envalid) so the app fails fast on misconfiguration. Real secrets go into a **secret manager** like GitHub Secrets, Doppler, or Vault, and are injected at deploy time, never baked into images. This keeps credentials out of the repo and rotatable without redeploying code.

CODE · TypeScript

```
import { z } from 'zod';

const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'production', 'test']),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
```

CODE · TypeScript (continued)

```
  JWT_SECRET: z.string().min(32),
});

// fail fast at boot if anything is missing/invalid
export const env = envSchema.parse(process.env);

// .env is gitignored; .env.example is committed
// secrets injected at deploy via CI secret store
```

ARCHITECTURE / FLOW

```
code (no secrets)
     |
     | reads process.env
     v
+------------------+   injected at deploy from:
|  validated env   | <-- GitHub Secrets / Vault / Doppler
+------------------+
 .env.example (committed)   .env (gitignored)
 fail fast if invalid
```

Q148 ARCHITECTURE, DOCKER & DEVOPS

Explain the testing pyramid with Jest, Vitest, and Playwright.

THEORY

The testing pyramid favors many fast unit tests at the base, fewer integration tests in the middle, and a small number of end-to-end tests at the top.

STRONG ANSWER

The base is **unit tests**, numerous, fast, isolated, run with **Vitest** or **Jest**, covering pure functions and use cases with mocked dependencies. The middle layer is **integration tests** that exercise several units together, such as a route hitting a real test database, catching wiring bugs unit tests miss. The thin top is **end-to-end tests** with **Playwright**, driving a real browser through critical user flows like login and checkout. E2E tests give the most confidence but are slow and brittle, so I keep them few and focused. This shape maximizes coverage while keeping the suite fast and stable.

CODE · TypeScript

```
// unit (Vitest) - fast, isolated
import { expect, test, vi } from 'vitest';
test('discount caps at 50%', () => {
  expect(applyDiscount(100, 0.9)).toBe(50);
});

// e2e (Playwright) - real browser, critical path
import { test, expect } from '@playwright/test';
test('user can log in', async ({ page }) => {
  await page.goto('/login');
  await page.fill('#email', 'a@b.com');
  await page.fill('#password', 'secret');
  await page.click('button[type=submit]');
  await expect(page).toHaveURL('/dashboard');
});
```

ARCHITECTURE / FLOW

```
          /\
         /e2e\        few, slow  (Playwright)
        /------\
       / integ  \     some       (Vitest/Jest + db)
      /----------\
     /   unit     \   many, fast (Vitest / Jest)
    /--------------\
  cost & runtime increase going up
```

Q149 ARCHITECTURE, DOCKER & DEVOPS

How do you handle monitoring, logging, and error tracking with Sentry?

THEORY

Observability combines structured logs, metrics, and error tracking so you can detect, diagnose, and alert on production issues.

STRONG ANSWER

I emit **structured JSON logs** (with pino or winston) carrying a **request/correlation ID** so a single request can be traced across services. For errors I integrate **Sentry**, which captures exceptions with **stack traces**, **breadcrumbs**, release version, and user context, then groups and alerts on them. **Source maps** are uploaded so minified stack traces map back to original code. Sentry's **performance tracing** surfaces slow transactions and N+1 queries, while metrics and uptime checks feed dashboards and alerts. Together these turn a vague "site is down" into an actionable, attributed report.

CODE · TypeScript

```typescript
import * as Sentry from '@sentry/node';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  release: process.env.GIT_SHA,
  tracesSampleRate: 0.1,
});

app.use(Sentry.Handlers.requestHandler());
app.use(Sentry.Handlers.errorHandler());

try {
  await chargeCustomer(order);
} catch (err) {
  Sentry.captureException(err, { tags: { orderId: order.id } });
  throw err;
}
```

ARCHITECTURE / FLOW

```
app ---> structured logs (pino) ---> log store / dashboard
  |
  +-----> Sentry (exceptions + traces)
  |          |
  |          +-> group, dedupe, alert (Slack/email)
  +-----> metrics + uptime ---> dashboards + alerts
 correlation id ties logs <-> errors <-> traces
```

Q150 ARCHITECTURE, DOCKER & DEVOPS

Compare blue-green vs rolling deploys and zero-downtime on a VPS.

THEORY

Both strategies aim for zero-downtime releases; blue-green swaps whole environments while rolling replaces instances incrementally.

STRONG ANSWER

In a **blue-green** deploy I run two identical environments; the new version goes live on the idle one, gets smoke-tested, then traffic is **switched** at the load balancer, giving an instant **rollback** by flipping back. A **rolling** deploy replaces instances a few at a time so old and new run side by side, using less capacity but requiring backward-compatible changes during the overlap. On a single **VPS** I get zero downtime by starting the new version on a fresh port, **health-checking** it, then reloading **Nginx** upstream or doing a **pm2 reload** that recycles workers one by one. The old process is drained and killed only after the new one is confirmed healthy.

CODE · Shell / Bash

```
#!/usr/bin/env bash
set -euo pipefail
# build & start new version on a fresh container/port
docker build -t myapp:$GIT_SHA .
docker run -d --name app-new -p 3001:3000 myapp:$GIT_SHA

# health-check before cutover
until curl -fs http://localhost:3001/health; do sleep 1; done

# flip nginx upstream to the new port, then reload
sed -i 's/3000/3001/' /etc/nginx/conf.d/app.conf
nginx -s reload

# drain & remove old version (instant rollback = flip back)
docker stop app-old && docker rm app-old
docker rename app-new app-old
```

ARCHITECTURE / FLOW

```
BLUE-GREEN                    ROLLING
[BLUE v1] <- live             [v1][v1][v1]
[GREEN v2]   test             [v2][v1][v1]  replace 1
   flip LB -->                [v2][v2][v1]  by 1
[GREEN v2] <- live            [v2][v2][v2]  done
rollback = flip back          rollback = redeploy v1
```